Tugas Besar 1

IF3070 Dasar Inteligensi Artifisial

Optimasi Penataan Muatan Kapal Kargo Menggunakan Algoritma Local Search

Disusun oleh:

Fathimah Nurhumaida Ramadhani - 18223052

Laras Hati Mahendra - 18223118

Tyara Penelope Lumban Gaol - 18224122



SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **2**
BAB I **3**
PENDAHULUAN & DESKRIPSI PERSOALAN **3**
1.1. Vessel Loading Problem (VLP) 3
1.2. Formulasi Local Search 3
BAB II **5**
PEMBAHASAN & IMPLEMENTASI **5**
2.1. Representasi Solusi & Inisialisasi 5
2.2. Objective Function (h(n)) & Constraints 6
2.3. Objective Function Bonus: ETA 9
2.4. Generate Class 9
2.5. Pembacaan Data 9
2.6. Implementasi Local Search 11
2.6.1. Steepest Ascent Hill-Climbing 11
2.5.2. Stochastic Ascent Hill-Climbing 13
2.5.3. Sideways Ascent Hill-Climbing 14
2.5.4. Random Restart Hill-Climbing 16
2.5.5. Simulated Annealing 17
2.5.6. Genetic Algorithm 20
2.7. Hasil Pengujian dan Analisis 27
2.7.1. Steepest Ascent Hill-Climbing 27
2.7.2. Stochastic Hill-Climbing Ascent Hill-Climbing 32
2.7.3. Hill-Climbing with Sideways Move 37
2.7.4. Random Restart Hill-Climbing 42
2.7.5. Simulated Annealing 47
a. Percobaan dengan Objective Function Shipping Fee 47
b. Percobaan dengan Objective Function ETA 53
2.7.6. Genetic Algorithm 59
BAB III **76**
ANALISIS **76**
BAB IV **79**
KESIMPULAN **79**

BAB I
PENDAHULUAN & DESKRIPSI PERSOALAN

1.1. Vessel Loading Problem (VLP)

Vessel Loading Problem (VLP) merupakan sebuah model optimasi yang difungsikan untuk menyelesaikan permasalahan penataan muatan pada geladak kapal kargo PT Owen Intim Line berbasis grid dua dimensi. Pemodelan ini mengelola penempatan sekumpulan kendaraan yang masing-masing dilengkapi dengan atribut spesifik, meliputi pengenal unik (ID), dimensi ukuran ($Width \times Length$) dalam satuan grid, pilihan orientasi posisi (horizontal atau vertikal), beban fisik (*Weight*), tarif keuntungan (*Shipping Fee*), serta estimasi waktu kedatangan (*Estimated Time of Arrival* / ETA). Kendaraan-kendaraan ini dialokasikan pada titik koordinat posisi tertentu (x, y) di kapal, atau dikategorikan sebagai *out of bounds* (tidak dimuat) apabila terdapat pelanggaran batasan.

Dalam keseharian, proses penataan kendaraan di atas geladak kapal kargo menghadapi berbagai kendala, seperti keterbatasan ruang dan kapasitas muatan. Kebutuhan untuk memperhitungkan atribut fisik kendaraan mengharuskan adanya susunan penempatan yang teratur agar setiap kendaraan yang masuk bisa tertata secara aman dan efisien. Proses pemilihan keputusan apakah suatu kendaraan layak diangkut pada koordinat tertentu atau diabaikan menjadi hal penting agar seluruh proses pengangkutan di pelabuhan berjalan lancar sekaligus menjaga keselamatan pelayaran.

Tujuan utama dari formulasi ini adalah mengoptimalkan *Shipping Fee* total dari seluruh kendaraan yang dapat dimuat ke atas geladak kapal dengan meninjau *constraints*. Susunan pencarian wajib memenuhi batasan *No Overlapping* agar tidak ada dua kendaraan atau lebih yang menempati sel grid geladak yang sama, serta aturan *In-Boundary* yang mengharuskan seluruh sel *grid* kendaraan terisi penuh di dalam ukuran area kapal. Selain itu, *Max Capacity* juga membatasi berat total kendaraan agar nilainya tidak melebihi kapasitas beban maksimum kapal.

1.2. Formulasi *Local Search*

Local Search merupakan pendekatan optimalisasi heuristik yang bekerja dengan cara menelusuri ruang keadaan (*state space*) secara berulang, dimulai dari *initial state* lalu berpindah ke *neighbor state* yang memiliki nilai evaluasi lebih baik atau memenuhi syarat tertentu. Pendekatan ini diterapkan dalam menjawab *Vessel Loading Problem* karena permasalahan penataan geladak kapal ini termasuk kategori *NP-Hard*, di mana pencarian solusi secara *exhaustive search* membutuhkan waktu yang tidak efisien. Ada tiga algoritma utama berbasis *Local Search,* yaitu *Hill Climbing*, *Simulated Annealing*, dan *Genetic Algorithm*.

Seperti yang ditunjukkan pada gambar, proses pencarian solusi bekerja dengan meneruskan kendaraan dari status daftar muatan (*Initial Objects*) menuju posisi spesifik pada *grid* geladak kapal (*Bin*) hingga mencapai nilai *Objective Function* yang optimal. Proses penelusuran solusi dilakukan dengan menggali *state space* melalui tiga jenis proses pembentukan *Neighbor State*:

1. *Move* untuk memindahkan posisi kendaraan ke koordinat (x, y) yang berbeda di dalam atau luar geladak.
2. *Swap* untuk menukar posisi dan status antara dua kendaraan.
3. *Rotate* untuk mengubah orientasi kendaraan (horizontal dan vertikal) sehingga dimensi panjang dan lebar saling berkebalikan pada grid.

BAB II
PEMBAHASAN & IMPLEMENTASI

2.1. Representasi Solusi & Inisialisasi

Kapal dimodelkan sebagai grid 2D dengan lebar *width* dan panjang *length*. Titik (0,0) ada di pojok kiri bawah, sumbu x bertambah ke kanan dan sumbu y bertambah ke atas. Satu *vehicle* menempati kotak mulai dari *(x, y)* sampai *(x + width, y + length)*.

Satu *state* adalah list *Vehicle*. Tiap *Vehicle* menyimpan id, dimensi, orientasi, shipping_fee, *weight*, ETA, koordinat (x, y), dan is_loaded. Kendaraan dengan is_loaded = True dihitung terangkut, sedangkan yang False diletakkan di luar kapal dan tidak ikut dihitung. Kapal disimpan di kelas *Ship* yang berisi dimensi dan maxCapacity.

Initial state dibuat oleh init_state. Tiap kendaraan di-*random* dengan peluang 50% untuk dimuat. Kendaraan yang dimuat diberi posisi acak di dalam kapal, dicoba maksimal 100 kali (agar tidak infinite loop) sampai tidak tumpang tindih dan tidak melebihi kapasitas, sedangkan yang tidak dimuat diberi posisi acak di luar kapal.

```
def init_state(vehicles: list[Vehicle], ship: Ship) -> list[Vehicle]:
    state = copy.deepcopy(vehicles)
    for v in state:
        v.is_loaded = random.random() < 0.5

        if v.is_loaded:
            for _ in range(100):
                v.x = random.randint(0, ship.dimension.width - v.dimension.width)
                v.y = random.randint(0, ship.dimension.length - v.dimension.length)
                if not_overlapping(state, ship) and not exceed_capacity_limit(state,
ship):
                    break

        if not v.is_loaded:
            v.is_loaded = True
            while True:
                v.x = random.randint(-ship.dimension.width, ship.dimension.width)
                v.y = random.randint(-ship.dimension.length, ship.dimension.length)
                if exceed_ship_limit([v], ship):
                    break
            v.is_loaded = False

    return state
```

Modul main.py digunakan untuk pemangilan seluruh fungsi, agar bisa menjalankan program dari folder src dengan *command* “python main.py”.

```
def main():
    while True:
        print("\nPilih algoritma yang ingin dijalankan:\n")
        print("1. Steepest Ascent Hill Climbing")
        print("2. Stochastic Hill Climbing")
        print("3. Sideways Move Hill Climbing")
        print("4. Random Restart Hill Climbing")
        print("5. Simulated Annealing")
        print("6. Genetic Algorithm")
        print("0. Keluar")
        pilihan = input("Masukkan pilihan (0-6): ").strip()

        if pilihan == "0":
            break
        elif pilihan == "1":
            main_hill_climbing.main_steepest_ascent()
        elif pilihan == "2":
            main_hill_climbing.main_stochastic()
        elif pilihan == "3":
            main_hill_climbing.main_sideways_move()
        elif pilihan == "4":
            main_hill_climbing.main_random_restart()
        elif pilihan == "5":
            main_simulated_annealing.main_simulated_annealing()
        elif pilihan == "6":
            ga_run_experiments.main()
        else:
            print("Pilihan tidak valid.")


if __name__ == "__main__":
    main()
```

2.2. Objective Function (h(n)) & Constraints

Objective function memaksimalkan total *shipping fee* kendaraan yang terangkut. Fungsi objective_function mengecek tiga kendala terlebih dulu. Kalau salah satu dilanggar, nilainya 0. Kalau semua lolos, nilainya adalah total *shipping fee* kendaraan yang dimuat.

Constraint pertama adalah Max Capacity, dicek oleh exceed_capacity_limit yaitu total berat kendaraan yang dimuat tidak boleh melebihi maxCapacity. *Constraint* kedua adalah batas-batas kapal, dicek oleh exceed_ship_limit yaitu seluruh kendaraan yang dimuat harus ada di dalam kapal. *Constraint* ketiga adalah *overlap*, dicek oleh not_overlapping yaitu tiap pasang kendaraan yang dimuat dibandingkan, dan keduanya dianggap aman kalau terpisah di sumbu x atau terpisah di sumbu y. Kalau tidak terpisah di keduanya, berarti bertumpuk.

```
def shipping_fee(vehicles: list[Vehicle]) -> int:
    return sum(vehicle.shippingFee for vehicle in vehicles if vehicle.is_loaded)
```

```
def not_overlapping(vehicles: list[Vehicle], ship: Ship) -> bool:
    vehicle_in_ship = []
    for vehicle in vehicles:
        if vehicle.is_loaded:
            vehicle_in_ship.append(vehicle)

    jumlah_vehicle = len(vehicle_in_ship)
    for i in range(jumlah_vehicle):
        vehicle_a = vehicle_in_ship[i]

        for j in range (i+1,jumlah_vehicle):
            vehicle_b = vehicle_in_ship[j]

            a_right = vehicle_a.x + vehicle_a.dimension.width
            a_below = vehicle_a.y + vehicle_a.dimension.length

            b_right = vehicle_b.x + vehicle_b.dimension.width
            b_below = vehicle_b.y + vehicle_b.dimension.length

            aman_x = (a_right <= vehicle_b.x) or (vehicle_a.x >= b_right)
            aman_y = (a_below <= vehicle_b.y) or (vehicle_a.y >= b_below)

            if not (aman_x or aman_y):
                return False

    return True


def exceed_ship_limit(vehicles: list[Vehicle], ship:Ship) -> bool:
    for vehicle in vehicles:
        if not vehicle.is_loaded:
            continue

        left = vehicle.x
        above = vehicle.y
        right = vehicle.x + vehicle.dimension.width
        below = vehicle.y + vehicle.dimension.length

        if left < 0 or above < 0 or right > ship.dimension.width or below >
ship.dimension.length:
            return True

    return False


def exceed_capacity_limit(vehicles: list[Vehicle], ship: Ship) -> bool:
    total_weight = sum(v.weight for v in vehicles if v.is_loaded)
    if total_weight > ship.maxCapacity:
        return True
    else:
        return False


def objective_function(vehicles: list[Vehicle], ship: Ship) -> int:
    if exceed_capacity_limit(vehicles,ship) == True:
        return 0
    if exceed_ship_limit(vehicles,ship) == True:
        return 0
    if not_overlapping(vehicles,ship)==False:
        return 0
    total_uang = shipping_fee(vehicles)
    return total_uang
```

Untuk membentuk *neighbor* ada tiga operator, yaitu *move*, *swap*, dan *rotate*. Ketiganya sama-sama memakai is_loaded, lalu membatalkan perubahan (abort) kalau hasilnya tumpang

tindih atau melebihi kapasitas. *Move* memindahkan satu kendaraan ke koordinat acak, di dalam atau di luar kapal, dan kendaraan jadi tidak dimuat kalau posisi barunya tidak utuh di dalam kapal. *Swap* menukar koordinat dua kendaraan, lalu menentukan ulang status is_loaded keduanya. *Rotate* membalik orientasi kendaraan dan menukar lebar dengan panjang, namun posisinya tetap.

```
def swap(vehicles: list[Vehicle], ship: Ship):
    vehicle_a, vehicle_b = random.sample(vehicles, 2)
    old_a = (vehicle_a.x, vehicle_a.y, vehicle_a.is_loaded)
    old_b = (vehicle_b.x, vehicle_b.y, vehicle_b.is_loaded)

    vehicle_a.x, vehicle_b.x = vehicle_b.x, vehicle_a.x
    vehicle_a.y, vehicle_b.y = vehicle_b.y, vehicle_a.y

    for v in (vehicle_a, vehicle_b):
        v.is_loaded = True
        if exceed_ship_limit([v], ship):
            v.is_loaded = False

    if not not_overlapping(vehicles, ship) or exceed_capacity_limit(vehicles, ship):
        vehicle_a.x, vehicle_a.y, vehicle_a.is_loaded = old_a
        vehicle_b.x, vehicle_b.y, vehicle_b.is_loaded = old_b


def move(vehicles: list[Vehicle], ship: Ship):
    vehicle = random.choice(vehicles)
    old = (vehicle.x, vehicle.y, vehicle.is_loaded)

    vehicle.x = random.randint(-ship.dimension.width, ship.dimension.width)
    vehicle.y = random.randint(-ship.dimension.length, ship.dimension.length)

    vehicle.is_loaded = True
    if exceed_ship_limit([vehicle], ship):
        vehicle.is_loaded = False

    if not not_overlapping(vehicles, ship) or exceed_capacity_limit(vehicles, ship):
        vehicle.x, vehicle.y, vehicle.is_loaded = old #abort


def rotate(vehicles: list[Vehicle], ship: Ship):
    vehicle = random.choice(vehicles)
    old_orientation = vehicle.orientation
    old_width, old_length = vehicle.dimension.width, vehicle.dimension.length
    old_loaded = vehicle.is_loaded

    if vehicle.orientation == "Horizontal":
        vehicle.orientation = "Vertical"
    else:
        vehicle.orientation = "Horizontal"

    vehicle.dimension.width, vehicle.dimension.length = vehicle.dimension.length,
vehicle.dimension.width

    vehicle.is_loaded = True
    if exceed_ship_limit([vehicle], ship):
        vehicle.is_loaded = False

    if not not_overlapping(vehicles, ship) or exceed_capacity_limit(vehicles, ship):
        vehicle.orientation = old_orientation
        vehicle.dimension.width, vehicle.dimension.length = old_width, old_length
        vehicle.is_loaded = old_loaded
```

2.3. Objective Function Bonus: ETA

Pada objective function bonus, bertujuan untuk me-*load vehicle* sebanyak mungkin ke dalam kapal dengan total ETA (estimated time of arrival) paling singkat. Karena total jumlah *vehicle* dan total ETA berbanding terbalik, nilai/*score* yang digunakan dihitung melalui fungsi berikut:

```
def objective_function_bonus(vehicles: list[Vehicle], ship: Ship) -> int:
    if exceed_capacity_limit(vehicles, ship):
        return 0
    if exceed_ship_limit(vehicles, ship):
        return 0
    if not not_overlapping(vehicles, ship):
        return 0

    jumlah_loaded = sum(v.is_loaded for v in vehicles)
    total_eta = sum(v.eta for v in vehicles if v.is_loaded)
    return jumlah_loaded * sum(v.eta for v in vehicles) - total_eta
```

2.4.*Generate Class*

Terdapat tiga kelas yang dipakai. *Vehicle* menyimpan atribut kendaraan, dengan is_loaded yang awalnya False dan koordinat x, y yang awalnya 0. *Ship* menyimpan dimensi kapal dan kapasitas maksimum. *Dimension* menyimpan lebar dan panjang, dan dipakai oleh *Vehicle* maupun *Ship*.

```
class Vehicle:
    def __init__(self, id: str, width: int, length: int, orientation: str,
shippingFee: int, weight: int, eta: int):
        self.id = id
        self.dimension = Dimension(width,length)
        self.orientation = orientation
        self.shippingFee = shippingFee
        self.weight = weight
        self.eta = eta
        self.is_loaded = False #awalnya ga di kapal semua

        self.x = 0
        self.y = 0


class Ship:
    def __init__(self, width: int, length: int, maxCapacity: int):
        self.dimension = Dimension(width,length)
        self.maxCapacity = maxCapacity


class Dimension:
    def __init__(self, width: int, length: int):
        self.width = width
        self.length = length
```

2.5. Pembacaan *Data*

Fungsi load_json membaca file JSON. Bagian *ship* dijadikan objek *Ship*, lalu tiap elemen *vehicles* dijadikan objek *Vehicle*, dan fungsi mengembalikan (*vehicles, ship*). Data disimpan di vehicles.json.

```
def load_json(path: str):
    with open(path) as file:
        data = json.load(file)
    s = data["ship"]
    ship = Ship(s["width"], s["length"], s["maxCapacity"])
    vehicles = []
    for v in data["vehicles"]:
        vehicles.append(Vehicle(v["id"], v["width"], v["length"], v["orientation"],
v["shippingFee"], v["weight"], v["eta"]))

    return vehicles, ship
```

vehicle.json

```
{
  "ship": {"width": 10, "length": 20, "maxCapacity": 400},
  "vehicles": [
    {"id": "Owen",        "width": 1, "length": 4, "orientation": "Horizontal",
"shippingFee": 118, "weight": 27, "eta": 4},
    {"id": "Kucing",      "width": 3, "length": 4, "orientation": "Vertical",
"shippingFee": 312, "weight": 64, "eta": 10},
    {"id": "Fathy",       "width": 1, "length": 2, "orientation": "Horizontal",
"shippingFee": 26, "weight": 8, "eta": 8},
    {"id": "Laras",       "width": 1, "length": 5, "orientation": "Vertical",
"shippingFee": 143, "weight": 20, "eta": 5},
    {"id": "Tyara",       "width": 1, "length": 5, "orientation": "Horizontal",
"shippingFee": 118, "weight": 22, "eta": 6},
    {"id": "Informatika", "width": 1, "length": 4, "orientation": "Vertical",
"shippingFee": 84, "weight": 13, "eta": 1},
    {"id": "Bumblebee",   "width": 2, "length": 3, "orientation": "Vertical",
"shippingFee": 165, "weight": 18, "eta": 2},
    {"id": "Pohon",       "width": 1, "length": 5, "orientation": "Vertical",
"shippingFee": 215, "weight": 34, "eta": 9},
    {"id": "AyamGeprek",  "width": 1, "length": 3, "orientation": "Horizontal",
"shippingFee": 136, "weight": 15, "eta": 10},
    {"id": "Cilok",       "width": 1, "length": 2, "orientation": "Horizontal",
"shippingFee": 24, "weight": 12, "eta": 10},
    {"id": "DosenKiller", "width": 1, "length": 2, "orientation": "Horizontal",
"shippingFee": 87, "weight": 13, "eta": 8},
    {"id": "EsTeh",       "width": 2, "length": 4, "orientation": "Vertical",
"shippingFee": 372, "weight": 59, "eta": 9},
    {"id": "Gorengan",    "width": 1, "length": 5, "orientation": "Vertical",
"shippingFee": 154, "weight": 29, "eta": 7},
    {"id": "HETI",        "width": 1, "length": 3, "orientation": "Vertical",
"shippingFee": 61, "weight": 8, "eta": 9},
    {"id": "Jom",         "width": 3, "length": 3, "orientation": "Vertical",
"shippingFee": 158, "weight": 54, "eta": 9},
    {"id": "MieAyam",     "width": 2, "length": 3, "orientation": "Horizontal",
"shippingFee": 159, "weight": 35, "eta": 5},
    {"id": "NasiPadang",  "width": 1, "length": 5, "orientation": "Vertical",
"shippingFee": 69, "weight": 30, "eta": 8},
    {"id": "QRIS",        "width": 3, "length": 2, "orientation": "Horizontal",
"shippingFee": 157, "weight": 37, "eta": 5},
    {"id": "RotiBakar",   "width": 3, "length": 3, "orientation": "Horizontal",
"shippingFee": 157, "weight": 61, "eta": 8},
    {"id": "Seblak",      "width": 3, "length": 2, "orientation": "Horizontal",
"shippingFee": 163, "weight": 31, "eta": 10},
    {"id": "UburUbur",    "width": 2, "length": 3, "orientation": "Vertical",
"shippingFee": 202, "weight": 38, "eta": 6},
    {"id": "Vox",         "width": 1, "length": 3, "orientation": "Vertical",
"shippingFee": 96, "weight": 19, "eta": 3},
    {"id": "Wibu",        "width": 2, "length": 3, "orientation": "Vertical",
"shippingFee": 90, "weight": 33, "eta": 2},
    {"id": "Xixixi",      "width": 1, "length": 2, "orientation": "Vertical",
"shippingFee": 30, "weight": 13, "eta": 9},
    {"id": "Yakult",      "width": 1, "length": 5, "orientation": "Horizontal",
"shippingFee": 149, "weight": 17, "eta": 2},
```

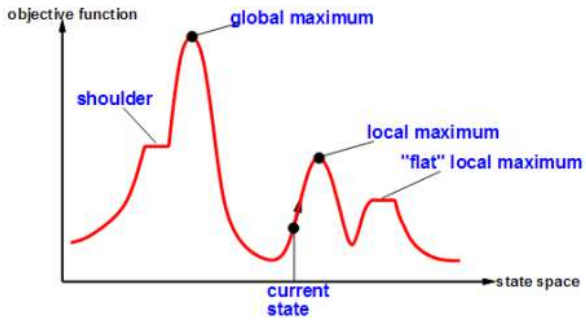
```
    {"id": "Zonk pls dai A",  "width": 1, "length": 2, "orientation": "Horizontal",
"shippingFee": 83, "weight": 11, "eta": 1}
  ]
}
```

2.6. Implementasi *Local Search*

2.6.1. Steepest Ascent Hill-Climbing

a. Deskripsi Umum:

Steepest Ascent Hill-Climbing merupakan varian algoritma yang mengevaluasi seluruh *neighbor* dari *current state* untuk memilih nilai tertinggi. Berbeda dengan *Simple Hill-Climbing* yang langsung memilih neighbor pertama yang lebih baik, algoritma ini hanya berpindah ke neighbor terbaik apabila nilainya lebih tinggi daripada current state.



b. Proses Pencarian dalam Vessel Loading Problem

Ada beberapa tahapan pencarian algoritma Steepest Ascent Hill-Climbing:

1. Menginisialisasi konfigurasi kendaraan (initial_vehicles) dan menghitung skor awal (current_score).
2. Membangkitkan seluruh neighbor menggunakan get_all_neighbors dan menghitung skor masing-masing.
3. Membandingkan seluruh skor neighbor untuk menentukan kandidat terbaik (max_score).
4. Memperbarui current state jika max_score lebih tinggi daripada current_score.
5. Mengulangi pencarian hingga tidak ada neighbor yang lebih baik atau batas iterasi tercapai.

c. Fungsi/Metode steepest_ascent_hill_climbing

Inisalisasi
`def steepest_ascent_hill_climbing(initial_vehicles, ship, max_iterations=3000,`

```
num_candidates=None, tampil=False):
    current_vehicles = copy.deepcopy(initial_vehicles)
    current_score = objective_function(current_vehicles, ship)
    history_score = [current_score]
    total_iterations = 0
    status = "Local Optima"

    if tampil:
        print(f"[0] State Awal. Skor: {current_score}")
```

Bagian awal yang memiliki fungsi menginisialisasi state awal (current_vehicles), nilai fungsi objektif (current_score), riwayat skor (history_score), jumlah iterasi (total_iterations), dan status penghentian (status). Variabel-variabel ini digunakan untuk memantau dan memperbarui pencarian pada setiap iterasi.

Proses Pencarian Utama

```
while total_iterations < max_iterations:
        total_iterations += 1
        better_neighbors = []
        if num_candidates is None or num_candidates == "all":
            for candidate in get_all_neighbors(current_vehicles, ship):
                score = objective_function(candidate, ship)
                if score > current_score:
                    better_neighbors.append((candidate, score))
        else:
            for _ in range(num_candidates):
                candidate = get_random_neighbor(current_vehicles, ship)
                score = objective_function(candidate, ship)
                if score > current_score:
                    better_neighbors.append((candidate, score))
        if not better_neighbors:
            status = "Local Optima"
            break
        max_score = max(item[1] for item in better_neighbors)
        best_candidates = [item for item in better_neighbors if item[1] ==
max_score]
        best_candidate, best_candidate_score = random.choice(best_candidates)
        current_vehicles, current_score = copy.deepcopy(best_candidate),
best_candidate_score
        history_score.append(current_score)
        if tampil:
            print(f"[{total_iterations}] Pindah. Skor Baru: {current_score}")
    if total_iterations >= max_iterations and status != "Local Optima":
        status = "Max Iteration Reached"
    return SearchResult(current_vehicles, current_score, total_iterations,
history_score, status=status)
```

Proses pencarian dilakukan dengan membangkitkan seluruh neighbor melalui get_all_neighbors atau mengambil sampel acak melalui get_random_neighbor sesuai parameter num_candidates. Setiap kandidat akan dinilai menggunakan objective_function, kemudian kandidat dengan skor lebih tinggi disimpan dalam better_neighbors. Kandidat dengan skor tertinggi (max_score) dipilih sebagai current state dan *summary*. Jika tidak ditemukan kandidat yang lebih baik, pencarian berhenti dengan status "Local Optima".

2.5.2. Stochastic Ascent Hill-Climbing

a. Deskripsi Umum:

Stochastic Hill-Climbing merupakan metode yang bekerja dengan mengevaluasi sejumlah kandidat neighbor melalui sampling acak berdasarkan parameter num_candidates. Algoritma ini memilih secara acak salah satu kandidat dengan skor lebih tinggi daripada current state. Proses pencarian berulang hingga tidak ditemukan kandidat yang lebih baik atau batas iterasi tercapai.

b. Proses Pencarian dalam Vessel Loading Problem

Proses pencarian Stochastic Hill-Climbing dimulai dengan inisialisasi konfigurasi kendaraan dan menghitung skor awal. Selanjutnya, algoritma membangkitkan neighbor acak dan mengevaluasi skor masing-masing. Kandidat dengan skor lebih tinggi dikumpulkan, kemudian salah satunya dipilih secara acak sebagai current state baru. Proses ini berulang hingga tidak ditemukan kandidat yang lebih baik .

c. Fungsi/Metode steepest_ascent_hill_climbing

Inisalisasi

```
def steepest_ascent_hill_climbing(initial_vehicles, ship, max_iterations=3000,
num_candidates=None, tampil=False):
    current_vehicles = copy.deepcopy(initial_vehicles)
    current_score = objective_function(current_vehicles, ship)
    history_score = [current_score]
    total_iterations = 0
    status = "Local Optima"

    if tampil:
        print(f"[0] State Awal. Skor: {current_score}")
```

Bagian ini dipakai untuk menyiapkan state awal, skor, riwayat pencarian, batas iterasi, jumlah sampel neighbor, dan status penghentian (local optima) untuk menjalankan algoritma Stochastic Hill-Climbing.

Proses Pencarian Utama

```
  while total_iterations < max_iterations:
        total_iterations += 1
        better_neighbors = []
        # Sampling kandidat tetangga acak
        for _ in range(num_candidates):
            candidate = get_random_neighbor(current_vehicles, ship)
            candidate_score = objective_function(candidate, ship)
            if candidate_score > current_score:
                better_neighbors.append((candidate, candidate_score))
        # Berhenti saat tidak ada tetangga yang lebih baik (local optima)
        if not better_neighbors:
            status = "Local Optima"
            break
        # Pilih 1 tetangga secara acak dari kumpulan tetangga yang lebih baik
```

```
        selected_neighbor, selected_score = random.choice(better_neighbors)
        current_vehicles, current_score = copy.deepcopy(selected_neighbor),
selected_score
        history_score.append(current_score)
        if tampil:
            print(f"[{total_iterations}] Pindah. Skor Baru: {current_score}")
    if total_iterations >= max_iterations and status != "Local Optima":
        status = "Max Iteration Reached"
    return SearchResult(current_vehicles, current_score, total_iterations,
history_score, status=status)
```

Proses pencarian Stochastic Hill-Climbing dilakukan dengan membangkitkan beberapa neighbor secara acak dan mengevaluasi skornya menggunakan objective_function. Kandidat dengan skor lebih tinggi disimpan dalam better_neighbors, dann salah satunya dipilih secara acak untuk memperbarui current state dan riwayat skor. Pencarian akan berhenti setelah tidak ditemukan kandidat yang lebih baik ("Local Optima") atau batas iterasi tercapai ("Max Iteration Reached"). Hasil akhirnya dikembalikan melalui SearchResult.

2.5.3. Sideways Ascent Hill-Climbing

A. Deskripsi Umum:

Algoritma Hill-Climbing with Sideways Move merupakan salah satu varian Hill-Climbing yang dirancang untuk mengatasi kondisi plateau, yaitu ketika sejumlah neighbor memiliki nilai fungsi objektif yang sama dengan current state. Algoritma ini mengizinkan perpindahan ke neighbor dengan skor setara, tetapi dibatasi oleh parameter max_sideways_move untuk mencegah perulangan tanpa henti. Ketika skor meningkat, sideways_count diatur ulang menjadi nol.

B. Proses Pencarian dalam *Vessel Loading Problem*

Prosesnya diawali dengan menginisialisasi konfigurasi kendaraan dan jumlah langkah menyamping (`sideways_count`). Selanjutnya, algoritma membangkitkan sejumlah neighbor secara acak dan membandingkan skor masing-masing dengan current state. Kandidat dengan skor tertinggi dipilih sebagai current state baru, baik melalui peningkatan skor maupun perpindahan menyamping jika skornya sama. Proses ini berulang hingga tidak ditemukan kandidat yang lebih baik atau setara.

C. Proses Pencarian dalam Vessel Loading Problem

Inisalisasi

```
def  sideways_move_hill_climbing(initial_vehicles,  ship,  max_iterations=3000,
max_sideways_move=100, num_candidates=10, tampil=False):
    current_vehicles = copy.deepcopy(initial_vehicles)
    current_score = objective_function(current_vehicles, ship)
    history_score = [current_score]
```

```
    total_iterations, sideways_count, total_sideways_executed = 0, 0, 0
    status = "Local Optima"

    if tampil: print(f"[0] State Awal. Skor: {current_score}")
```

Bagian inisialisasi berfungsi untuk menentukan konfigurasi awal kendaraan, menghitung skor awal, dan mencatat riwayat pencarian. Selain itu, ditetapkan jumlah iterasi maksimum (max_iterations=3000), jumlah sampel neighbor (num_candidates=10), serta jumlah maksimum langkah menyamping berturut-turut (max_sideways_move=100). Variabel lainnya dipakai untuk menghitung iterasi, mencatat langkah menyamping, dan menentukan status penghentian algoritma.

Proses Pencarian Utama

```
while total_iterations < max_iterations:
        total_iterations += 1
        eligible_neighbors = []
        for _ in range(num_candidates):
            candidate = get_random_neighbor(current_vehicles, ship)
            candidate_score = objective_function(candidate, ship)
            if candidate_score >= current_score:
                eligible_neighbors.append((candidate, candidate_score))
        if not eligible_neighbors:
            status = "Local Optima"
            break
        max_score = max(item[1] for item in eligible_neighbors)
        best_candidates = [item for item in eligible_neighbors if item[1] ==
max_score]
        selected_neighbor, selected_score = random.choice(best_candidates)
        if selected_score > current_score:
            sideways_count = 0
            current_vehicles = copy.deepcopy(selected_neighbor)
            current_score = selected_score
            if tampil: print(f"[{total_iterations}] Naik Skor. Skor Baru:
{current_score}")
        else:
            sideways_count += 1
            total_sideways_executed += 1

            if sideways_count >= max_sideways_move:
                status = "Max Sideways Reached"
                break

            current_vehicles = copy.deepcopy(selected_neighbor)
            current_score = selected_score
            if tampil: print(f"[{total_iterations}] Pindah Samping
(Sideways). Skor: {current_score}")

        history_score.append(current_score)

    if total_iterations >= max_iterations and status == "Local Optima":
        status = "Max Iteration Reached"

    return SearchResult(current_vehicles, current_score, total_iterations,
history_score, status=status, sideways_moves=total_sideways_executed)
```

Proses pencarian dilakukan dengan membangkitkan beberapa neighbor secara acak dan menyaring kandidat yang memiliki skor lebih tinggi atau sama dengan current state ke dalam eligible_neighbors. Kandidat dengan skor tertinggi dipilih untuk memperbarui state. Jika skor meningkat, sideways_count diatur ulang menjadi 0, sedangkan jika skor tetap, jumlah langkah menyamping bertambah. Pencarian berhenti ketika tidak ditemukan kandidat yang memenuhi syarat ("Local Optima"), jumlah maksimum langkah menyamping tercapai ("Max Sideways Reached"), atau batas iterasi terpenuhi.

2.5.4. Random Restart Hill-Climbing

A. Deskripsi Umum

Algoritma *Random Restart Hill-Climbing* merupakan varian Hill-Climbing yang dirancang untuk mengurangi risiko terjebak pada local maxima dan plateau dengan mengulangi pencarian dari konfigurasi awal yang dibangkitkan secara acak. Setiap pengulangan (restart) menghasilkan solusi yang dibandingkan untuk menentukan solusi terbaik secara keseluruhan (global best). Proses ini berlangsung hingga mencapai batas max_restart.

B. Proses Pencarian dalam *Vessel Loading Problem*

Prosesnya diawali dengan menentukan jumlah maksimum restart dan invariabel penyimpan solusi terbaik. Pada setiap restart, algoritma menjalankan Stochastic Hill-Climbing menggunakan konfigurasi awal yang diberikan atau dibangkitkan secara acak. Hasil setiap pencarian kemudian dibandingkan untuk mencari solusi terbaik secara keseluruhan. Proses ini berulang hingga mencapai batas limit_restarts, kemudian algoritma mengembalikan solusi terbaik dari seluruh percobaan.

a. **Fungsi**

Inisialisasi

```
def random_restart_hill_climbing(
    ship,
    max_restart=10,
    max_restarts=None,
    max_iterations_per_restart=1000,
    initial_vehicles=None,
    num_candidates=10,
    tampil=False,
):
    limit_restarts = max_restarts if max_restarts is not None else
max_restart
    if isinstance(ship, list) and hasattr(limit_restarts, "maxCapacity"):
        initial_vehicles, ship, limit_restarts = ship, limit_restarts,
max_iterations_per_restart
```

```
    has_custom_initial = initial_vehicles is not None
    if initial_vehicles is None:
        initial_vehicles = bikin_kendaraan_awal()
    global_best_vehicles = None
    global_best_score = -1
    iterations_per_restart = []
    cumulative_history = []
    initial_score = None
    first_initial_state = None
```

Tahap ini meliputi limit_restarts, maksimal iterasi per siklus (max_iterations_per_restart), jumlah kandidat (num_candidates), dan ketersediaan *state* awal kustom. Selain itu, dipersiapkan juga variabel pelacak pencarian lintas restart, seperti solusi dan skor terbaik global, riwayat iterasi maupun skor kumulatif, serta informasi *state* awal pertama. Seluruh konfigurasi dasar ini akan menjadi landasan eksekusi pada blok perulangan restart selanjutnya.

Proses Pencarian Utama

```
  for restart_idx in range(1, limit_restarts + 1):
        if restart_idx == 1 and has_custom_initial:
            start_state = copy.deepcopy(initial_vehicles)
        else:
            start_state = bikin_state_acak(initial_vehicles, ship)
        if first_initial_state is None:
            first_initial_state = copy.deepcopy(start_state)
        best_v, best_s, iters, hist = stochastic_hill_climbing(
            start_state, ship,
            max_iterations=max_iterations_per_restart,
            num_candidates=num_candidates,
            tampil=False,
        )
        if initial_score is None and hist:
            initial_score = hist[0]
        iterations_per_restart.append(iters)
        if best_s > global_best_score or global_best_vehicles is None:
            global_best_vehicles = copy.deepcopy(best_v)
            global_best_score = best_s
        for s in hist:
            if s > global_best_score or global_best_score == -1:
                global_best_score = s
            cumulative_history.append(global_best_score)
        if tampil:
            print(f"[{restart_idx}] Restart. Skor Baru: {global_best_score}")
    return RandomRestartResult(
        global_best_vehicles,
        global_best_score,
        len(iterations_per_restart),
        iterations_per_restart,
        cumulative_history,
        initial_score,
        status="Max Restart Reached",
        initial_state=first_initial_state,
    )
```

Proses ini menjalankan siklus pencarian lokal secara berulang menggunakan *state* awal kustom maupun acak untuk terus mengevaluasi dan memperbarui solusi terbaik global,

sebelum mengembalikan objek RandomRestartResult yang merangkum seluruh metrik pelacakan serta hasil akhir pencarian.

2.5.5. Simulated Annealing

Simulated Annealing (SA) membangkitkan satu *neighbor*, dan boleh pindah ke *neighbor* yang lebih buruk. Peluangnya ditentukan oleh suhu T. Rumusnya adalah $e^{\frac{\Delta E}{T}}$,dimana jika suhu tinggi, langkah buruk lebih sering diterima. Kalau suhu turun, SA makin selektif dan jika suhu sudah melewati batas dingin maka tidak boleh pindah ke *neighbor* yang lebih buruk. Maka dari itu, SA punya peluang keluar dari *local maximum*.

```
def simulated_annealing(initial_state, ship, max_iter=1000, init_temp=30,
cooling_rate=0.99, stuck_threshold=10):
    start_time = time.perf_counter()
    current_state = copy.deepcopy(initial_state)
    current_score = utils.objective_function(current_state, ship)
    best_state = copy.deepcopy(current_state)
    best_score = current_score
    temperature = init_temp

    objective_function_value = [current_score]
    acceptance_probabilities = []
    stuck_count = 0
    no_improvement = 0

    for i in range(max_iter):
        neighbor_state = get_neighbor(current_state, ship)
        neighbor_score = utils.objective_function(neighbor_state, ship)

        delta = neighbor_score - current_score

        if delta > 0:
            current_state = neighbor_state
            current_score = neighbor_score
            acceptance_probabilities.append(1.0)

        else:
            probability = random.random()
            acceptance_prob = math.exp(delta / temperature)
            acceptance_probabilities.append(acceptance_prob)
            if probability < acceptance_prob:
                current_state = neighbor_state
                current_score = neighbor_score

        if current_score > best_score:
            best_state = copy.deepcopy(current_state)
            best_score = current_score

        objective_function_value.append(current_score)

        if current_score == objective_function_value[-2]:
            no_improvement += 1
        else:
            no_improvement = 0

        if no_improvement == stuck_threshold:
            stuck_count += 1

        temperature = temperature * cooling_rate
```

```
    duration = time.perf_counter() - start_time

    return best_state, best_score, objective_function_value, stuck_count,
duration, acceptance_probabilities
```

Fungsi get_neighbor menyalin *state* dengan *deepcopy*, lalu memilih satu operator (*swap, move, atau rotate*) dengan peluang sama dan menjalankannya.

```
def get_neighbor(vehicles, ship):
    neighbor = copy.deepcopy(vehicles)
    action = random.choice(["swap", "move", "rotate"])
    if action == "swap":
        utils.swap(neighbor, ship)
    elif action == "move":
        utils.move(neighbor, ship)
    else: #action == "rotate"
        utils.rotate(neighbor, ship)
    return neighbor
```

Fungsi main_simulated_annealing memuat data lewat load_json, lalu meminta pilihan *objective function* (1 untuk shipping fee, 2 untuk ETA). Setelah itu fungsi membuat output di folder output untuk tiap eksekusi dan menjalankan SA sebanyak 3 kali. Tiap *run* dimulai dari *initial state* acak. Pada tiap run, gambar state awal, state akhir, plot objective function, dan plot $e^{\frac{\Delta E}{T}}$ disimpan ke folder tersebut. Di akhir, fungsi mencetak tabel hasil 3 run berisi skor awal, skor akhir, jumlah stuck, dan durasi, lalu menyimpannya ke dokumen csv.

```
def main_simulated_annealing():
    vehicles, ship = utils.load_json(os.path.join(src, "vehicles.json"))
    state = utils.init_state(vehicles, ship) #ini bisa di initialize disini
atau di main yang real main.py, tapi sementara disini dulu buat testing

    choice = input("Objective Function (1 = Max Shipping Fee, 2 = Min ETA):
").strip()
    if choice == "2":
        objective = utils.objective_function_bonus

    else:
        objective = utils.objective_function

    out = new_final_folder(output) #folder final[x] baru tiap run
    prefix = "eta" if choice == "2" else "shipping_fee"

    summary = []

    for run in range(1, 4): #run 3x
        for i in range(100):

            state = utils.init_state(vehicles, ship)
            if objective(state, ship) > 0:
                break

        initial_score = objective(state, ship)

        print(f"\nRUN {run}")
        print(f"State awal (objective function = {initial_score})")
```

```
        #utils.display(state, ship)
        fig, ax = plt.subplots(figsize=(12, 6))
        colors = utils_visualisation.build_color_map(vehicles)
        utils_visualisation.draw_state(ax, state, ship, colors, f"Run {run} -
State Awal")
        fig.savefig(os.path.join(out, f"{prefix}_run{run}_state_awal.png"),
dpi=150, bbox_inches="tight")

        best_state, best_score, objective_function_value, stuck_count,
duration, acceptance_probabilities = simulated_annealing.simulated_annealing(
            copy.deepcopy(state), ship, max_iter=1000, init_temp=30,
cooling_rate=0.99, stuck_threshold=10, objective=objective)

        print(f"State akhir (objective function = {best_score})")
        #utils.display(best_state, ship)
        fig, ax = plt.subplots(figsize=(12, 6))
        colors = utils_visualisation.build_color_map(vehicles)
        utils_visualisation.draw_state(ax, best_state, ship, colors, f"Run
{run} - State Akhir")
        fig.savefig(os.path.join(out, f"{prefix}_run{run}_state_akhir.png"),
dpi=150, bbox_inches="tight")

        summary.append((run, initial_score, best_score, stuck_count,
duration))

        plot(objective_function_value, "Objective Function", run)
        plt.gcf().savefig(os.path.join(out,
f"{prefix}_run{run}_plot_objFunc.png"), dpi=150, bbox_inches="tight")
        plot(acceptance_probabilities, "e^(deltaE / T)", run)
        plt.gcf().savefig(os.path.join(out, f"{prefix}_run{run}_plot_e.png"),
dpi=150, bbox_inches="tight")

    print("\nHASIL 3X RUN")
    print(f"{'Run':<5}{'Initial':<10}{'Final':<10}{'Stuck':<8}{'Durasi
(s)':<12}")
    for run, init_s, final_s, stuck, dur in summary:
        print(f"{run:<5}{init_s:<10}{final_s:<10}{stuck:<8}{dur:<12.4f}")

    with open(os.path.join(out, "summarySA.csv"), "w", newline="",
encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["run", "initial_of", "final_of", "stuck", "durasi_detik"])
        for run, init_s, final_s, stuck, dur in summary:
            w.writerow([run, init_s, final_s, stuck, f"{dur:.4f}"])
```

2.5.6. **Genetic Algorithm**

Genetic Algorithm (GA) adalah metode optimasi berbasis populasi yang meniru evolusi biologis. Setiap solusi kandidat menjadi individu dalam populasi. Individu dengan *fitness* lebih tinggi lebih berpeluang menjadi induk *(selection*), dua induk digabung menjadi satu keturunan (*crossover*), dan keturunan diubah sedikit secara acak (mutation) untuk menjaga keragaman. Proses ini diulang selama sejumlah generasi dan hasil akhirnya adalah individu terbaik yang pernah ditemukan.

Pada *Vessel Loading Problem*, satu individu adalah satu state lengkap, dimana seluruh kendaraan beserta keputusan muat atau tidak, koordinat, dan orientasinya. *Fitness*

individu adalah utils.objective_function, yaitu total *shipping fee* kendaraan terangkut (0 bila melanggar kendala), sehingga nilai yang lebih besar lebih baik. Populasi berisi population_size individu dan GA berjalan selama max_iter generasi. Variasi yang dipakai adalah seleksi turnamen berukuran 3, *uniform crossover,* mutasi lewat operator *move*, *rorate*, dan *swap*, individu terbaik selalu dibawa ke generasi berikutnya dan heuristik repair_state yang menjaga individu tetap valid.

```
import copy
import random
import time
import utils
def genetic_algorithm(vehicles, ship, population_size = 30, max_iter = 100,
mutation_rate = 0.15, seed = None):
    if type(population_size) != int:
        raise ValueError("Jumlah population harus bilangan bulat.")
    if population_size < 2:
        raise ValueError("Jumlah populasi minimal 2.")
    if type(max_iter) != int:
        raise ValueError("Jumlah iterasi harus bilangan bulat.")
    if max_iter < 0:
        raise ValueError("Jumlah iterasi tidak boleh negatif.")
    if mutation_rate < 0 or mutation_rate > 1:
        raise ValueError("Peluang mutasi harus antara 0 dan 1.")
    for vehicle in vehicles:
        if vehicle.shippingFee < 0:
            raise ValueError("Tarif pengiriman tidak boleh negatif.")
    used_ids = []
    for vehicle in vehicles:
        if vehicle.id in used_ids:
            raise ValueError("ID kendaraan tidak boleh sama.")
        used_ids.append(vehicle.id)
    if seed is not None:
        random.seed(seed)

    start_time = time.perf_counter()

    population = []

    for i in range(population_size):
        state = create_individual(vehicles,ship)
        population.append(state)

    initial_population = copy.deepcopy(population)

    scores = []

    for state in population:
        score = utils.objective_function(state, ship)
        scores.append(score) #tambahkan angka ke list scores

    best_index = scores.index(max(scores))

    best_state = copy.deepcopy(population[best_index])
    best_score = scores[best_index]

    max_history = [max(scores)]
    mean_history = [sum(scores)/len(scores)]

    initial_state=copy.deepcopy(best_state)
    initial_score=best_score

    for iteration in range(max_iter):
```

```
        #simpan satu individiu paling terbaik ke generaksi berikutnya
        new_population = [copy.deepcopy(best_state)]

        while len(new_population) < population_size:
            parent_a, parent_b = select_parents(population,scores)
            child = crossover(parent_a, parent_b)
            child = mutate(child,ship,mutation_rate)

            new_population.append(child)

        population = new_population


        scores = []
        for state in population:
            score=utils.objective_function(state,ship)
            scores.append(score)
        best_index= scores.index(max(scores))


        if scores[best_index]>best_score:
            best_state = copy.deepcopy(population[best_index])
            best_score = scores[best_index]


        max_history.append(max(scores))
       mean_history.append(sum(scores)/len(scores))


    duration=time.perf_counter()-start_time

    info = {
        "initial_population": initial_population,
        "initial_state": initial_state,
        "initial_score": initial_score,
        "max_history": max_history,
        "mean_history": mean_history,
        "duration": duration,
        "population_size": population_size,
        "iterations": max_iter,
        "mutation_rate": mutation_rate,
        "seed": seed,
    }

    return best_state, best_score, info

def create_individual(vehicles,ship):
    state = copy.deepcopy(vehicles)

    for vehicle in state:
        if random.random() < 0.5:
            vehicle.dimension.width, vehicle.dimension.length =
(vehicle.dimension.length, vehicle.dimension.width,)
            if vehicle.orientation == "Horizontal":
                vehicle.orientation = "Vertical"
            else:
                vehicle.orientation = "Horizontal"

        max_x = ship.dimension.width-vehicle.dimension.width
        max_y = ship.dimension.length-vehicle.dimension.length

        vehicle.is_loaded = False


        if random.random() < 0.5:
            if max_x >= 0 and max_y >= 0:
                vehicle.is_loaded = True
                vehicle.x = random.randint(0,max_x) #pilih koordinat acak
                vehicle.y = random.randint(0,max_y)
```

```
    repair_state(state,ship) #fix benturan and kapasitas
    return state

def repair_state(state, ship):
    candidates = []
    for vehicle in state:
        if vehicle.is_loaded:
            candidates.append(vehicle)


    for vehicle in state:
        vehicle.is_loaded = False

    random.shuffle(candidates)


    for vehicle in candidates:
        max_x = ship.dimension.width-vehicle.dimension.width
        max_y = ship.dimension.length-vehicle.dimension.length
        if max_x < 0 or max_y < 0:
            continue

        vehicle.is_loaded = True
        if utils.exceed_capacity_limit(state,ship):
            vehicle.is_loaded = False
            continue
   position_is_valid = (not utils.exceed_ship_limit(state,ship) and
utils.not_overlapping(state,ship))
        if not position_is_valid:
            positions=[]

            for x in range (max_x + 1):
                for y in range (max_y+1):
                    positions.append((x,y))

            random.shuffle(positions)

            for x, y in positions:
                vehicle.x = x
                vehicle.y = y

                position_is_valid = (
                    not utils.exceed_ship_limit(state,ship)
                    and utils.not_overlapping(state,ship)
                )

                if position_is_valid:
                    break

        if not position_is_valid:
            vehicle.is_loaded = False

    park_unloaded(state,ship)
    return state


def select_parents(population, scores):
    parents=[]

    #lakukan dua turnamen untuk pilih dua parent
    for i in range (2):
        #ambil maks 3 individual secara acak
        tournament_size = min(3,len(population))

        candidate_indices =
random.sample(range(len(population)),tournament_size)

        pemenang_index = candidate_indices[0] #sementara
        for index in candidate_indices:
            if scores[index] > scores[pemenang_index]:
```

```
                pemenang_index = index

        parents.append(population[pemenang_index])

    return parents[0], parents[1]

def crossover(parent_a, parent_b):
    child = []
    for i in range(len(parent_a)):
        #peluang 50 persen mengambil gene dari masing-masing parent
        if random.random() < 0.5:
            vehicle = copy.deepcopy(parent_a[i])
        else:
            vehicle = copy.deepcopy(parent_b[i])

        child.append(vehicle)
    return child




def mutate (state,ship,mutation_rate):
    repair_state(state,ship)

    if len(state) == 0:
        return state

    #tentuin apakah anak ini menjalani mutasi
    if random.random() < mutation_rate:
        operations = [utils.move, utils.rotate]

        #swap butuh minimal 2 kendaraan
        if len(state) >= 2:
            operations.append(utils.swap)

        #pilih dan jalankan satu operasi
        operation = random.choice(operations)
        operation(state,ship)

    park_unloaded(state,ship)
    return state

def park_unloaded(state,ship):
    outside_x = ship.dimension.width + 2
    next_y = 0
    for vehicle in state:
        if not vehicle.is_loaded:
            vehicle.x = outside_x
            vehicle.y = next_y
            next_y += vehicle.dimension.length + 1
```

Penjelasan *Source Code* Algorithm *Genetic Algorithm*

Fungsi genetic_algorithm() yang menjalankan seluruh pencarian. Fungsi ini menerima enam parameter yang dijelaskan pada tabel berikut.

Parameter	Keterangan
vehicles	Daftar kendaraan hasil utils.load_json
ship	Objek kapal (dimensi dan kapasitas maksimum)

population_size	Jumlah individu per generasi
max_iter	Jumlah iterasi
mutation_rate	Peluang satu anak bermutasi (antara 0 dan 1)
seed	Benih acak agar percobaan dapat diulang

Hasilnya *tuple* (best_state, best_score, info). Kamus info berisi intial_population, initial_state, max_history, mean_history, duration, population_size, iterations, mutation_rate, dan *seed*.
Variabel yang digunakan dalam fungsi adalah sebagai berikut.

- population dan scores: individu generasi saat ini dan nilai objective function masing-masing.
- best_state dan best_score: individu terbaik yang pernah ditemukan.
- initial_state dan initial_score: individu terbaik pada populasi awal, dipakai sebagai state awal gambar.
- max_history dan mean_history: skor maksimum dan rata-rata populasi pada tiap generasi (termasuk generasi 0), untuk bahan plot konvergensi.
- new_population: populasi generasi berikutnya yang sedang dibentuk.
- Start_time dan duration: pengukur durasi pencarian dengan time.perf_counter().

Validasi dan Inisialisasi

Pada tahap awal, fungsi memeriksa tipe dan rentang setiap parameter serta keunikan ID kendaraan, dan memunculkan ValueError apabila ada yang tidak sesuai. Jika *seed* diberikan, fungsi memanggil random.seed(seed) agar percobaan dapat diulang. Selanjutnya, populasi awal dibentuk dengan memanggil create_individual sebanyak population_size kali, setiap individu dinilai dengan utils.objective_function, dan individu dengan skor tertinggi disimpan sebagai best_state.

```
for i in range(population_size):
    state = create_individual(vehicles,ship)
    population.append(state)

scores = []
for state in population:
    score = utils.objective_function(state, ship)
    scores.append(score)

best_index = scores.index(max(scores))
best_state = copy.deepcopy(population[best_index])
best_score = scores[best_index]
```

Fungsi create_individual menyalin kendaraan, memutar tiap kendaraan dengan peluang 50% pada posisi acak, lalu memanggil repair_state sehingga setiap individu awal dipastikan valid. Dengan demikian populasi awal sudah beragam tanpa memboroskan evaluasi pada *state* berskor 0.

Heuristik repair_state

Fungsi ini dipanggil di dua tempat yaitu pada akhir create_individual dan pada awal mutate, karena penempatan acak maupun hasi *crossover* dapat melanggar kendala, misalnya dua kendaraan yang saling bertumpuk atau berat total yang melebihi kapasitas. Perbaikan dilakukan dengan mengosongkan geladak, lalu memuat kandidat satu per satu dalam urutan acak hanya jika kapasitas masih cukup dan tersedia posisi tanpa tumpang tindih (posisi sekarang dicoba terlebih dahulu, kemudian seluruh posisi lain dalam urutan acak). Kendaraan yang tidak dapat ditempatkan dibiarkan tidak dimuat, lalu park_unloaded memindahkan ke luar geladak (di sisi kanan kapal, pada x = lebar kapal +2) agar tidak ikut dihitung. Dengan cara ini, setiap individu selalu valid sehingga evaluasi tidak terbuang pada *state* berskor 0.

Proses Perulangan Generasi

Pada setiap iterasi, populasi baru diawali dengan salinan best_state sehingga individu terbaik tidak pernah hilang. Sisa populasi diisi oleh anak yang dibentuk melalui tiga tahap, yaitu seleksi, *crossover,* dan mutasi. Setelah populasi baru penuh, seluruh individu dinilai kembali dengan *objective function.*

```
for iteration in range(max_iter):
    new_population = [copy.deepcopy(best_state)]

    while len(new_population) < population_size:
        parent_a, parent_b = select_parents(population,scores)
        child = crossover(parent_a, parent_b)
        child = mutate(child,ship,mutation_rate)
        new_population.append(child)

    population = new_population
```

Membangkitkan Tetangga dan Mencari Tetangga Terbaik

Proses individu anak dilakukan melalui seleksi induk, *crossover,* dan mutasi. Ketiga tahap ini menghasilkan calon solusi baru yang akan dievaluasi untuk membentuk populasi generasi berikutnya.

1. Seleksi induk (select_parents): Pemilihan dua induk menggunakan metode *tournament selection.* Pada setiap turnamen, sebanyak min(3, ukuran populasi) individu dipilih secara

acak Individu dengan nilai *objective* tertinggi menjadi pemenang dan dipilih sebagai induk. Proses turnamen dilakukan dua kali untuk memperoleh dua induk.

2. *Crossover* (crossover)

   Pembentukan anak menggunakan metode *uniform crossover.* Untuk setiap kendaraan, atribut posisi, orientasi, dan status muat diambil dari salah satu induk dengan peluang masing-masing 50%. Penggabungan ini dapat menghasilkan susunan yang melanggar kendala, seperti kendaraan yang saling bertumpuk, sehingga diperlukan perbaikan *state.*

3. Mutasi (mutate)

   State anak terlebih dahulu diperbaiki menggunakan repair_state. Selanjutnya, dengan peluang sebesar mutation_rate, anak mengalami satu perubahan melalui operator yang dipilih dengan acak, yaitu *move, rotate,* atau swap. Mutasi memberikan variasi pada solusi yang dihasilkan. Kendaraan yang tidak dimuat kemudian ditempatkan di luar geladak menggunakan park_unloaded.

Memperbarui Individu Terbaik

Setelah seluruh individu dinilai, best_state hanya diganti apabila ditemukan individu dengan skor yang lebih tinggi. Mekanisme ini menjamin bahwa skor terbaik tidak pernah menurun sepanjang proses evolusi.

```
best_index= scores.index(max(scores))
if scores[best_index]>best_score:
    best_state = copy.deepcopy(population[best_index])
    best_score = scores[best_index]
```

Menyimpan Hasil

Pada setiap generasi, skor maksimum dan skor-rata-rata populasi dicatat ke max_history dan mean_history untuk keperluan plot konvergensi. Setelah seluruh generasi selesai, durasi pencarian dihitung dan fungsi mengembalikan best_state, best_score, serta info.

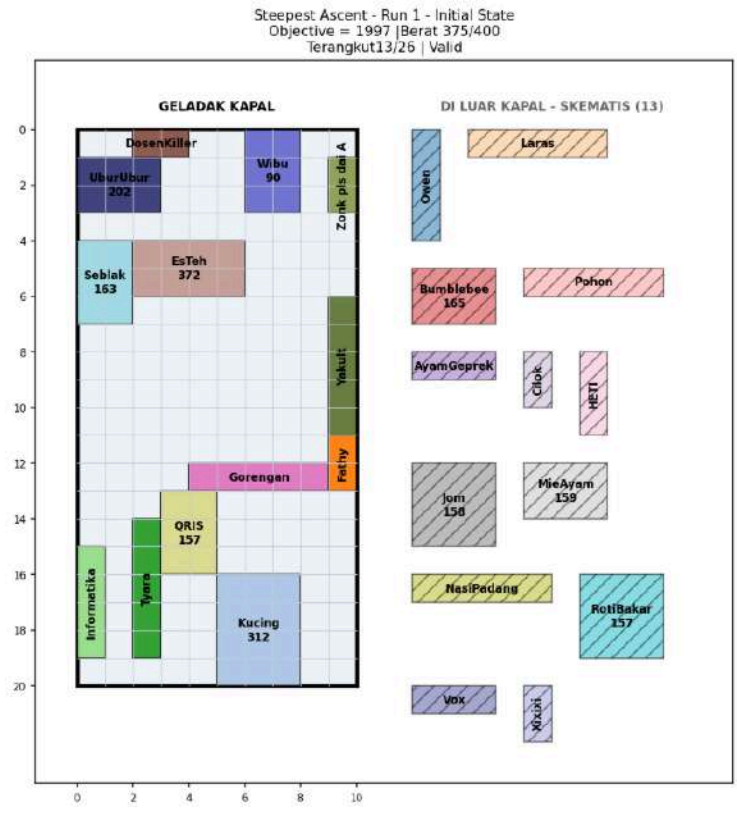
Kesimpulan *Genetic Algorithm*

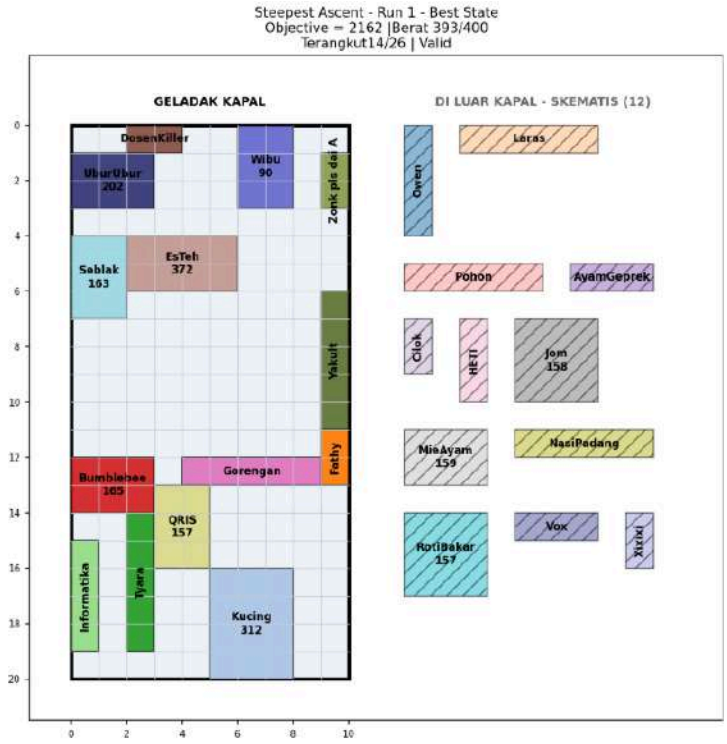
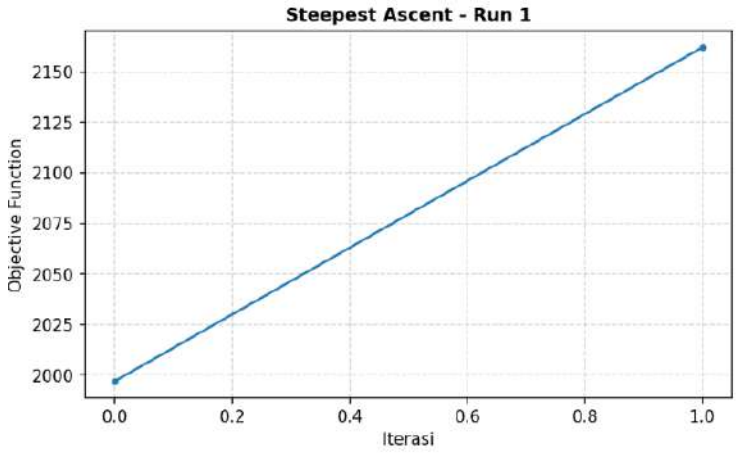
Secara keseluruhan, GA mempertahankan banyak solusi sekaligus sehingga tidak bergantung pada satu jalur pencarian. Seleksi menjaga individu yang baik, *crossover* menggabungkan keputusan penempatan dari dua induk, mutasi menjaga keragaman, dan repair_state memastikan hasilnya valid. Konsekuensinya adalah waktu komputasi yang tumbuh sebanding dengan jumlah populasi dikali jumlah iterasi, karena setiap individu pada setiap generasi harus dinilai.

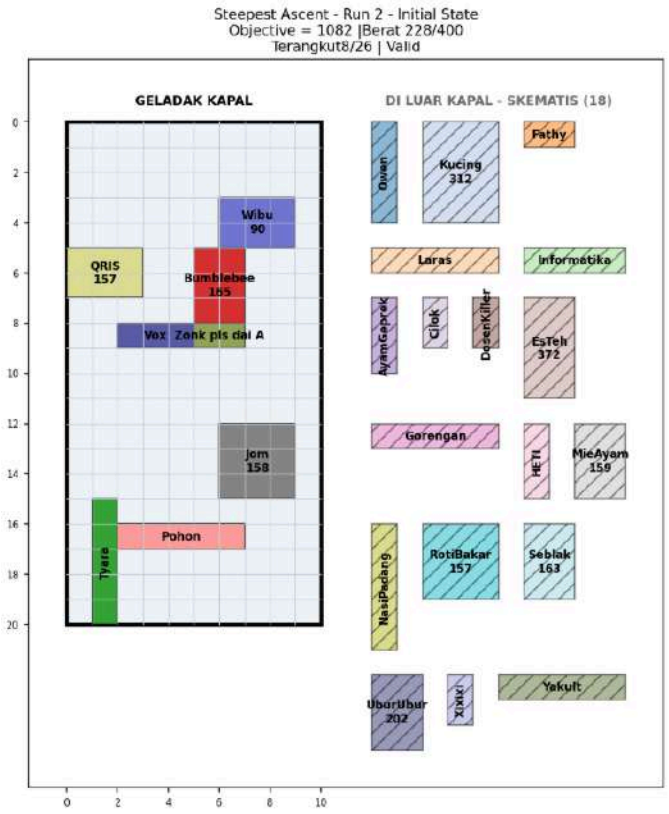
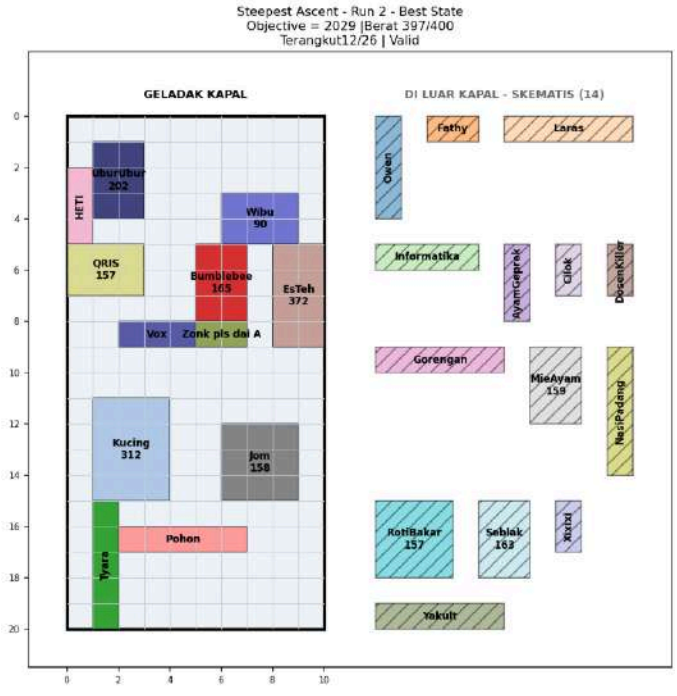
2.7. Hasil Pengujian dan Analisis

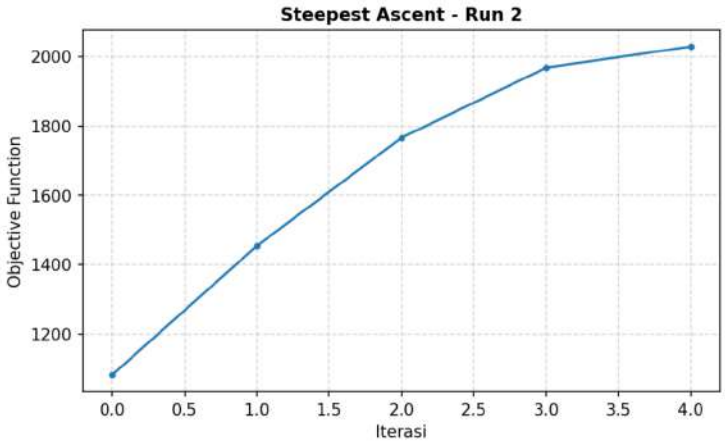
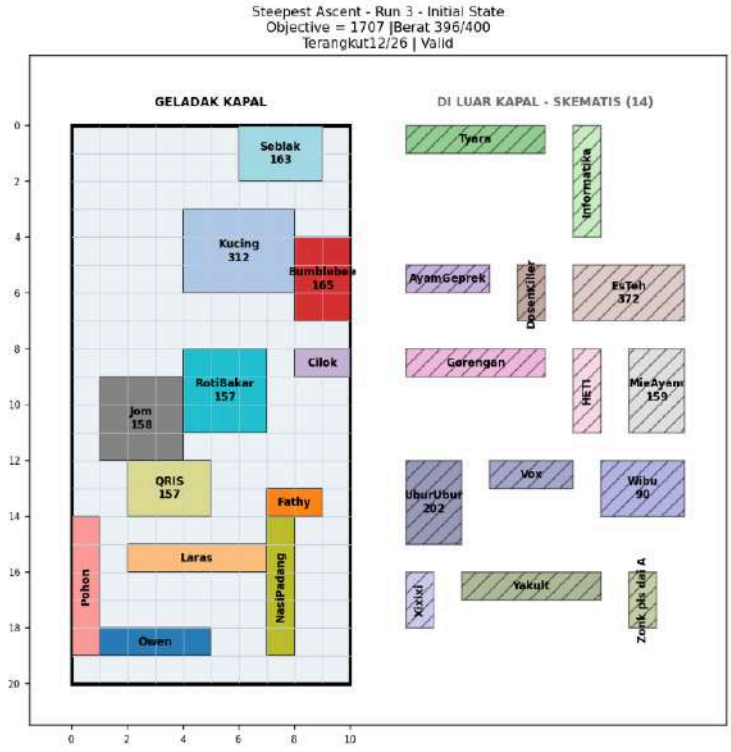
2.7.1. **Steepest Ascent Hill-Climbing**

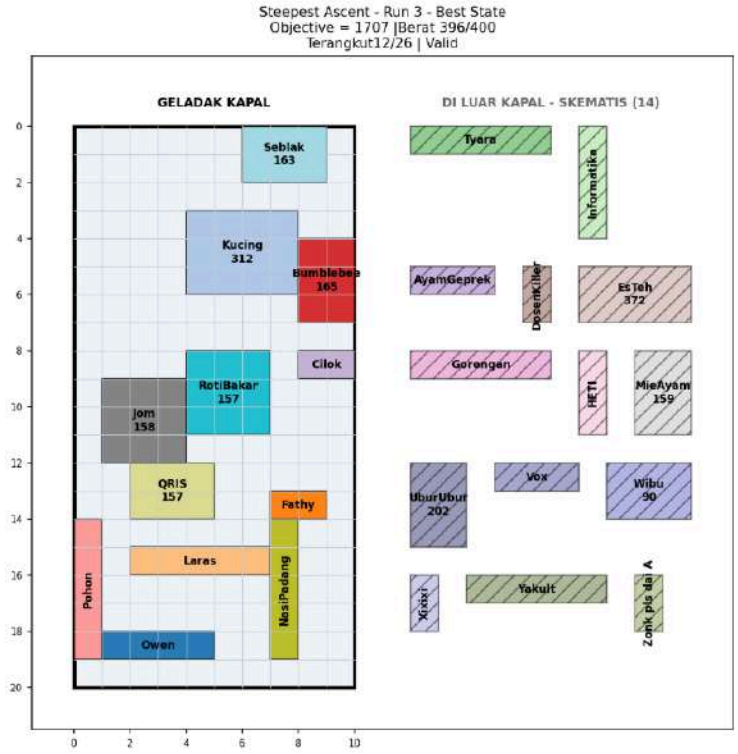
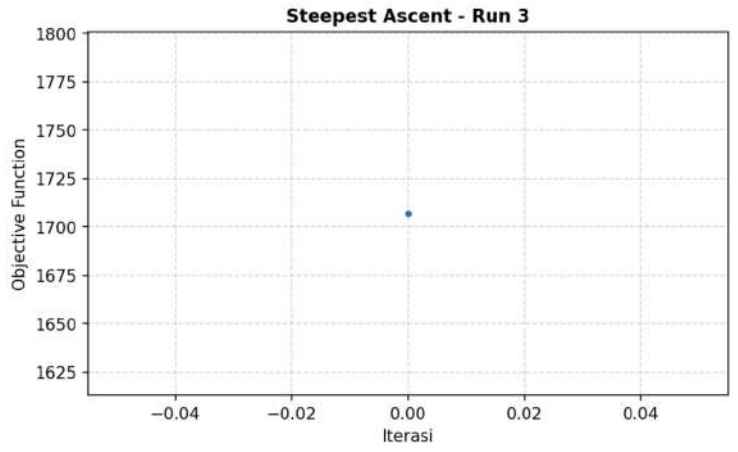
Percobaan ke-	Nilai initial objective function	Nilai final objective function	Durasi	Jumlah Iterasi	Jumlah iterasi
1	1997	2162	0.7366	2	Local Optima
2	1082	2029	1.6975	5	Local Optima
3	1707	1707	0.2586	1	Local Optima

Percobaan 1	
State awal	

State akhir	
Plot objective function terhadap banyak iterasi	

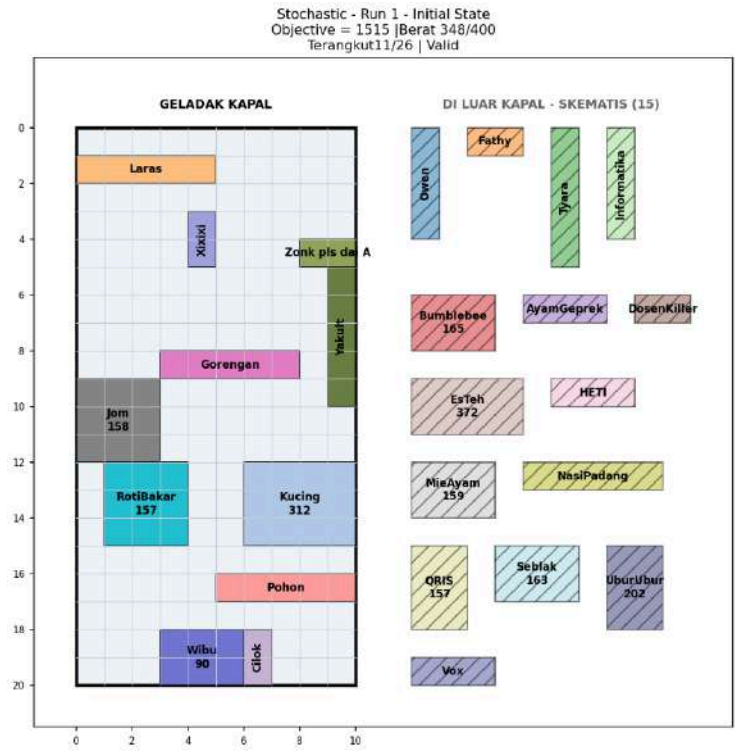
Percobaan 2	
State awal	
State akhir	

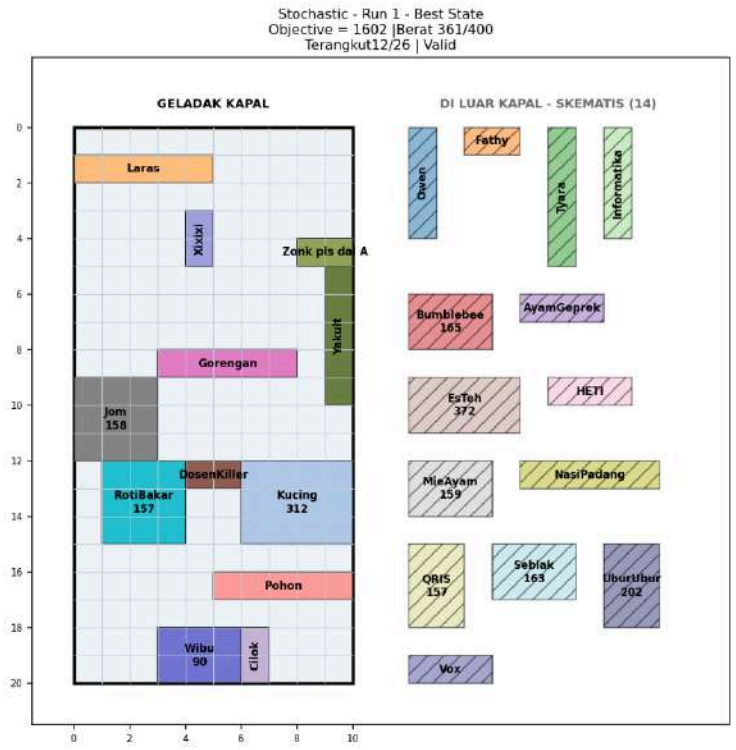
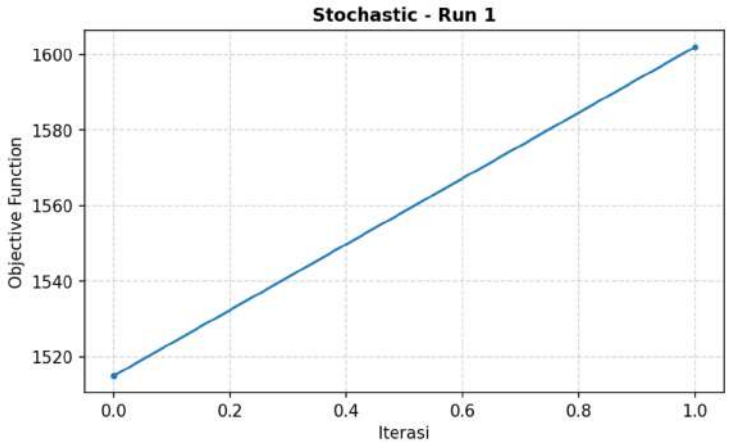
Plot objective function terhadap banyak iterasi	
Percobaan 3	
State awal	

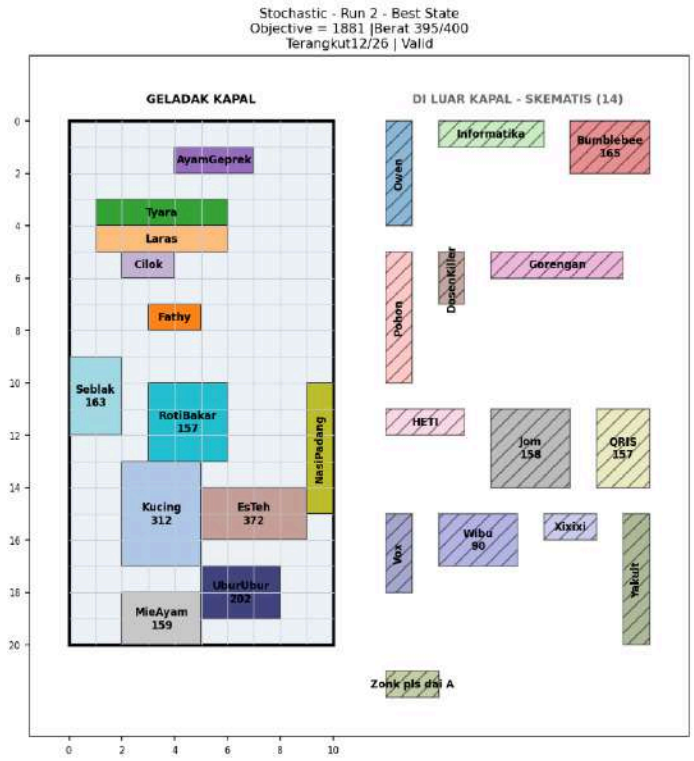
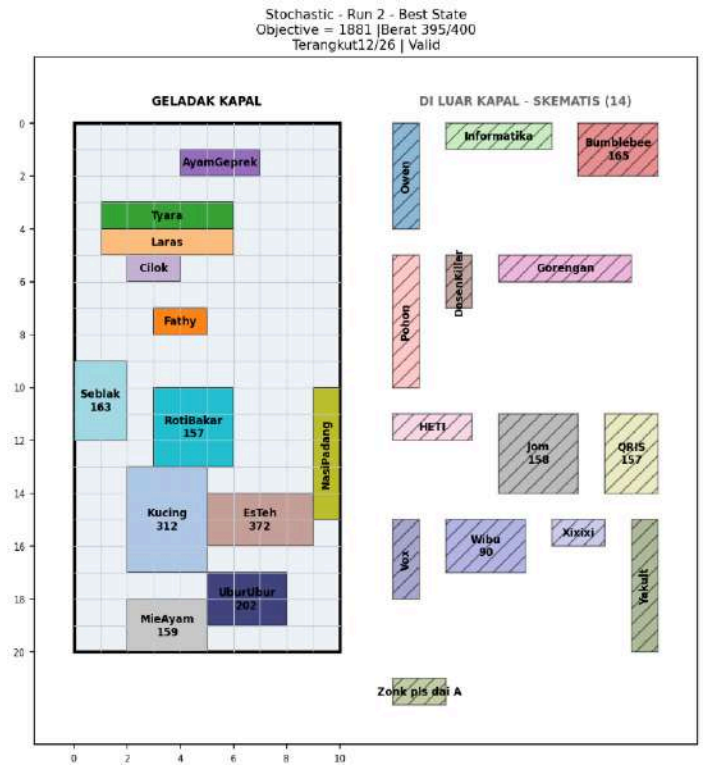
State akhir	
Plot objective function terhadap banyak iterasi	

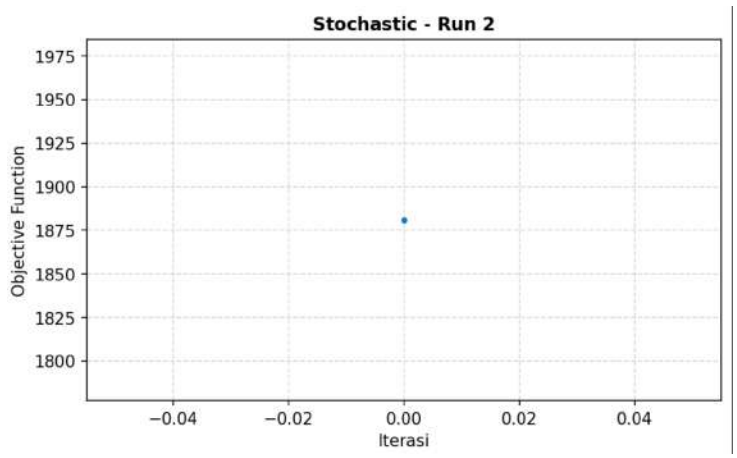
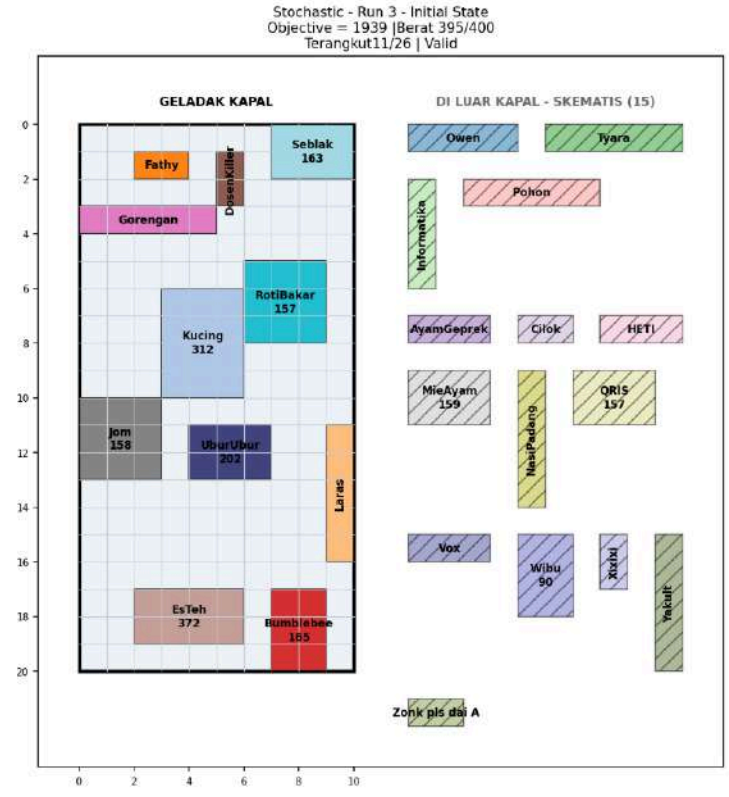
2.7.2. **Stochastic Hill-Climbing Ascent Hill-Climbing**

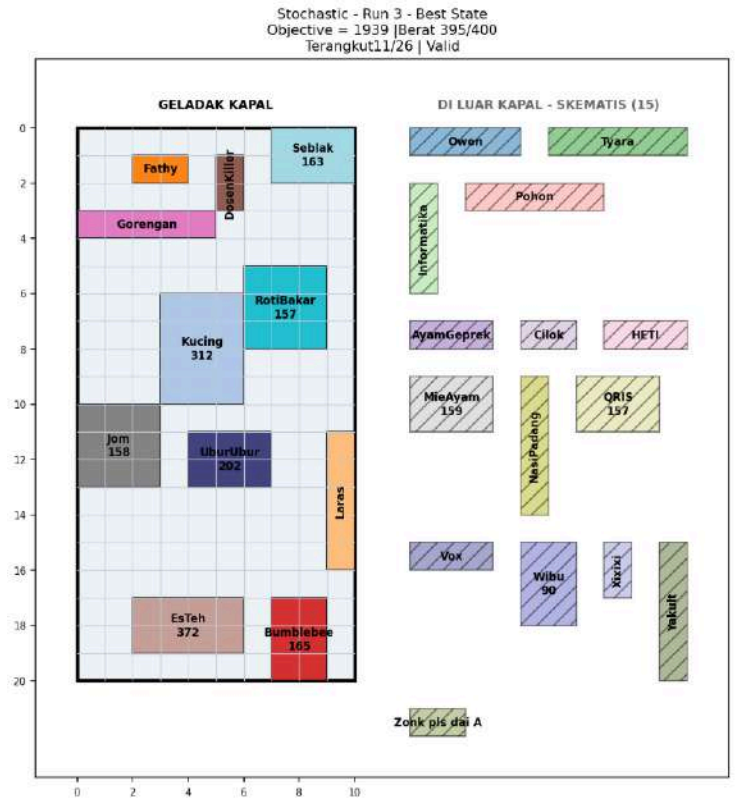
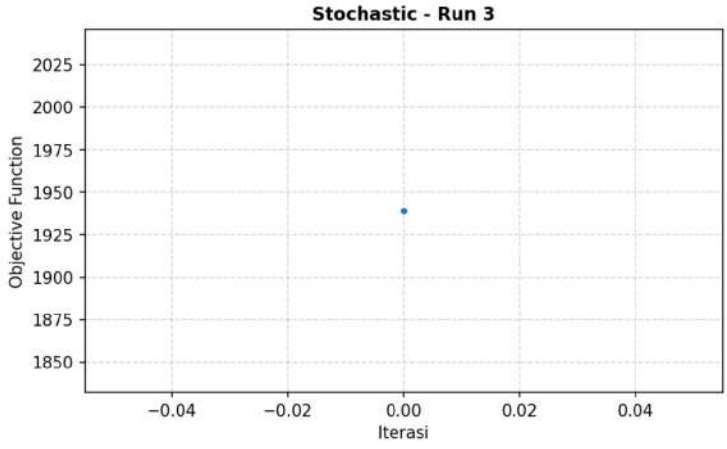
Percobaan ke-	Nilai initial objective function	Nilai final objective function	Durasi	Jumlah iterasi
1	1515	1602	0.0082	Local Optima
2	1881	1881	0.0043	Local Optima
3	1939	1939	0.0039	Local Optima

Percobaan 1	
State awal	

State akhir	
Plot objective function terhadap banyak iterasi	

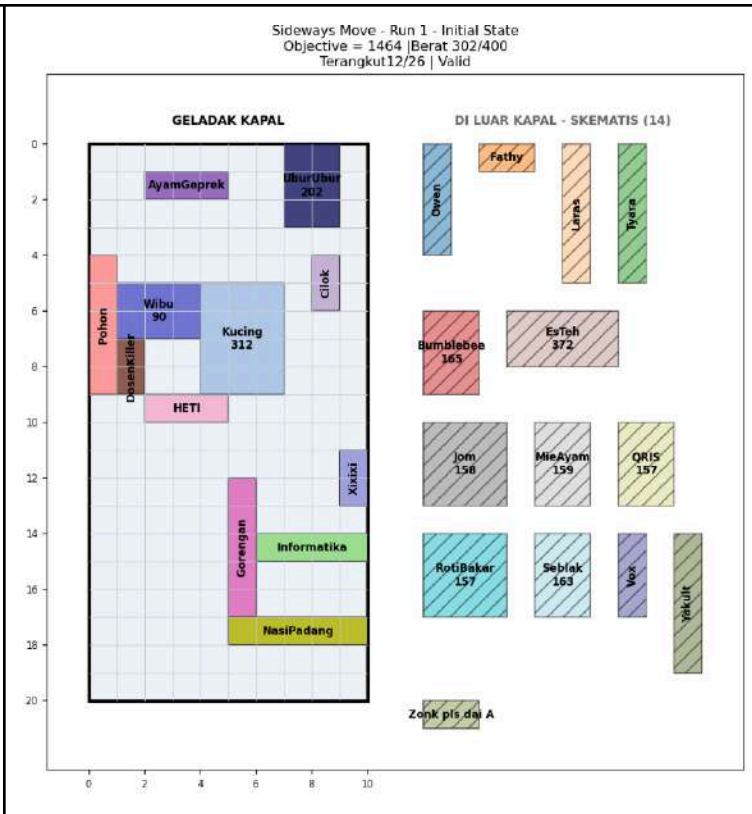
Percobaan 2	
State awal	
State akhir	

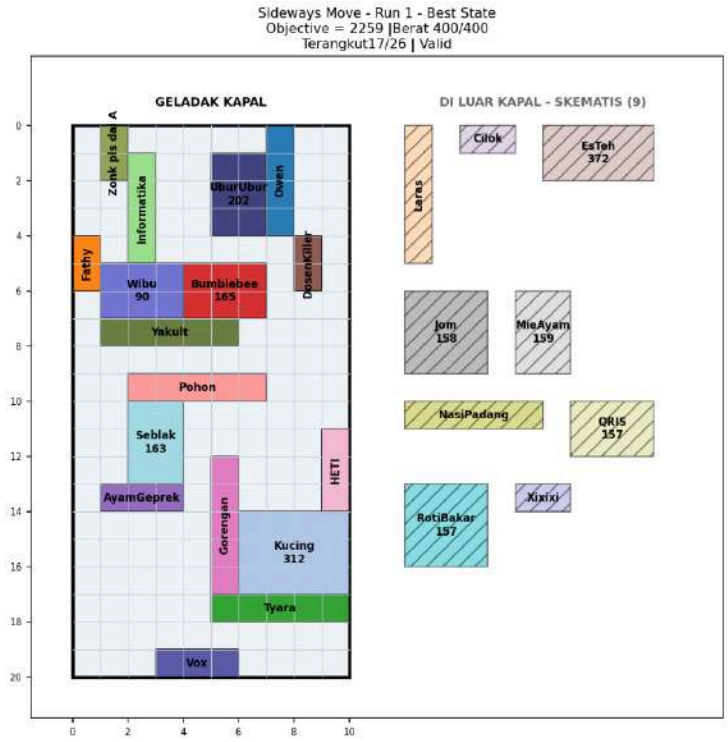
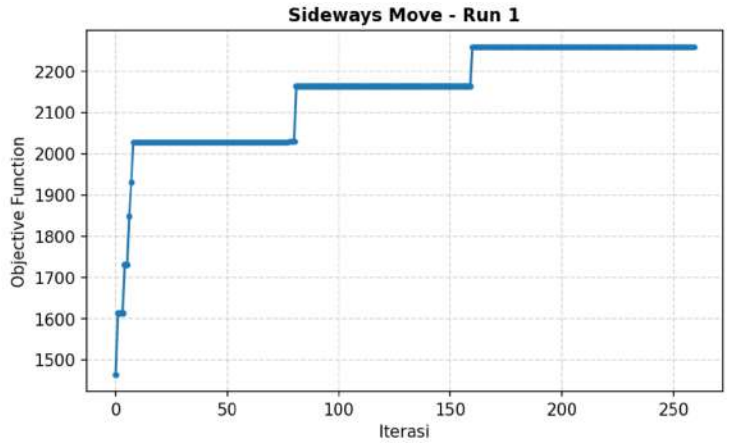
Plot objective function terhadap banyak iterasi	
Percobaan 3	
State awal	

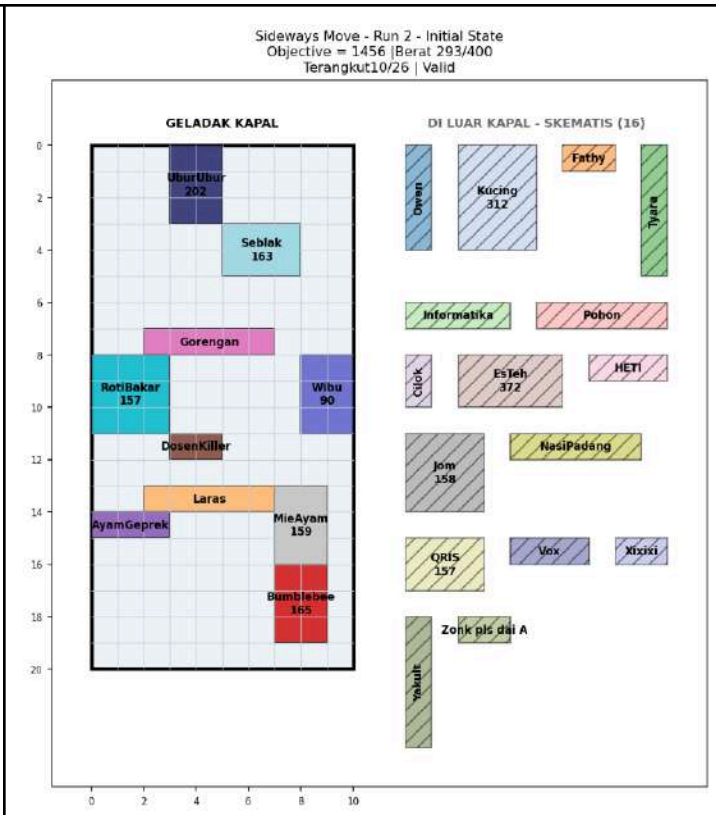
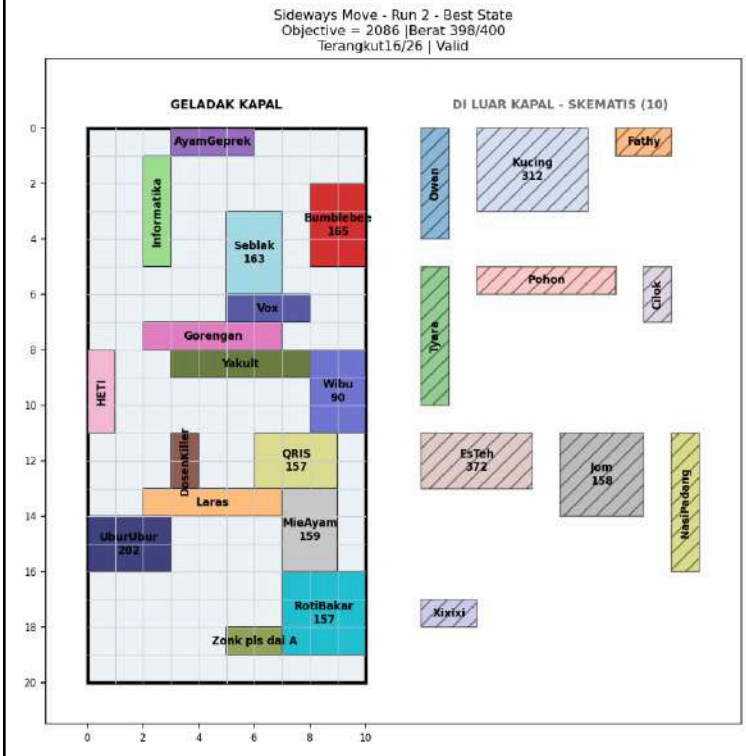
State akhir	
Plot objective function terhadap banyak iterasi	

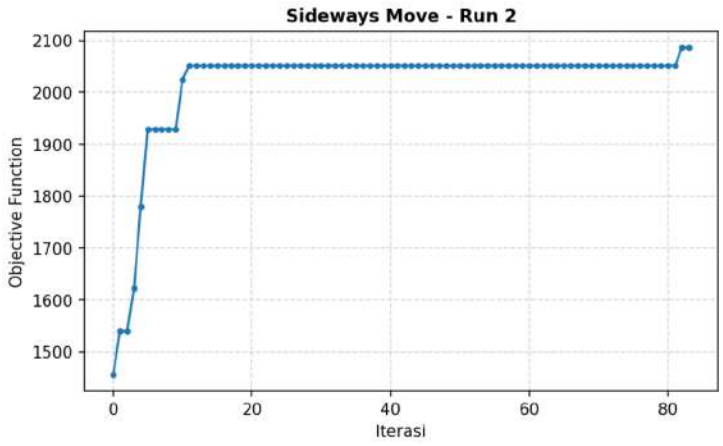
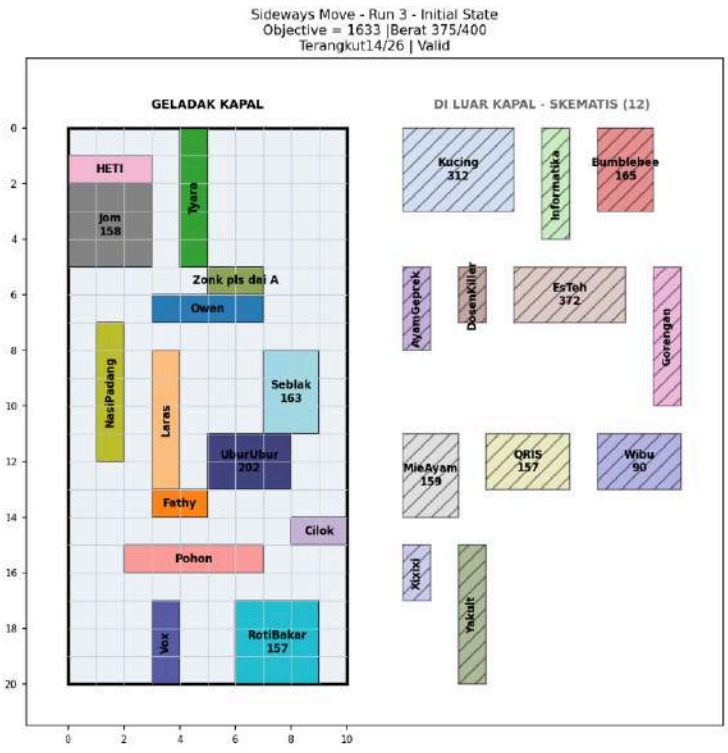
2.7.3. **Hill-Climbing with Sideways Move**

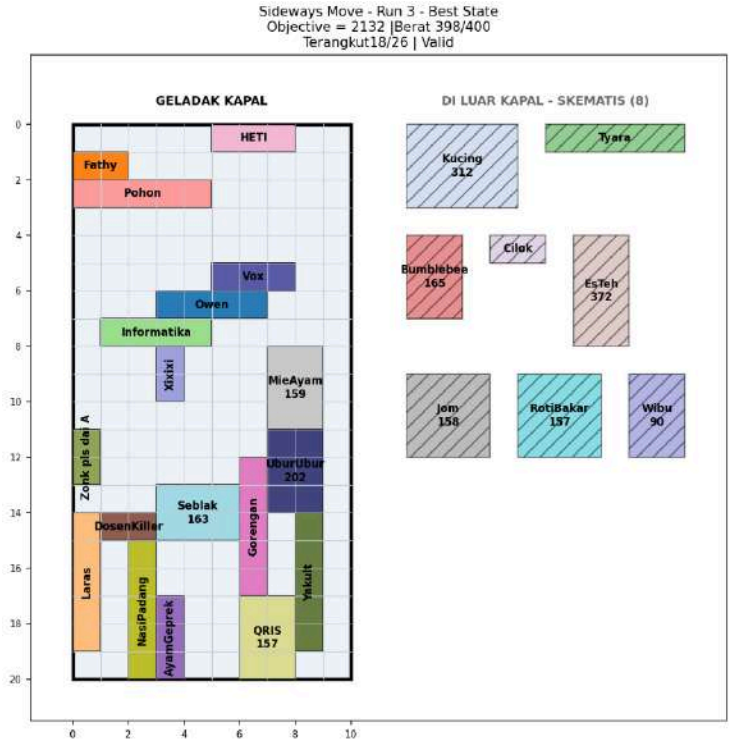
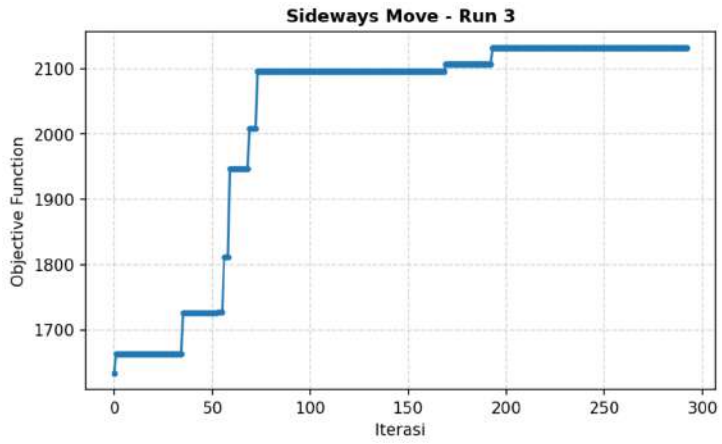
Percobaan ke-	Nilai initial objective function	Nilai final objective function	Durasi	Jumlah iterasi
1	1464	2259	1.0372	260 (252 sideways)
2	1456	2086	0.3491	84 (76 sideways)
3	1633	2132	1.1369	293 (284 sideways)

Percobaan 1	
State awal	

State akhir	
Plot objective function terhadap banyak iterasi	

Percobaan 2	
State awal	
State akhir	

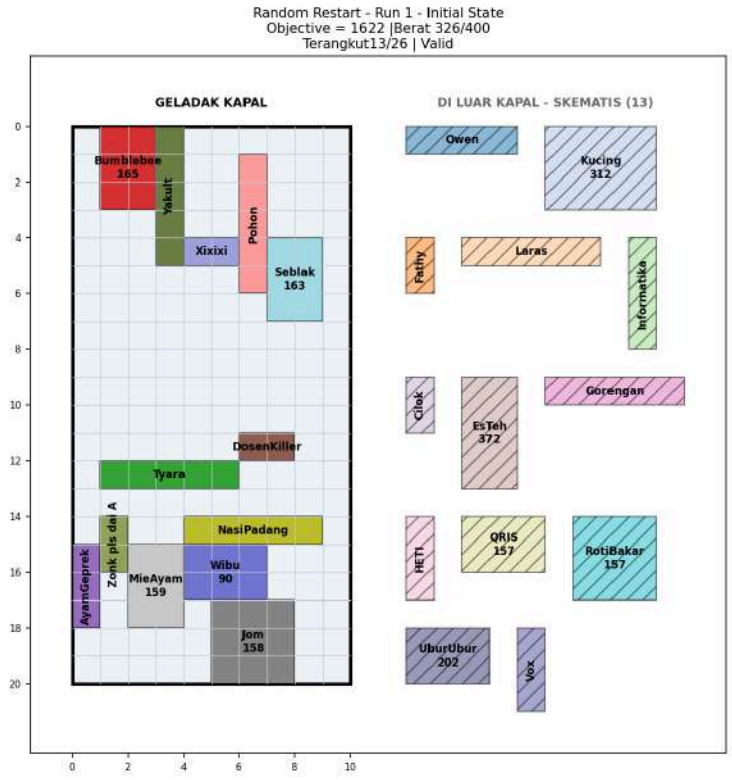
Plot objective function terhadap banyak iterasi	
Percobaan 3	
State awal	

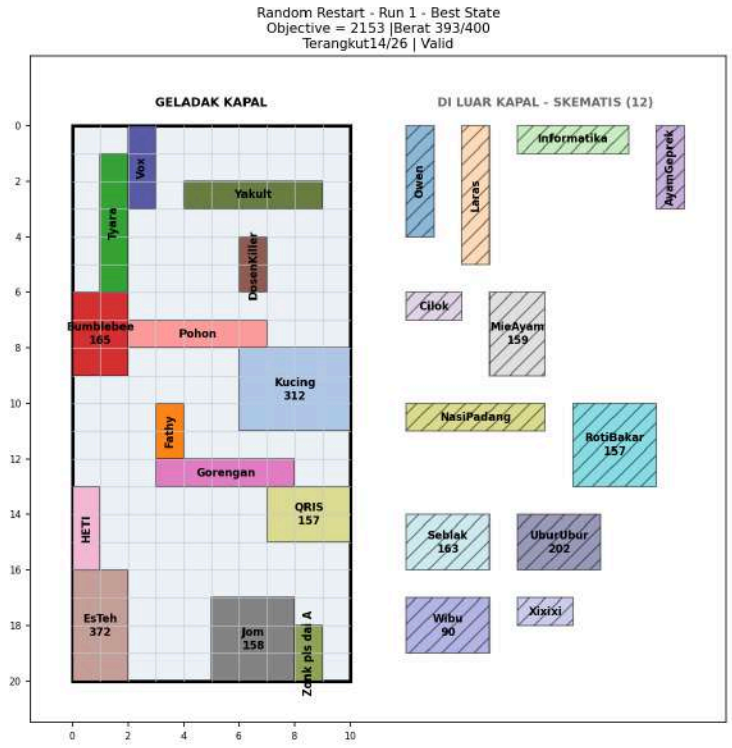
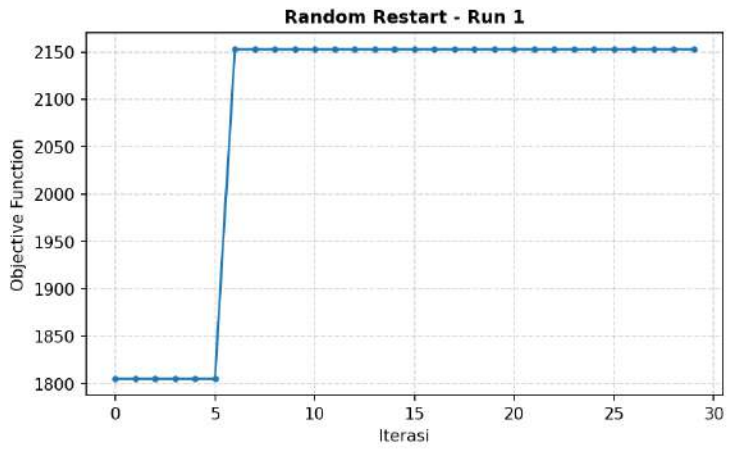
State akhir	
Plot objective function terhadap banyak iterasi	

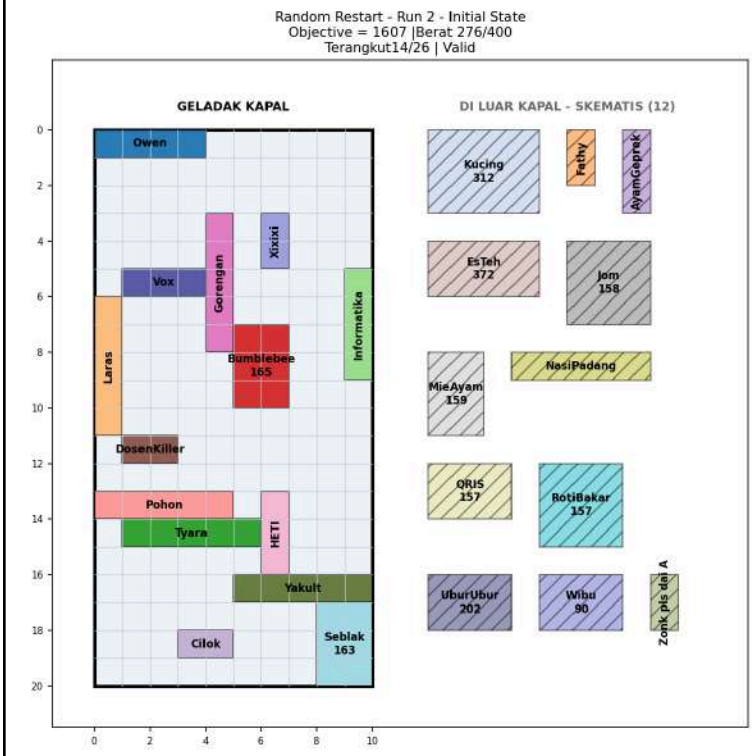
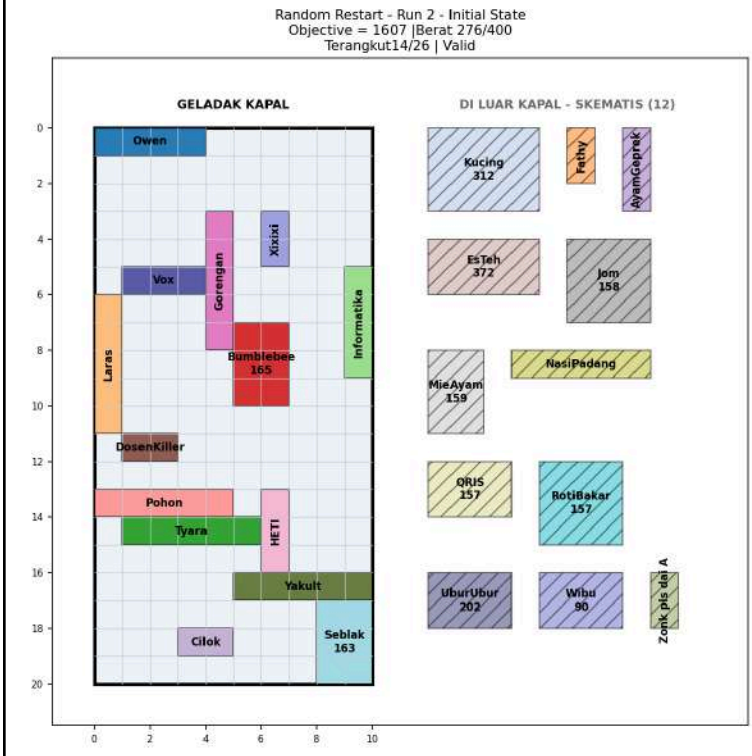
2.7.4. **Random Restart Hill-Climbing**

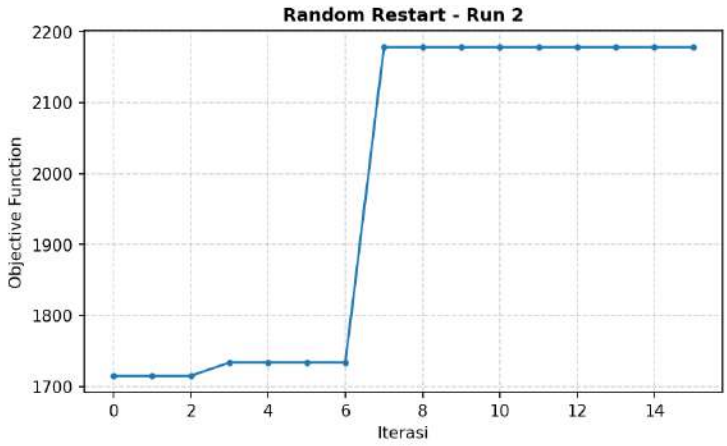
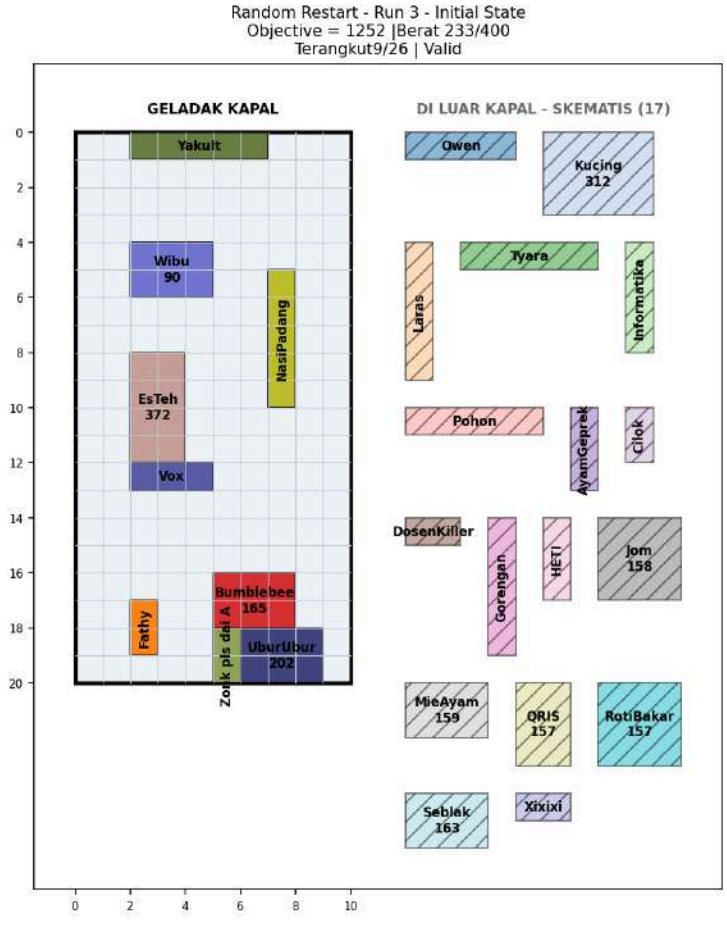
Percobaan ke-	Nilai initial objective function	Nilai final objective function	Durasi	Jumlah Iterasi	Jumlah iterasi
1	1622	2153	0.1646	30	Max Resart Reached

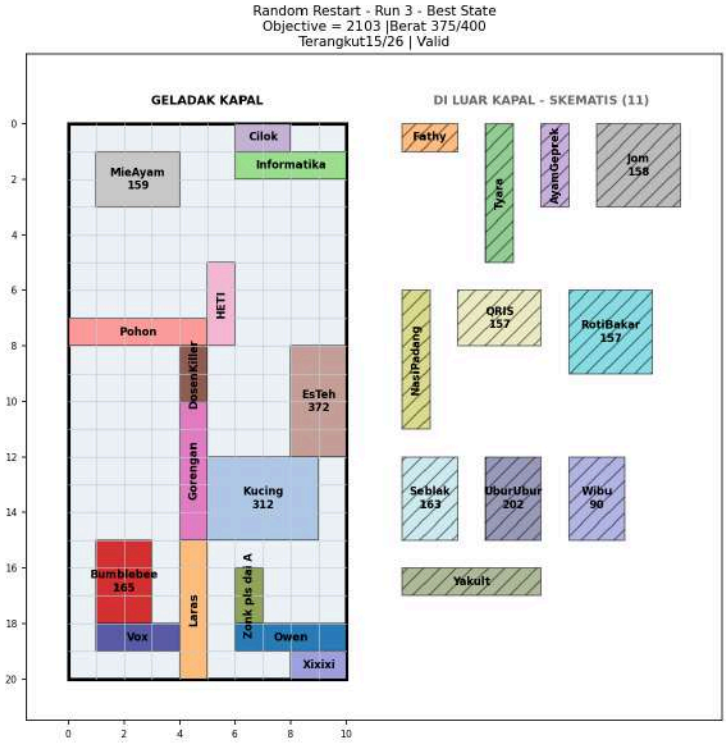
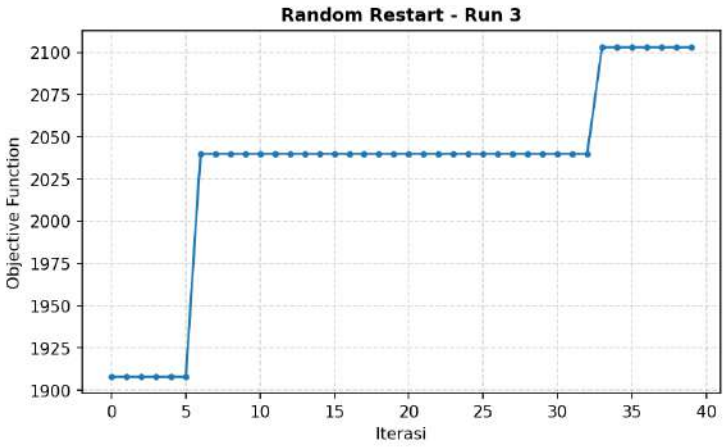
2	1607	2178	0.1205	16	Max Restart Reached
3	1252	2103	0.2051	40	Max Restart Reached

Percobaan 1	
State awal	

State akhir	
Plot objective function terhadap banyak iterasi	

Percobaan 2	
State awal	
State akhir	

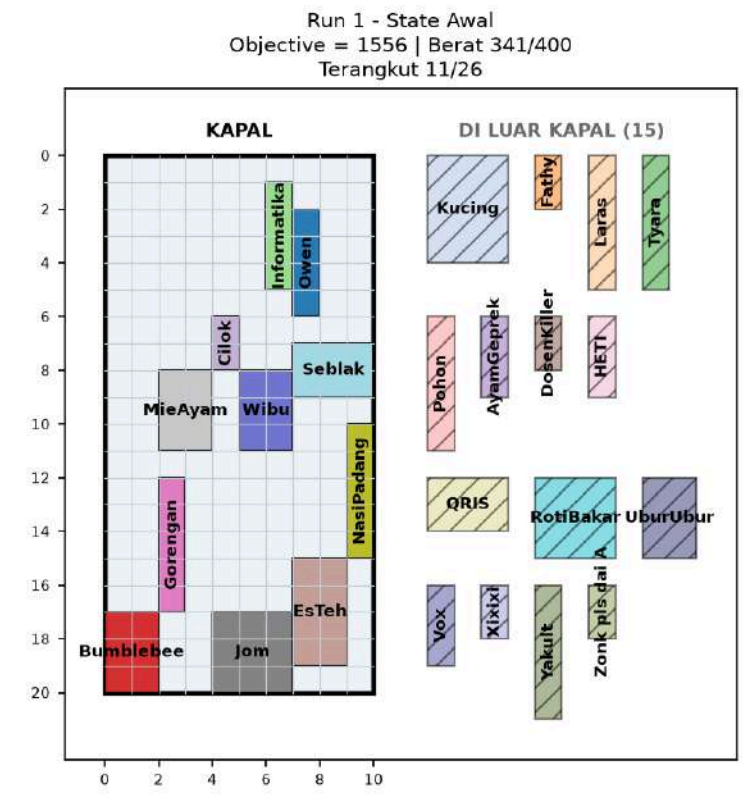
Plot objective function terhadap banyak iterasi	
Percobaan 3	
State awal	

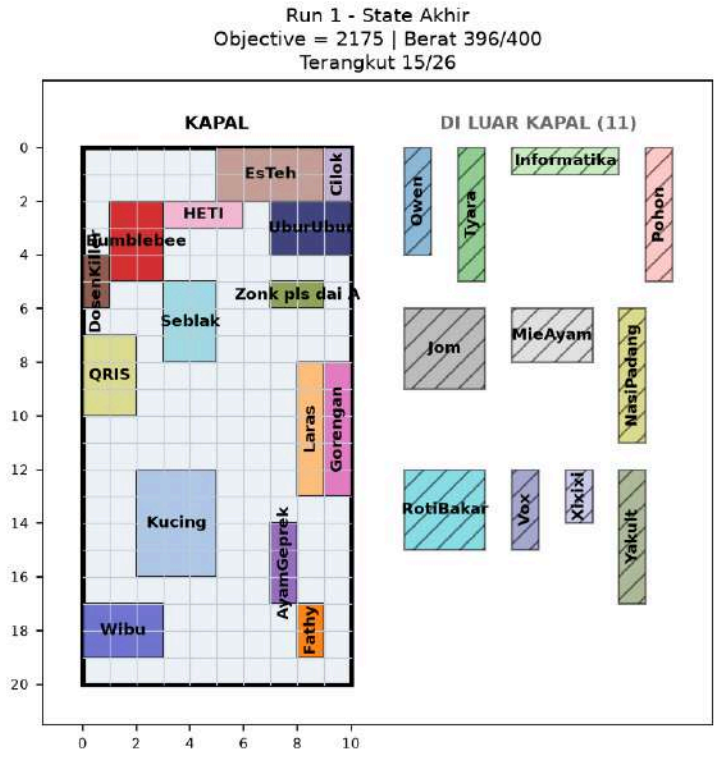
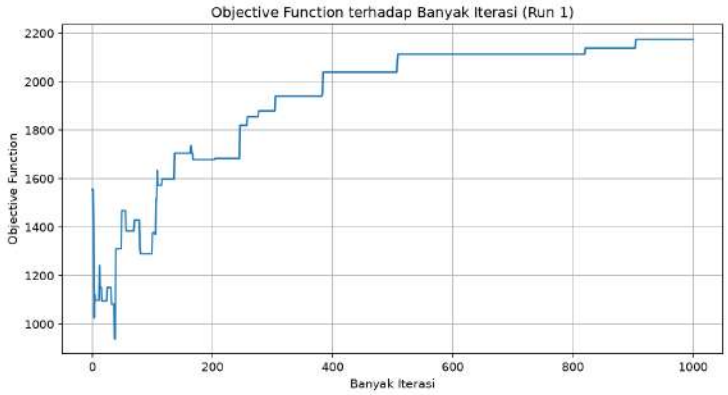
State akhir	
Plot objective function terhadap banyak iterasi	

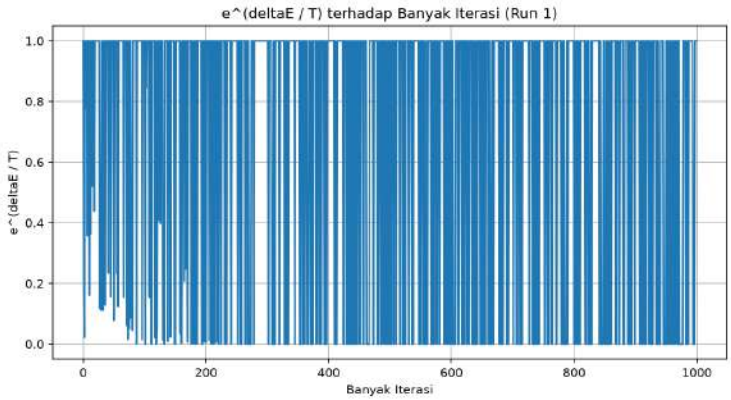
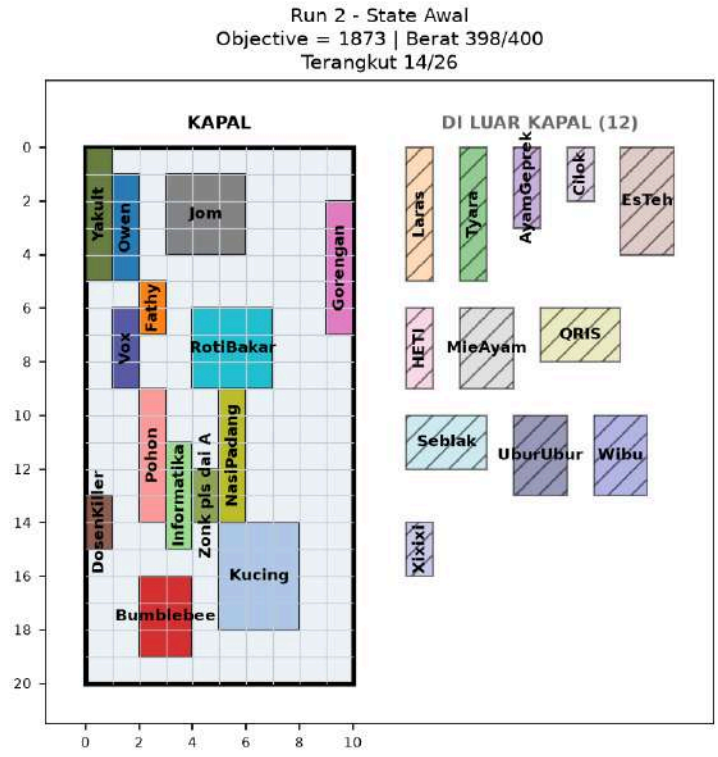
2.7.5. **Simulated Annealing**

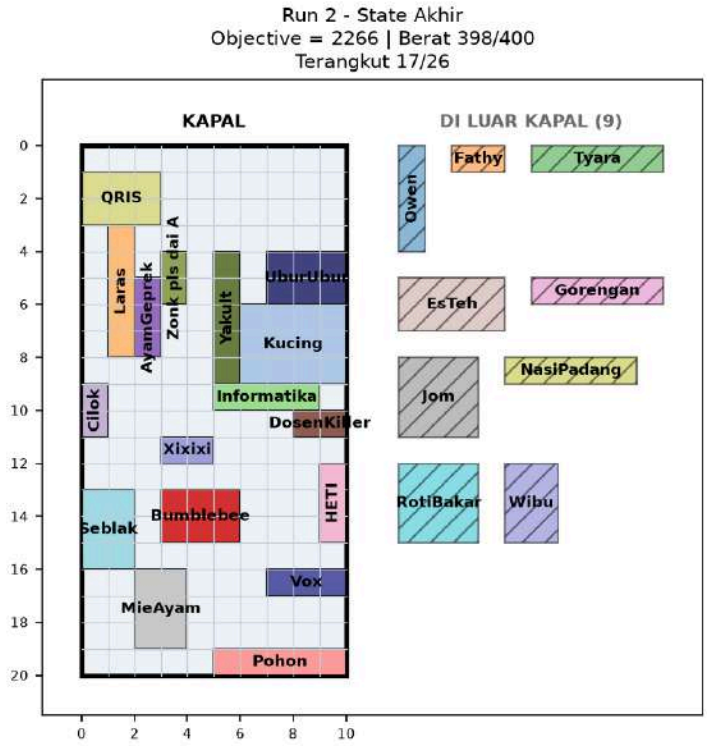
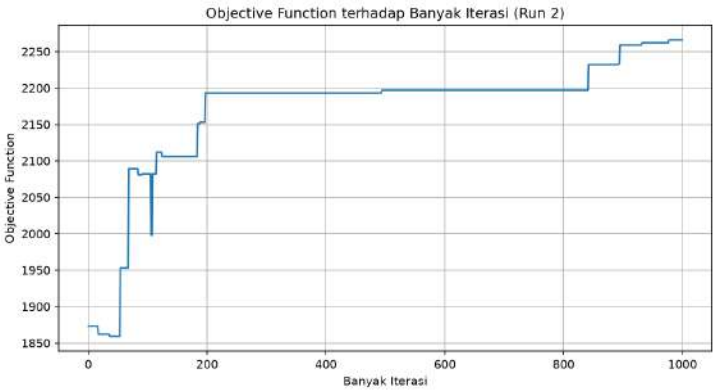
a. Percobaan dengan Objective Function Shipping Fee

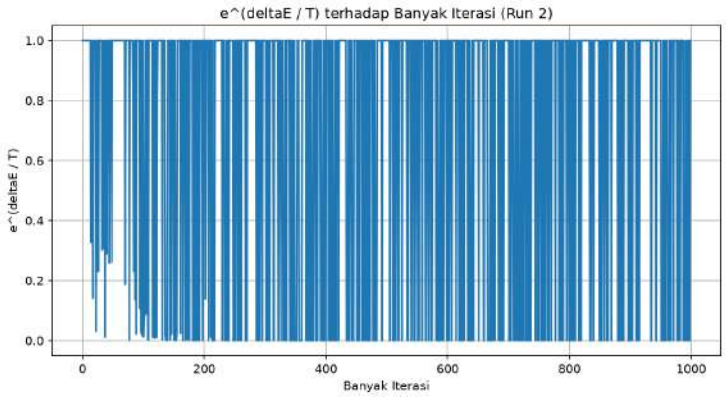
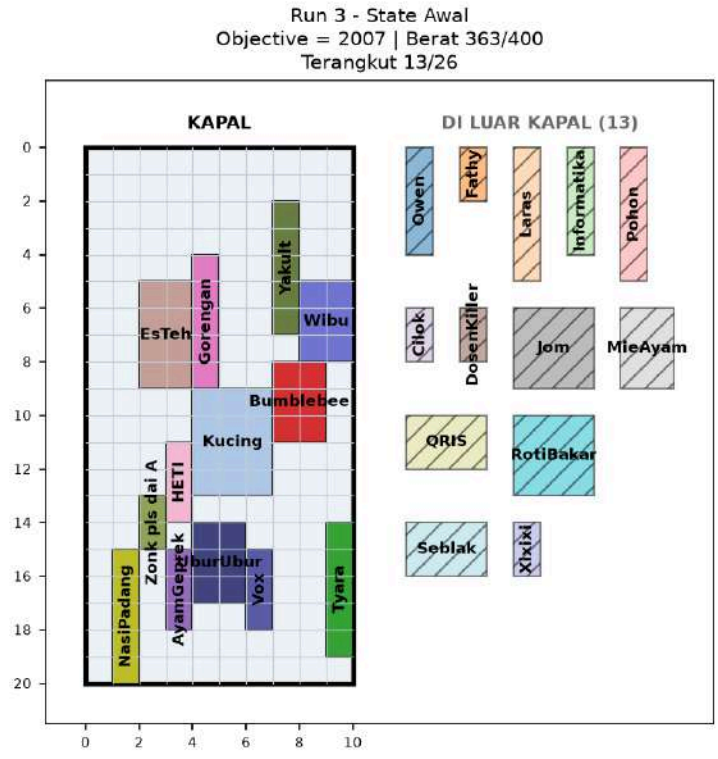
Percobaan ke-	Nilai initial objective function	Nilai final objective function	Durasi (s)	Frekuensi *'stuck'* di *local optimal*
1	1556	2175	0.2687	14
2	1873	2266	0.2884	13
3	2007	2272	0.2675	12

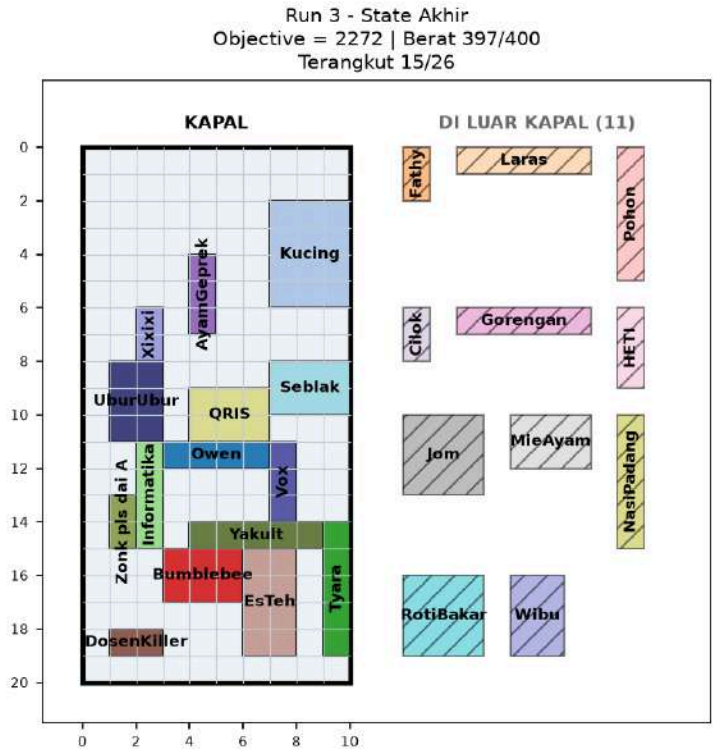
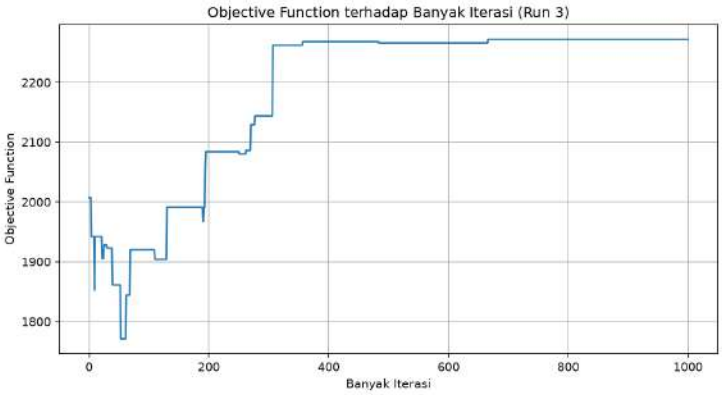
Percobaan 1	
State awal	

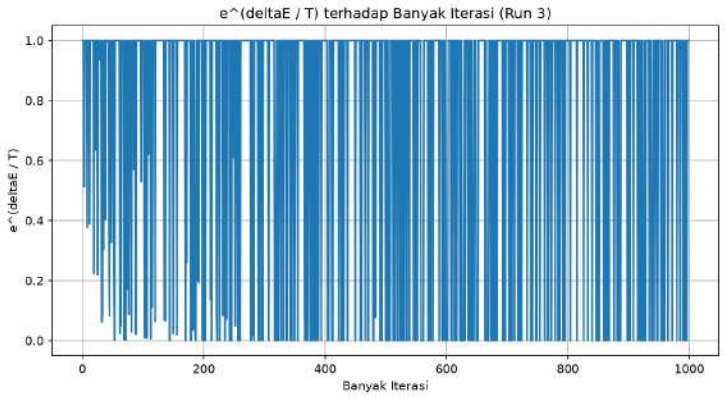
State akhir	
Plot objective function terhadap banyak iterasi	

Plot $e^{\frac{\Delta E}{T}}$ terhadap banyak iterasi yang telah dilewati	
Percobaan 2	
State awal	

State akhir	
Plot objective function terhadap banyak iterasi	

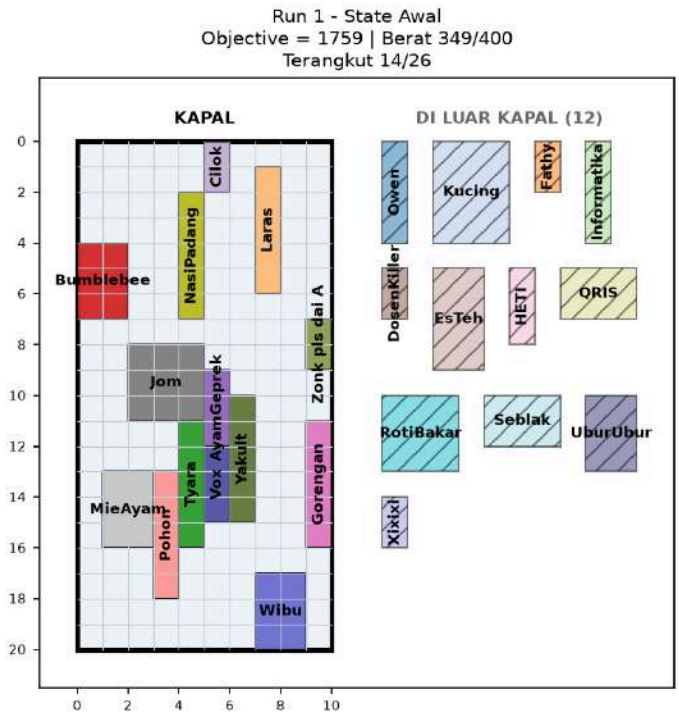
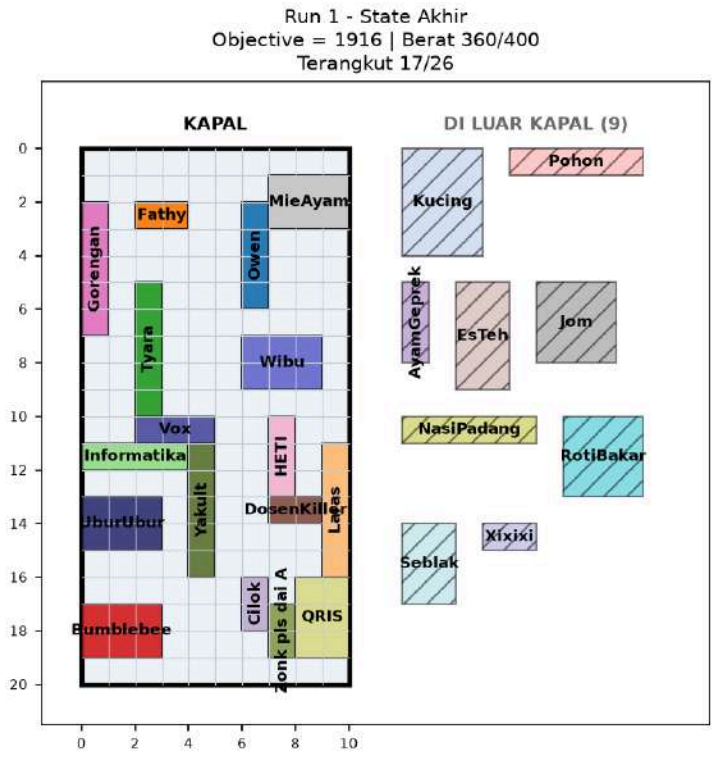
Plot $e^{\frac{\Delta E}{T}}$ terhadap banyak iterasi yang telah dilewat	
Percobaan 3	
State awal	

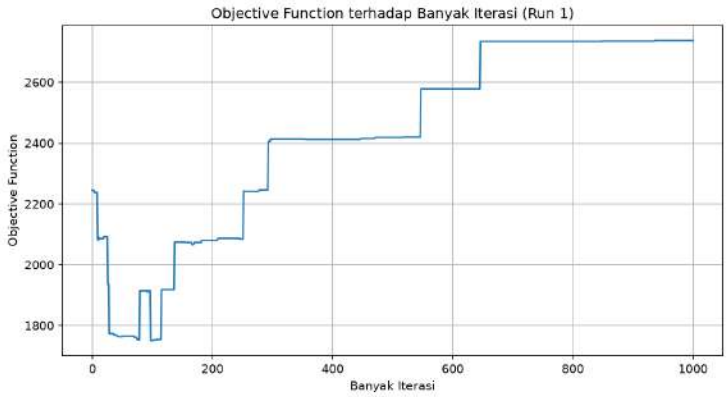
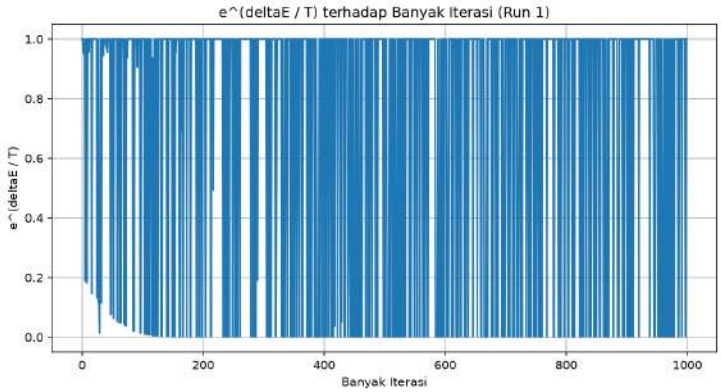
State akhir	
Plot objective function terhadap banyak iterasi	

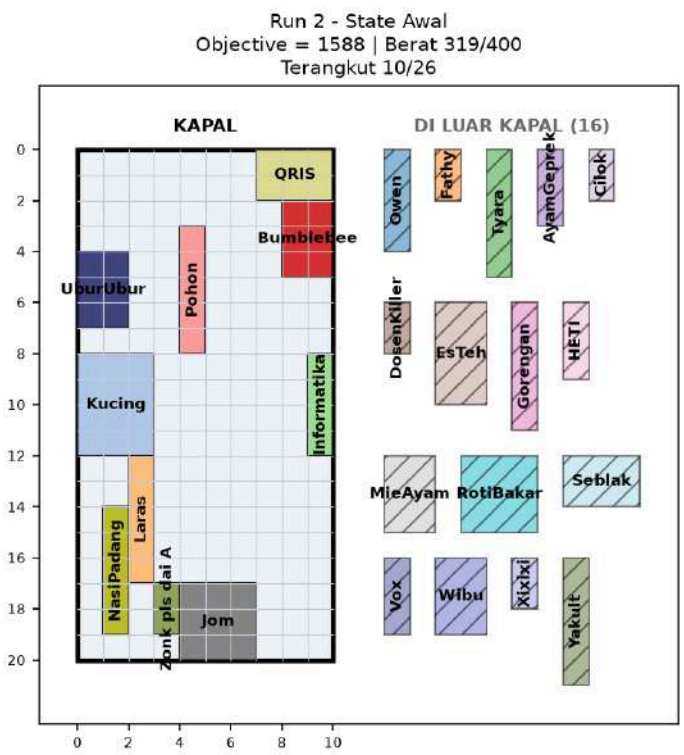
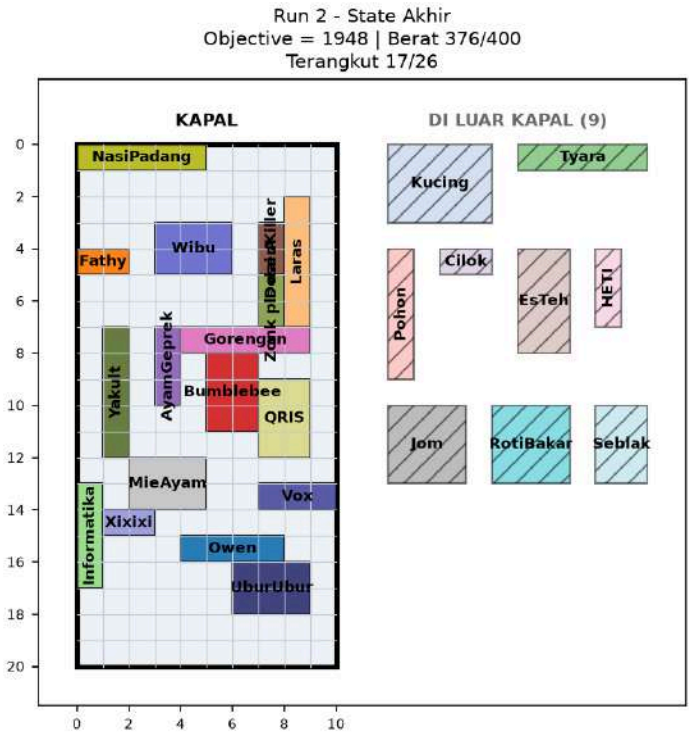
Plot $e^{\frac{\Delta E}{T}}$ terhadap banyak iterasi yang telah dilewati	

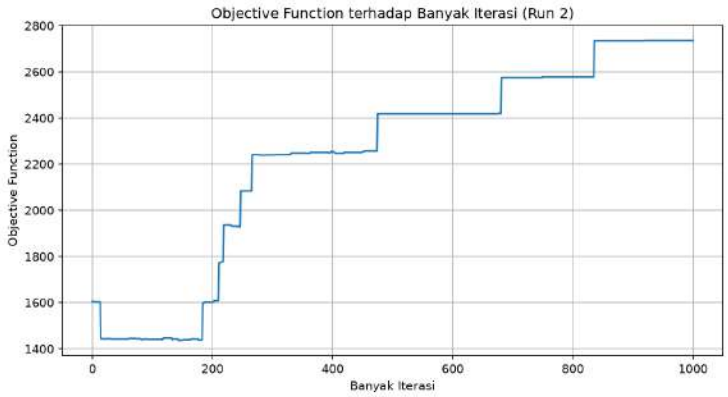
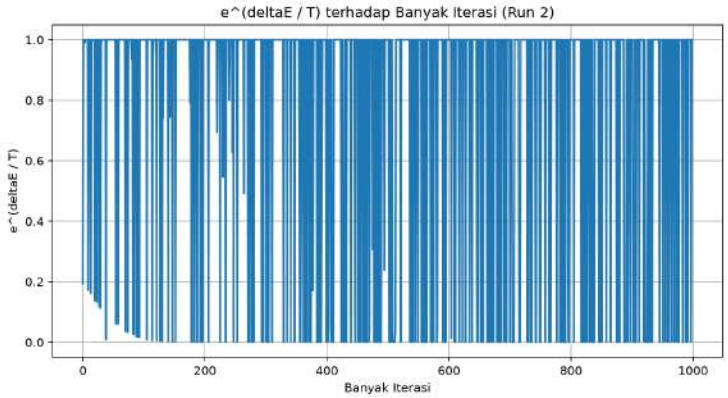
b. Percobaan dengan Objective Function ETA

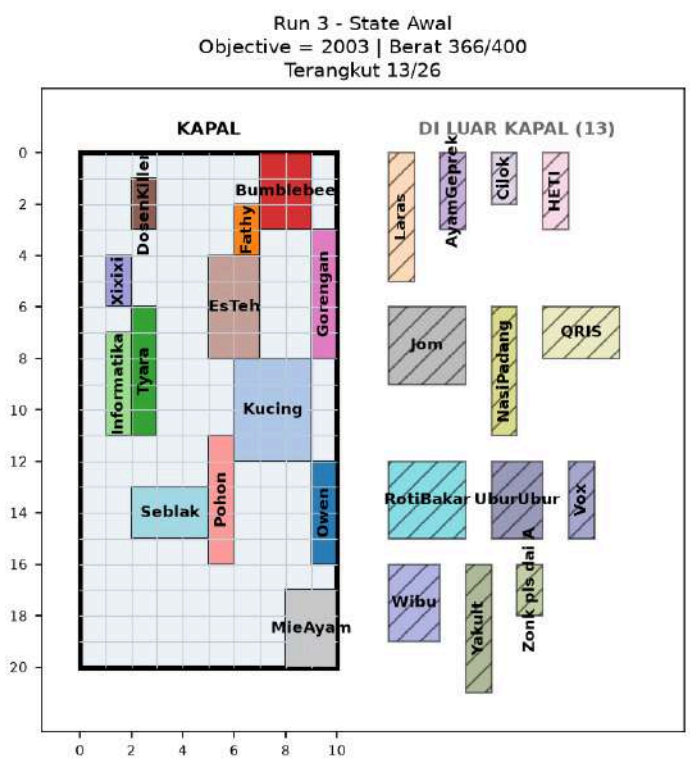
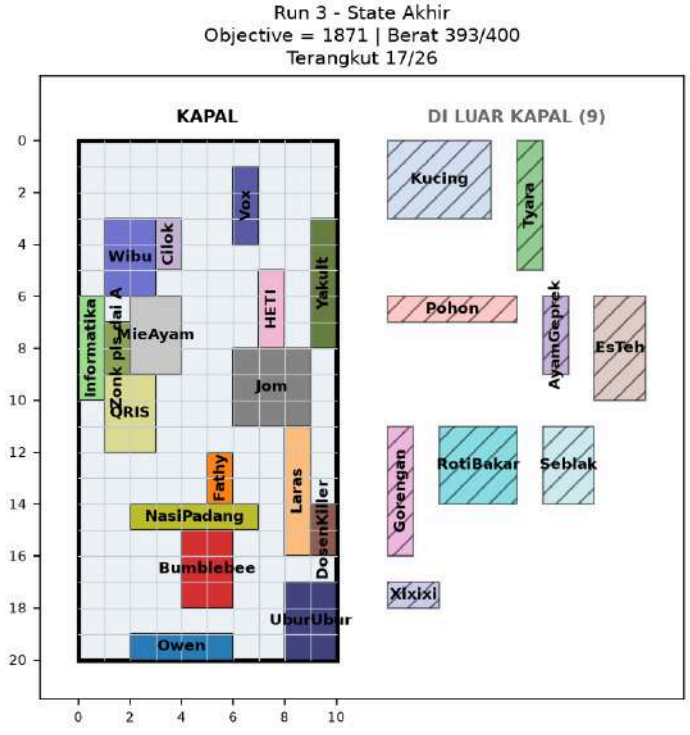
Percobaan ke-	Nilai initial objective function	Nilai final objective function	Durasi	Frekuensi *'stuck'* di *local optimal*
1	2245	2738	0.2706	18
2	1604	2736	0.2928	22
3	2070	2734	0.3026	22

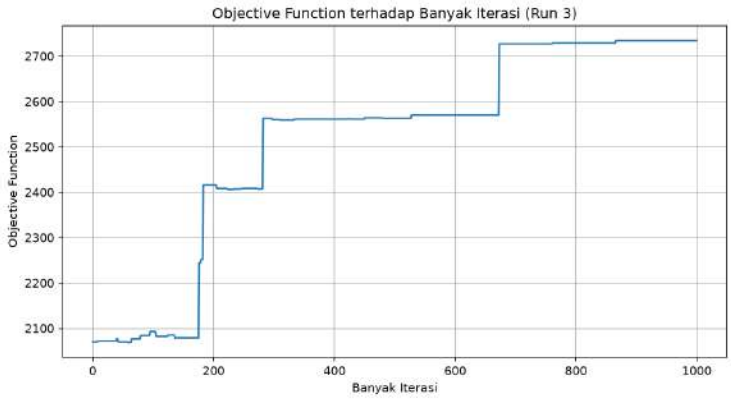
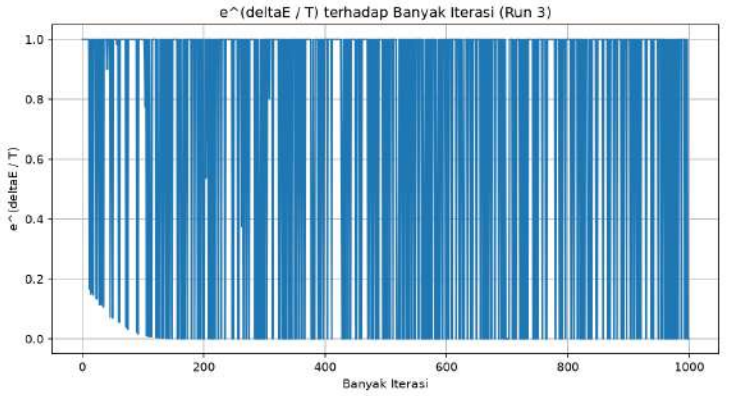
Percobaan 1	
State awal	
State akhir	

Plot objective function terhadap banyak iterasi	
Plot $e^{\frac{\Delta E}{T}}$ terhadap banyak iterasi yang telah dilewati	

Percobaan 2	
State awal	
State akhir	

Plot objective function terhadap banyak iterasi	
Plot $e^{\frac{\Delta E}{T}}$ terhadap banyak iterasi yang telah dilewati	

Percobaan 3	
State awal	
State akhir	

Plot objective function terhadap banyak iterasi	
Plot $e^{\frac{\Delta E}{T}}$ terhadap banyak iterasi yang telah dilewati	

2.7.6. **Genetic Algorithm**

Dua eksperimen dijalankan, masing-masing 3 run per konfigurasi (*seed* 1,2,3, *mutation rate* 0.15.

- Eksperimen 1 menjadikan populasi sebagai kontrol (30) dengan iterasi 50,100,200
- Eksperimen 2 menjadikan iterasi sebagai kontrol (100) dengan populasi 10,30,60

Konfigurasi populasi 30 dan iterasi 100 dipakai bersama oleh kedua eksperimen. “Awal” adalah skor individu terbaik pada populasi awal.

Setiap konfigurasi di bawah dilaporkan dengan satu tabel hasil per run (Run 1 sampai 3 memakai *seed* 1 sampai 3 dan jumlah iterasi sama dengan konfigurasinya), gambar state awal dan state akhir untuk ketiga run, dan plot *objective function* maksimum (garis tebal), serta rata-rata populasi (garis putus-putus) terhadap iterasi. Kendaran di dalam kapal digambar di geladak, kendaraan di luar kapal (bergaris miring) di kolom kanan, dan judul tiap panel memuat nilai *objective function.* Kolom “kendaraan” dan “berat” menghitung kendaraan terangkut pada state akhir dari total 26 kendaraan, dengan kapasitas berat 400. Gambar *state* dibuat oleh fungsi

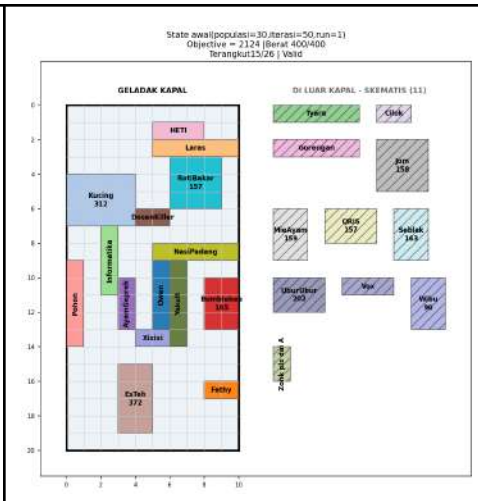
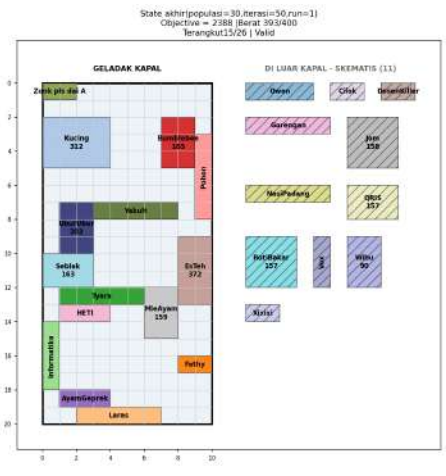
display_image pada utils.py (kendaraan di luar kapal ikut digambar di sisi kanan), sedangkan *plot objective* dibuat oleh ga_visualization.py

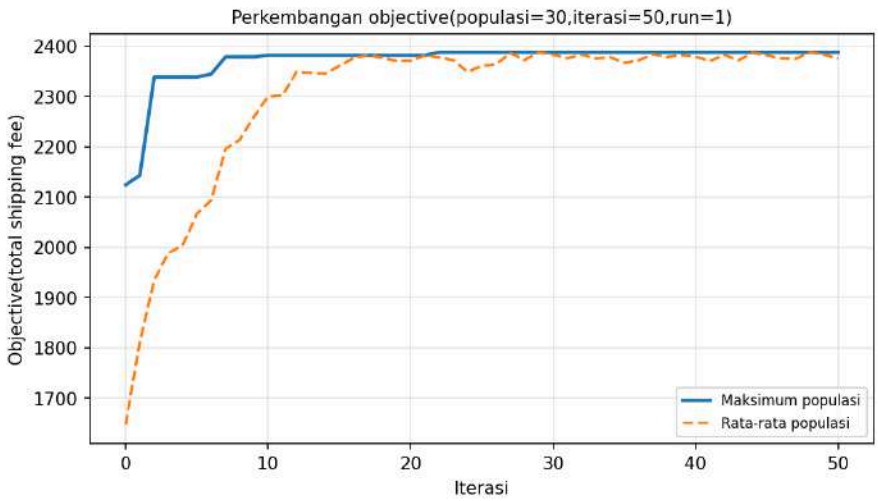
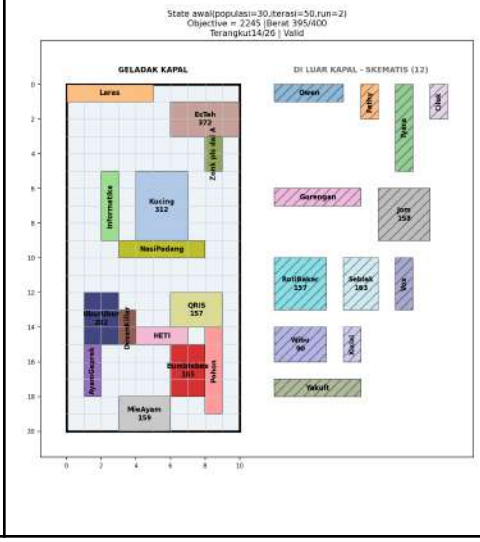
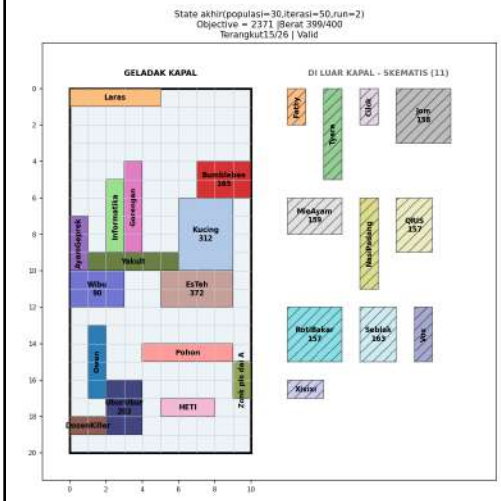
Eksperimen 1: populasi sebagai kontrol (30), iterasi 50, 100, 200

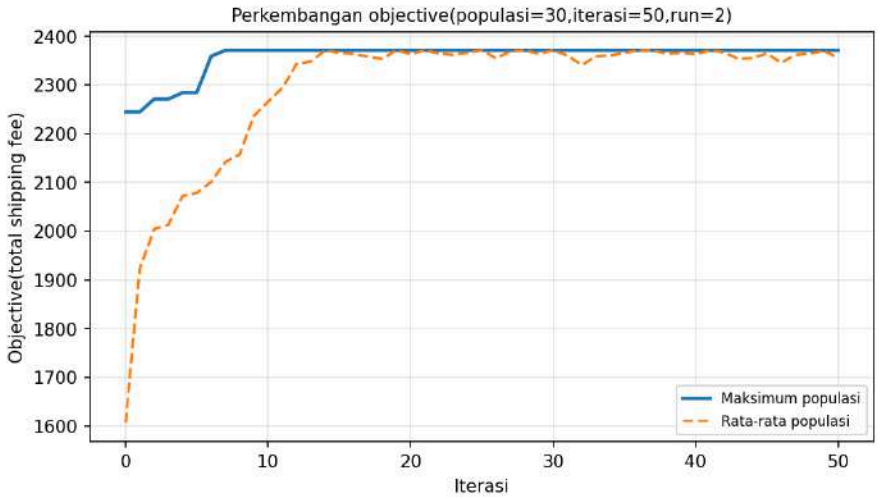
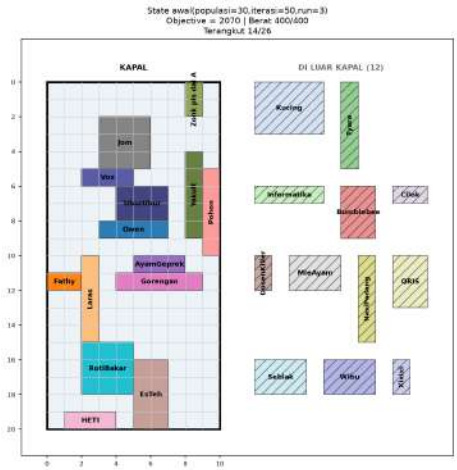
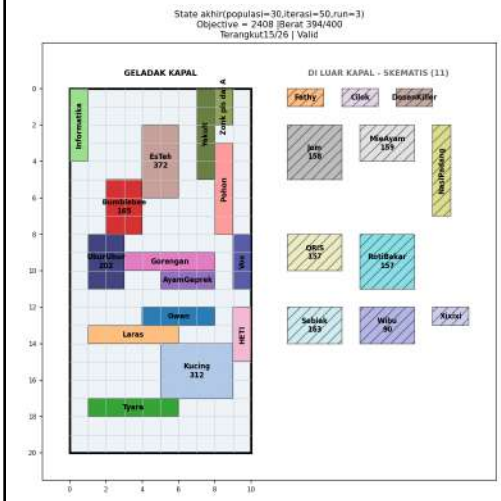
Tabel Hasil *Genetic Algorithm* 1 (populasi 30, iterasi 50)

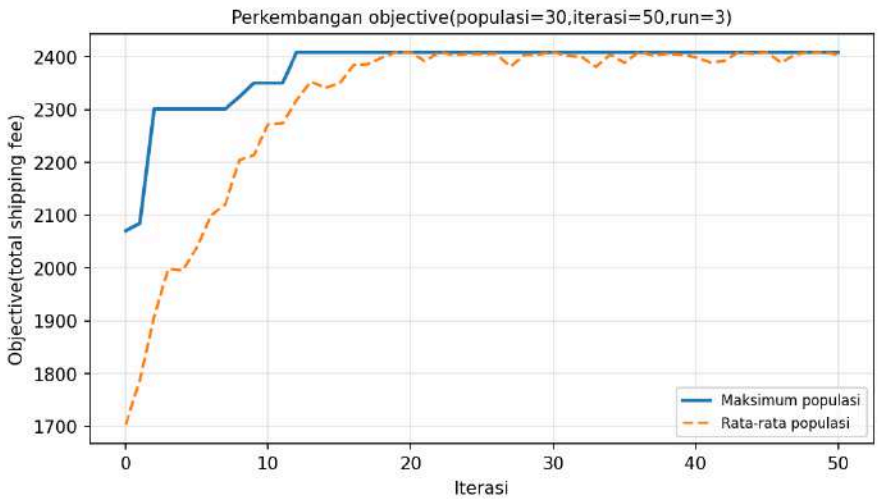
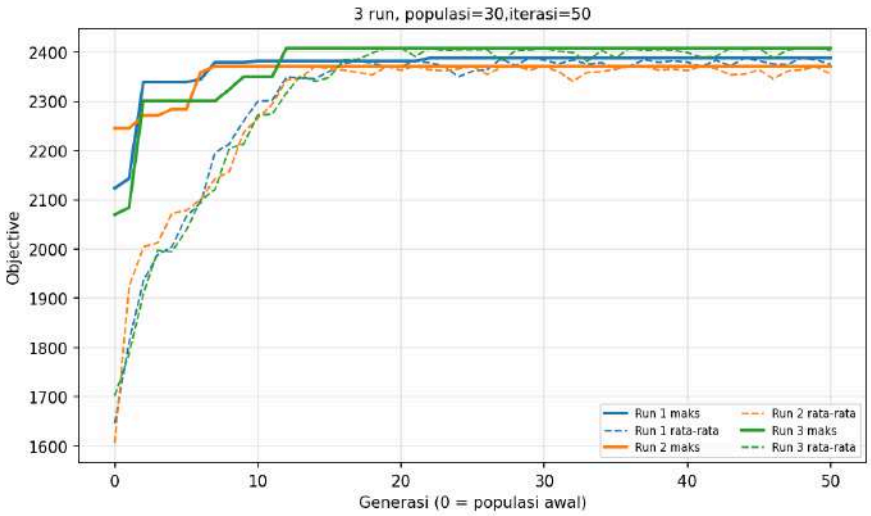
Run	Nilai initial *objective function*	Nilai final *objective function*	Kendaraan	Berat	Durasi
1	2124	2388	15	393	1,45
2	2245	2371	15	399	1,52
3	2070	2408	15	394	1,83

Tabel gambar *state* awal, *state* akhir, dan *plotting* GA (populasi 30, iterasi 50)

Percobaan 1		
State awal dan akhir		

Plot objective function terhadap banyak iterasi	
Percobaan 2	
State awal akhir	 

Plot objective function terhadap banyak iterasi	
Percobaan 3	
State awal dan akhir	 

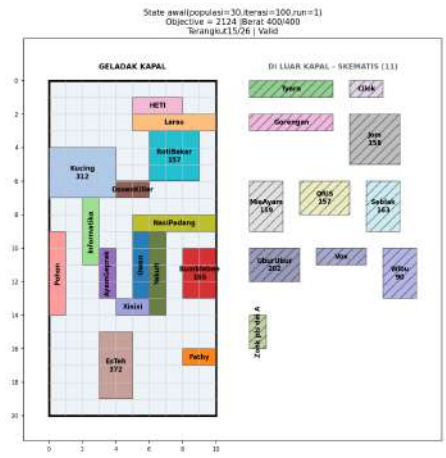
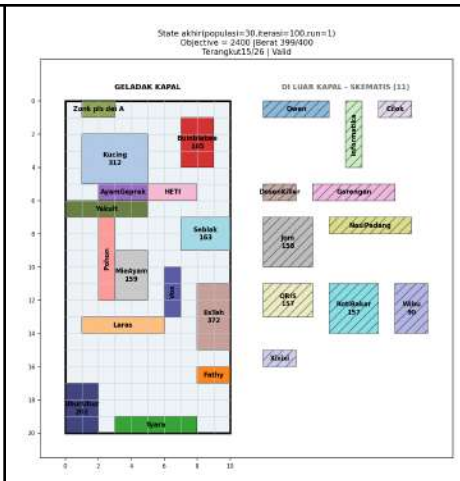
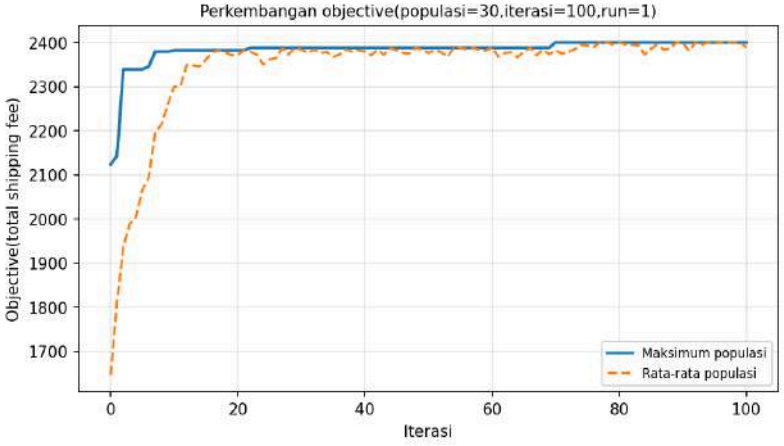
Plot objective function terhadap banyak iterasi	
Plotting Semua Percobaan	
	

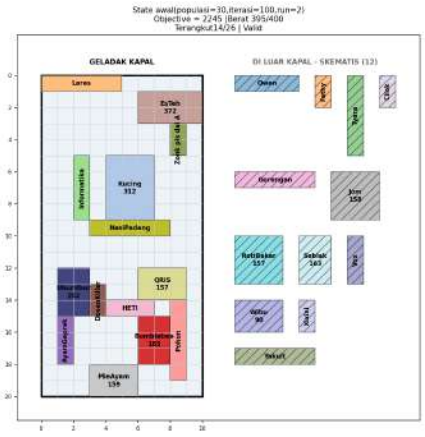
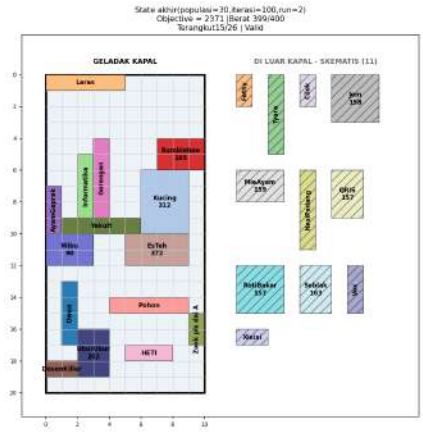
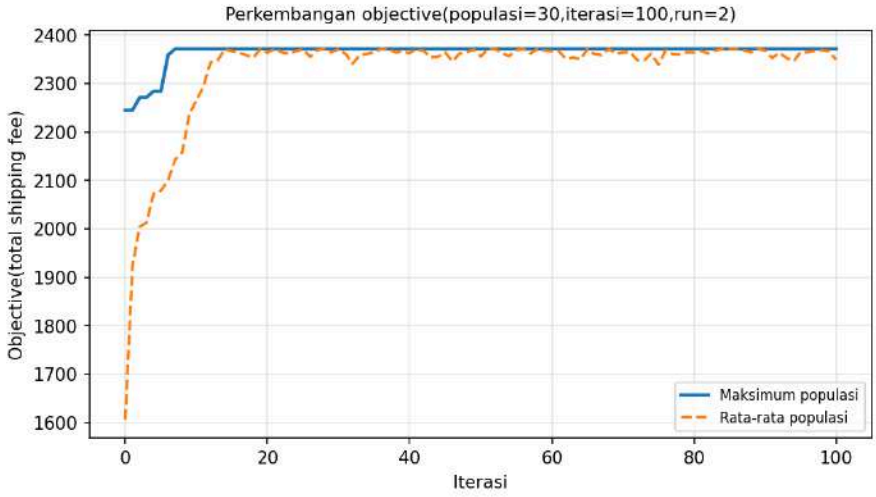
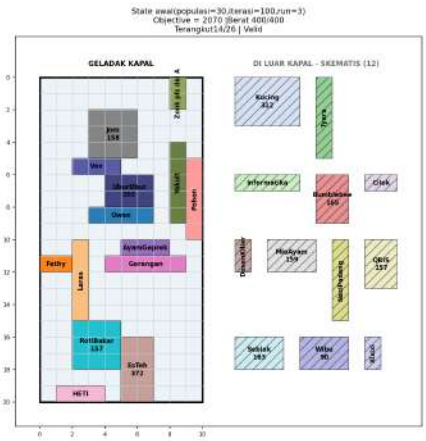
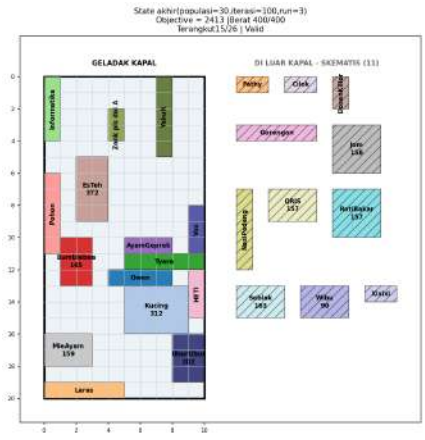
Tabel Hasil *Genetic Algorithm* 2 (populasi 30, iterasi 100)

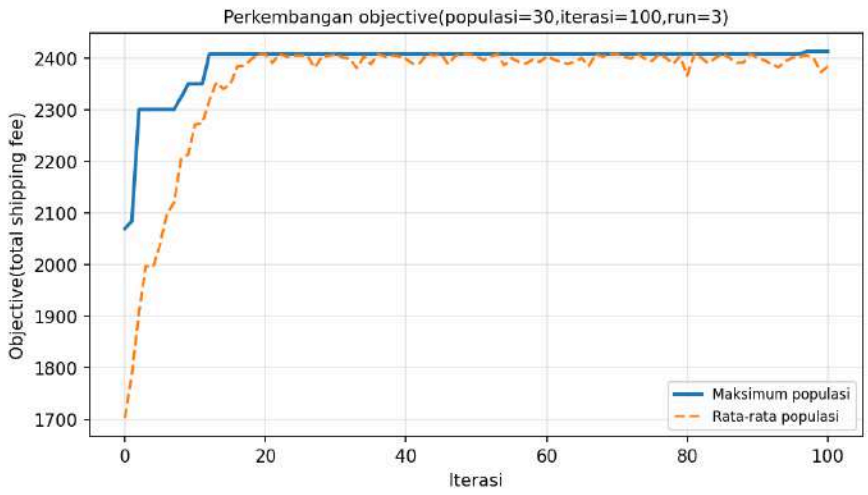
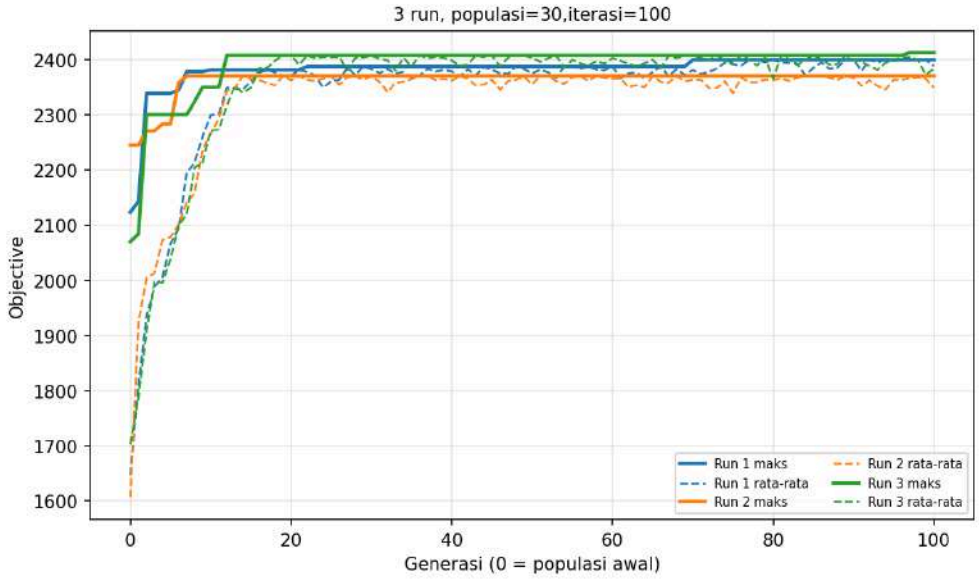
Run	Nilai initial *objective function*	Nilai final *objective function*	Kendaraan	Berat	Durasi
1	2124	2400	15	399	3,03
2	2245	2371	15	399	3,06

3	2070	2413	15	400	3,06

Tabel gambar *state* awal, *state* akhir, dan *plotting* GA (populasi 30, iterasi 100)

Percobaan 1		
State awal dan akhir		
Plot objective function terhadap banyak iterasi		
Percobaan 2		

State awal akhir		
Plot objective function terhadap banyak iterasi		
Percobaan 3		
State awal dan akhir		

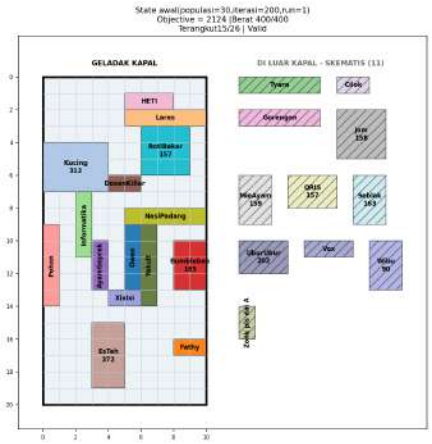
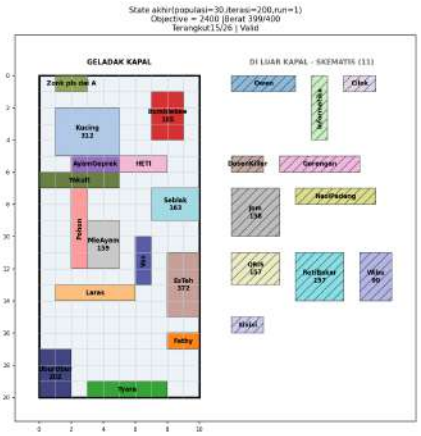
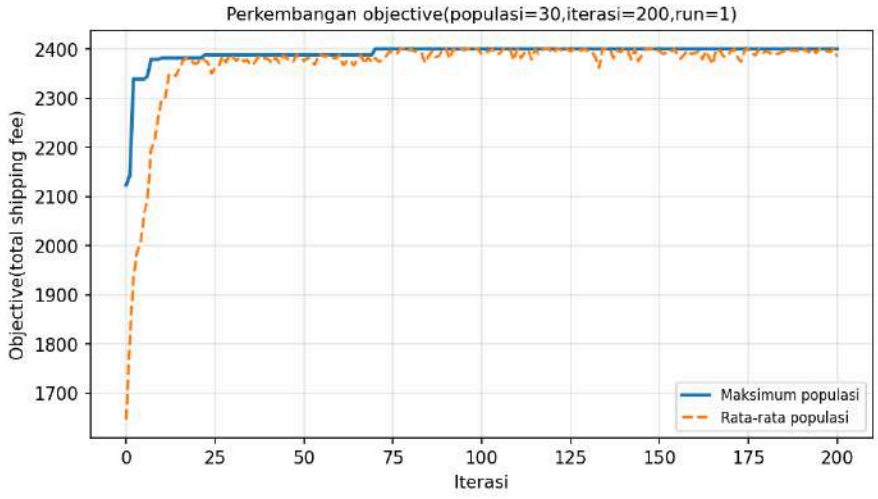
Plot objective function terhadap banyak iterasi	
Plotting Semua Percobaan	
	

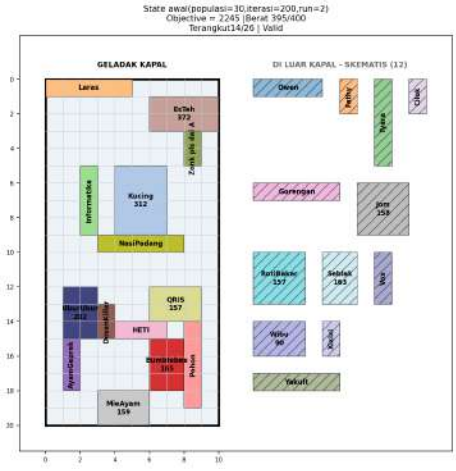
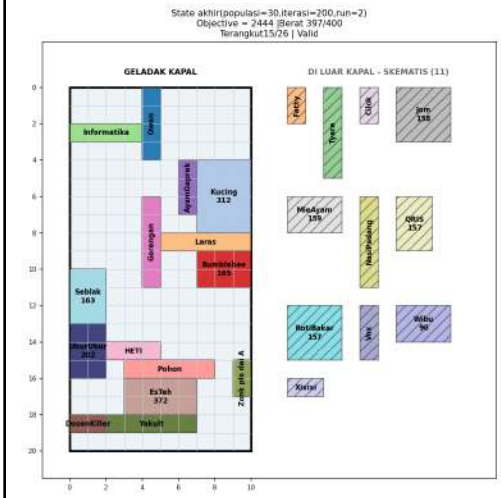
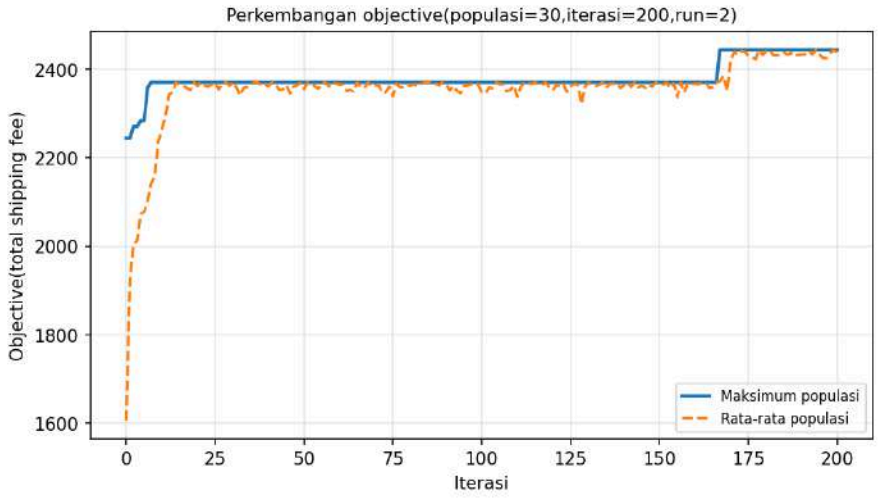
Tabel Hasil *Genetic Algorithm* 3 (populasi 30, iterasi 200)

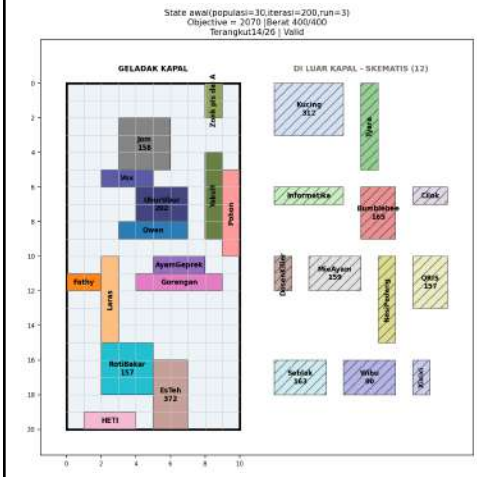
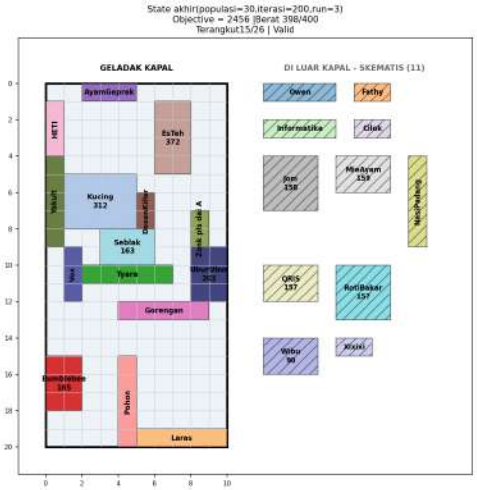
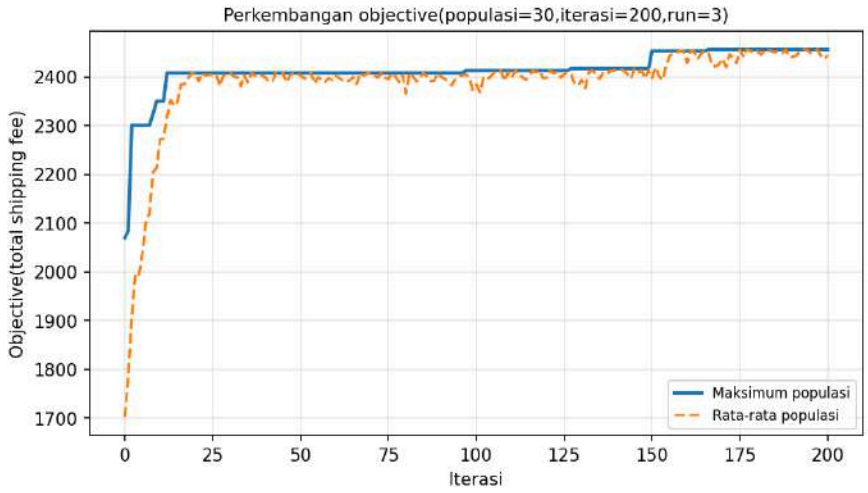
Run	Nilai initial *objective function*	Nilai final *objective function*	Kendaraan	Berat	Durasi
1	2124	2400	15	399	6,06

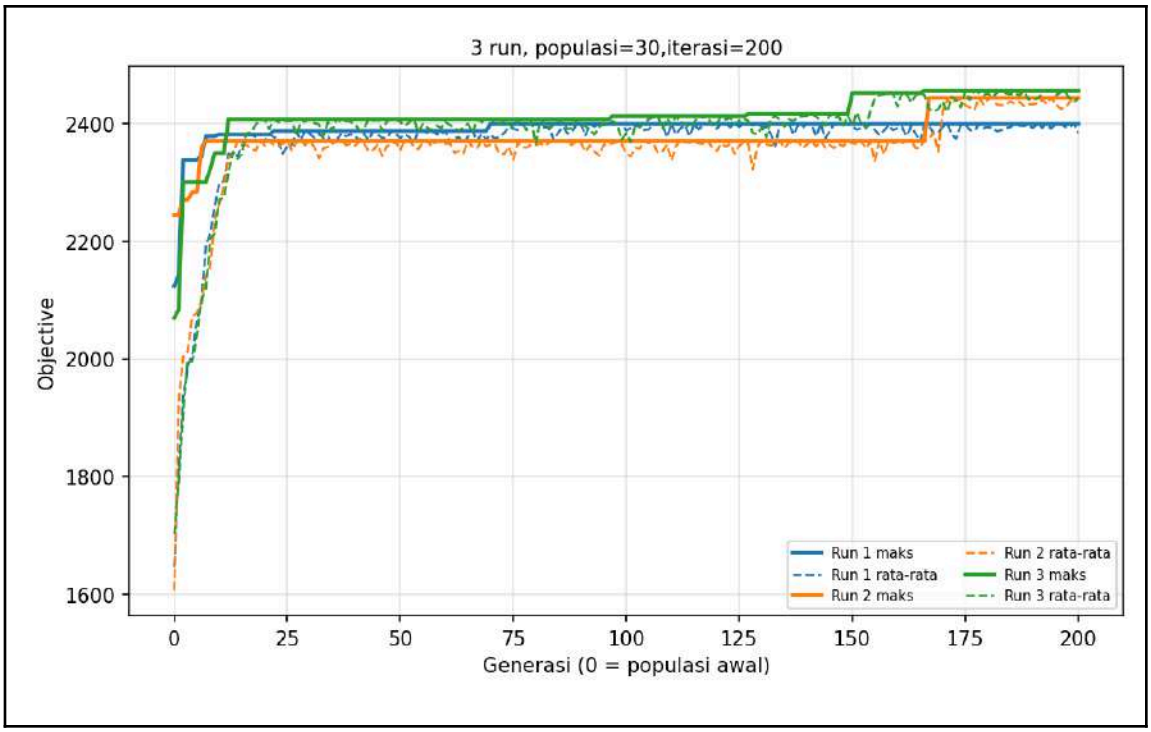
2	2245	2444	15	397	5,98
3	2070	2456	15	398	6,08

Tabel gambar *state* awal, *state* akhir, dan *plotting* GA (populasi 30, iterasi 200)

<table>
<tr><td colspan="3">Percobaan 1</td></tr>
<tr><td>*State* awal dan akhir</td><td> 
</td><td> 
</td></tr>
<tr><td>*Plot objective function* terhadap banyak iterasi</td><td colspan="2"> 
</td></tr>
<tr><td colspan="3">Percobaan 2</td></tr>
</table>

State awal		
Plot objective function terhadap banyak iterasi		
Percobaan 3		

State awal dan akhir		
Plot objective function terhadap banyak iterasi		
Plotting Semua Percobaan		



Eksperimen 2: iterasi sebagai kontrol (100), populasi 10,30,60

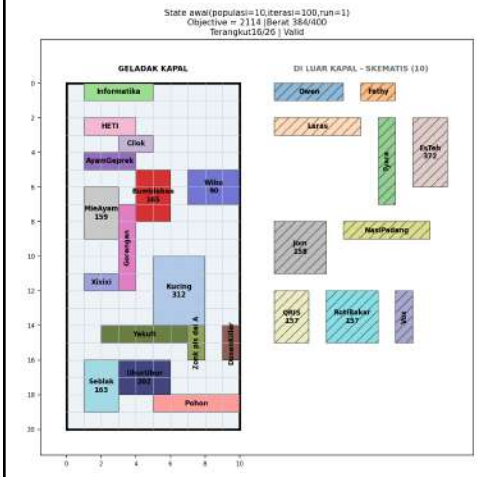
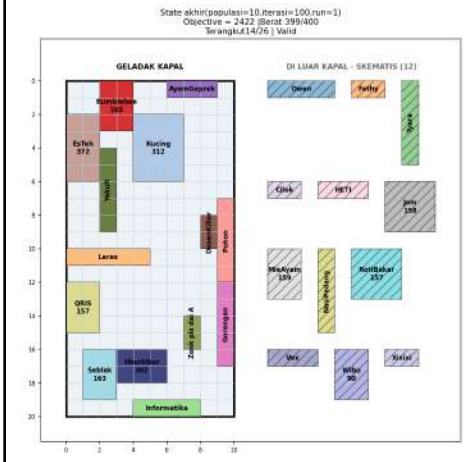
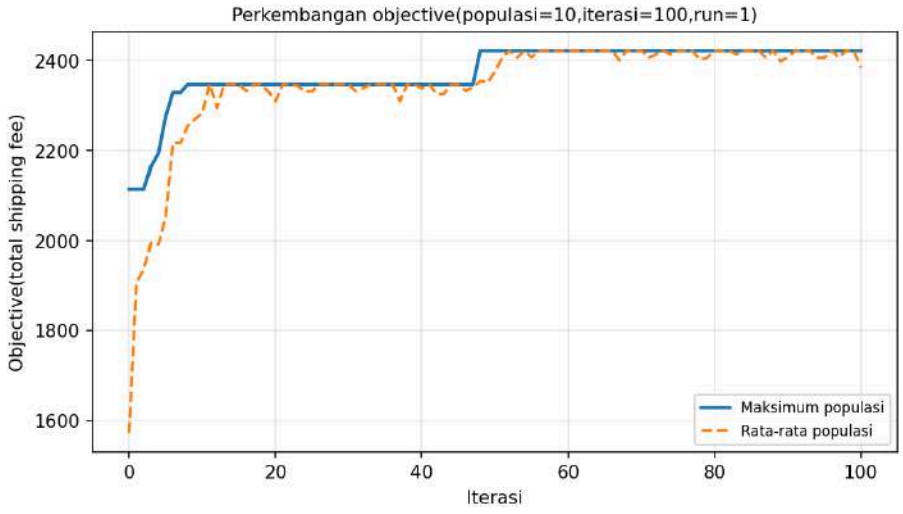
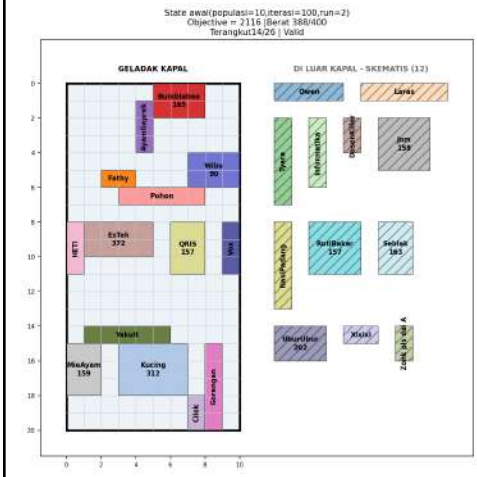
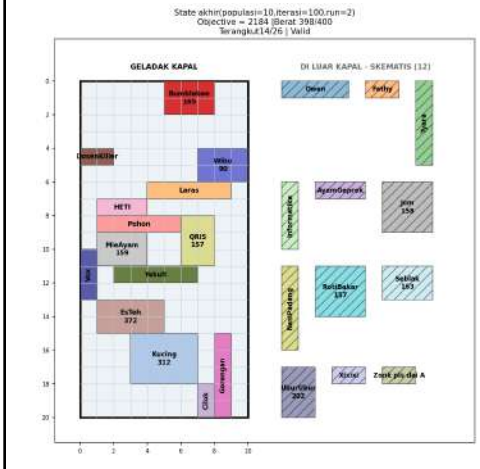
Populasi 30 dan iterasi 100 dipakai bersama oleh kedua eksperimen. (refer kalau pake daftar tabel)

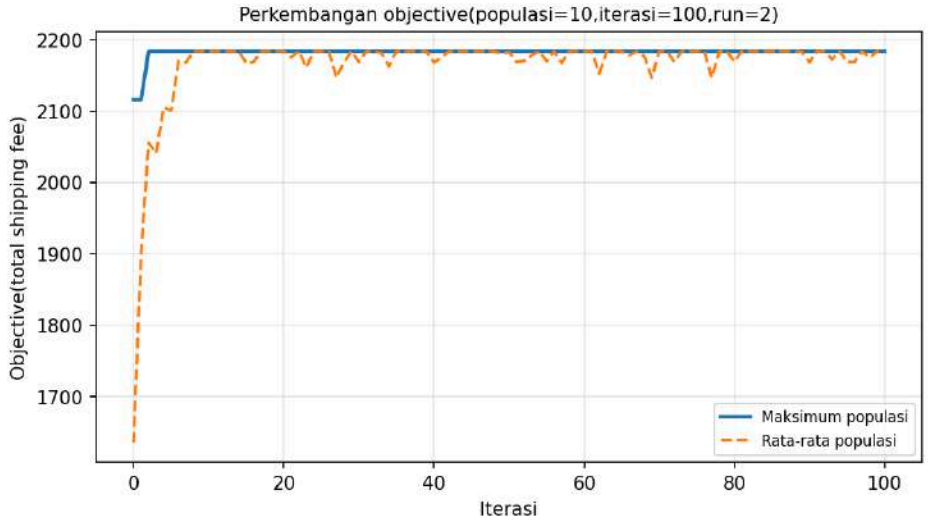
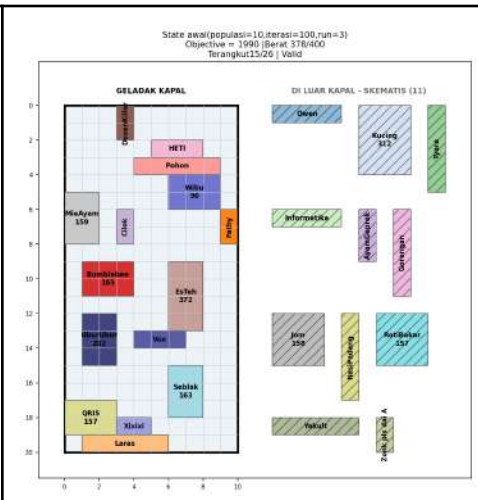
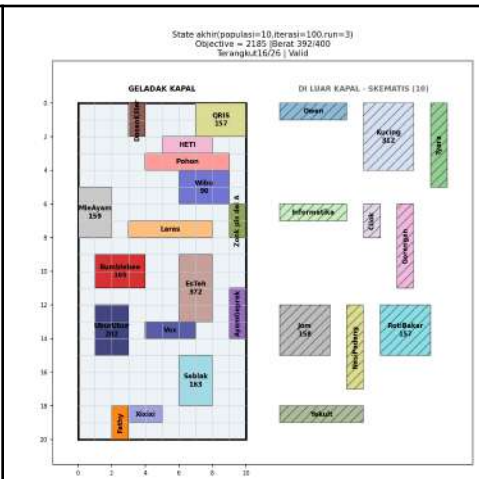
Tabel Hasil *Genetic Algorithm* 4 (populasi 10, iterasi 100)

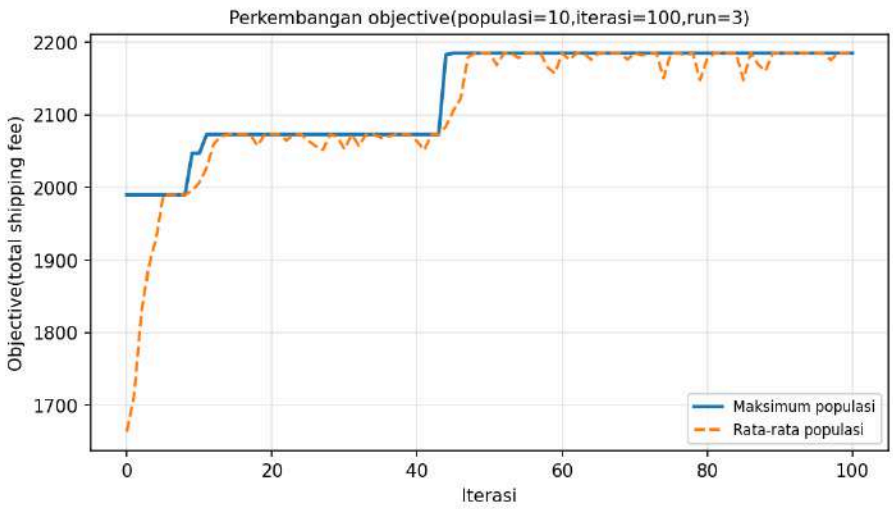
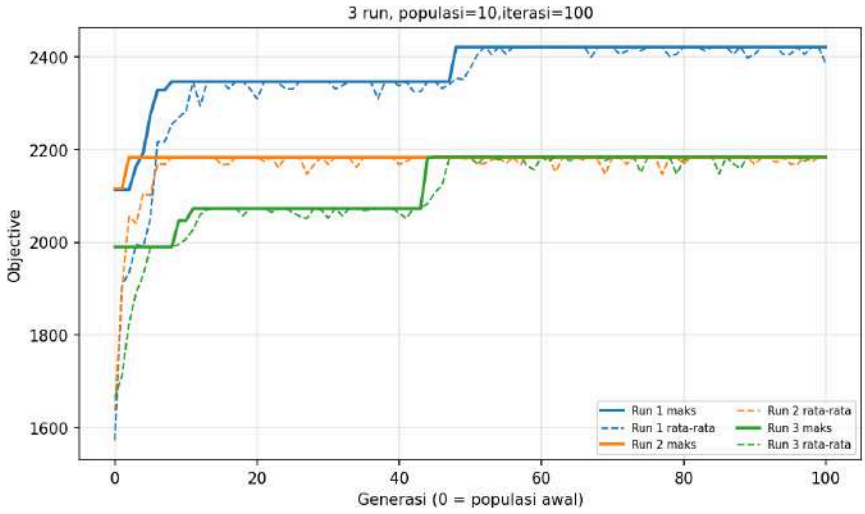
Run	Nilai initial *objective function*	Nilai final *objective function*	Kendaraan	Berat	Durasi
1	2124	2422	14	399	0,74
2	2116	2184	14	398	0,74
3	1990	2185	16	392	0,78

Tabel gambar *state* awal, *state* akhir, dan *plotting* GA (populasi 10, iterasi 100)

Percobaan 1

State awal dan akhir		
Plot objective function terhadap banyak iterasi		
Percobaan 2		
State awal akhir		

Plot objective function terhadap banyak iterasi	
Percobaan 3	
State awal dan akhir	 

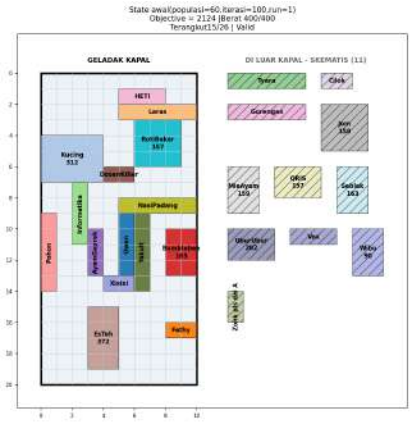
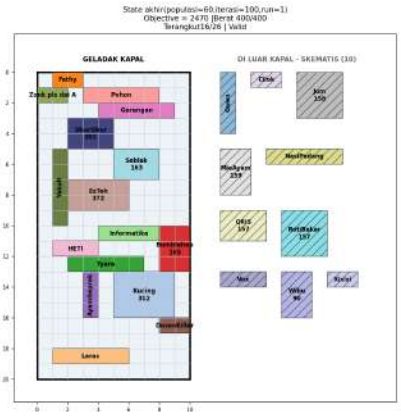
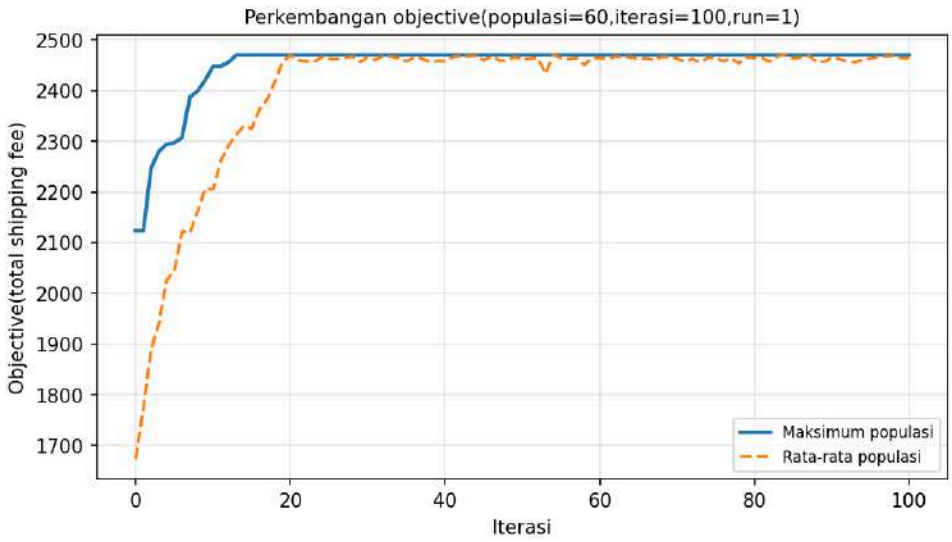
Plot objective function terhadap banyak iterasi	
Plotting Semua Percobaan	
	

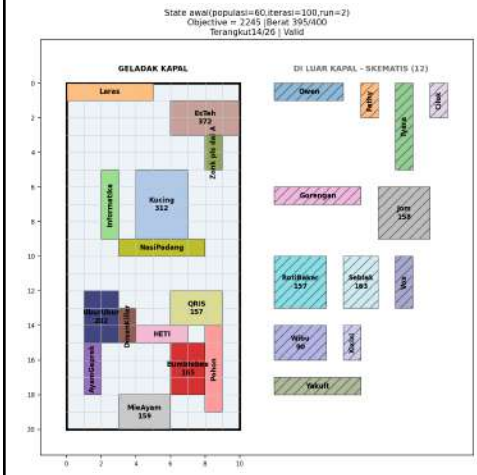
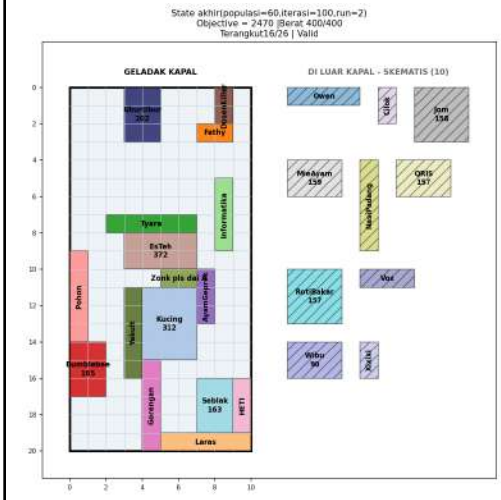
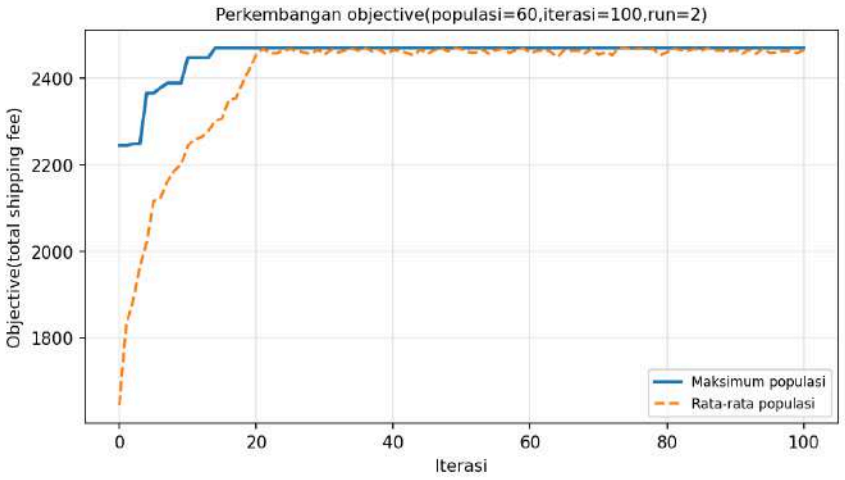
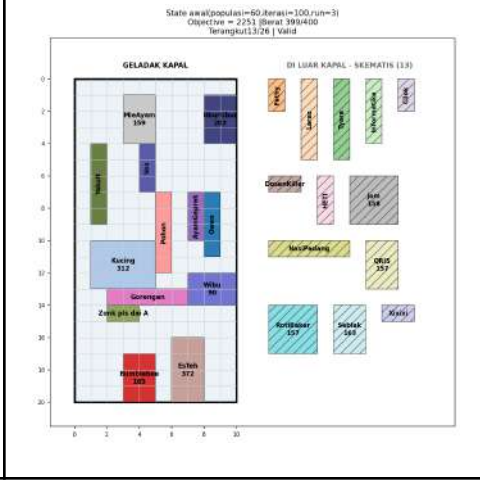
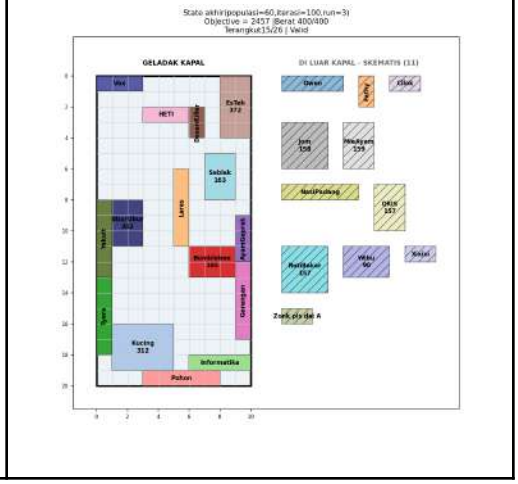
Tabel Hasil *Genetic Algorithm* 5 (populasi 60, iterasi 100)

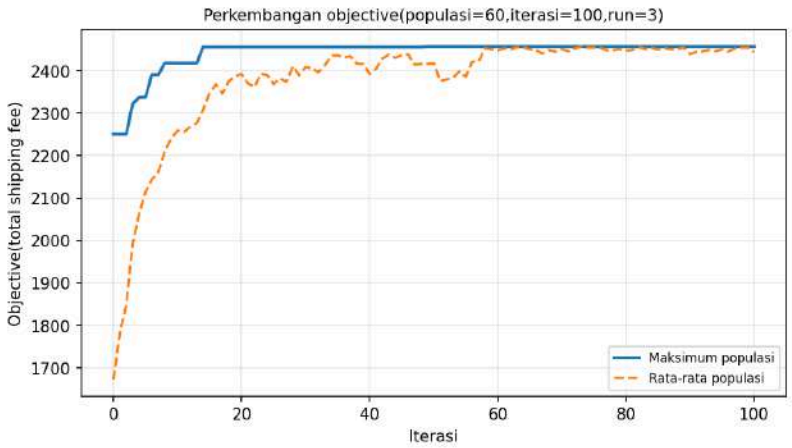
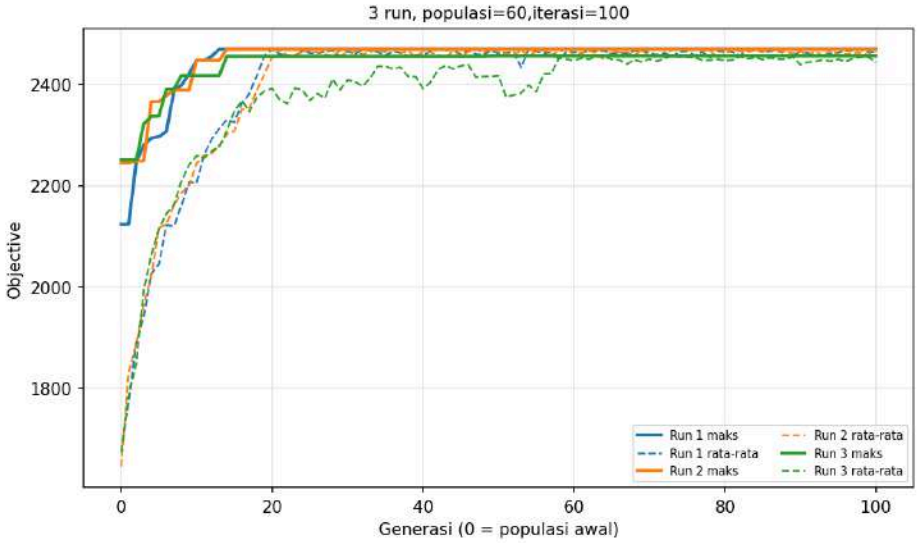
Run	Nilai initial *objective function*	Nilai final *objective function*	Kendaraan	Berat	Durasi
1	2124	2470	16	400	6,59
2	2245	2470	16	400	6,65

3	2251	2457	15	400	6,30

Tabel gambar *state* awal, *state* akhir, dan *plotting* GA (populasi 60, iterasi 100)

Percobaan 1		
State awal dan akhir		
Plot objective function terhadap banyak iterasi		
Percobaan 2		

State awal akhir		
Plot objective function terhadap banyak iterasi		
Percobaan 3		
State awal dan akhir		

Plot objective function terhadap banyak iterasi	
Plotting Semua Percobaan	
	

Ringkasan seluruh konfigurasi

Populasi	Iterasi	Rata-rata akhir	Simpangan Baku	Terbaik	Rata-rata durasi (s)
10	100	2263,7	112,0	2422	0,75
30	50	2389,0	15,1	2408	1.60
30	100	2394,7	17,6	2413	3,05
30	200	2433,3	24,1	2456	6,04

60	100	2465,7	6,1	2470	6,51

Eksperimen tambahan: *mutation rate*

Sebagai tambahan, mutation rate diubah pada populasi 30 dan 100 iterasi (3 run, seed 1,2,3). Implementasi ini tidak punya *crossover rate* karena setiap anak selalu dibentuk lewat crossover, sehingga *crossover rate* tidak diuji.

Tabel Pengaruh *Mutation Rate*

Mutation Rate	**Objective (Run1,2,3)**	**Rata-rata akhir**	**Simpangan Baku**
0,05	2387,2425,2412	2408,0	15,8
0,15	2400,2371,2413	2394,7	17,6
0,30	2453,2447,2421	2440,3	13,9

Eksperimen Tambahan *Objective Function Bonus*

Sebagai tambahan, dibuat *objective function* alternatif objective_function_bonus yang hanya memakai variabel yang sudah ada (ETA) tanpa menambah variabel baru. Fungsi ini memberikan skor 0 pada state yang melanggar kendala (kapasitas, batas geladak, atau tumpang tindih). Pada *state* yang valid, skornya dihitung dengan mengalikan jumlah kendaraan terangkut, kemudian menguranginya dengan total ETA kendaraan yang terangkut. Dengan perhitungan tersebut, setiap kendaraan menyumbang 166 dikurangi ETA-nya. Oleh karena itu, fungsi ini terutama mendorong pengangkutan lebih banyak kendaraan dan, jika jumlahnya sama, mengutamakan kendaraan dengan ETA lebih kecil.

Untuk menggunakan fungsi alternatif ini, GA cukup menerima fungsi objective melalui parameter objective, sementara proses seleksi, *crossover,* mutasi, dan repair_state tetap menggunakan mekanisme yang sama. Sebagai acuan evaluasi, optimum global dihitung menggunakan pendekatan yang sama, yaitu memilih kombinasi kendaraan dengan total berat tidak lebih dari 400 yang jumlah nilainya paling besar, dengan nilai setiap kendaraan sebesar 166 dikurangi ETA-nya. Perhitungan tersebut menghasilkan nilai optimum sebesar 3051. Selanjutnya, eksperimen dijalankan menggunakan konfigurasi yang sama, dengan tiga run untuk setiap konfigurasi menggunakan seed 1, 2, dan 3.

Populasi	Iterasi	Skor run 1,2,3	Rata-rata	Persentasi dari Optimum
10	100	2728, 2571, 2734	2677,7	87,8%
30	50,100, dan 200	3051, 2897, 3049	2999,0	98,3%
60	100	3051, 3051, 3051	3051,0	100%

BAB III
ANALISIS

3.1. Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?

Hill Climbing

Mengacu pada hasil simulasi, seluruh varian algoritma *Hill Climbing* masih sangat jauh untuk mendekati *global optima* (2470 untuk *shipping fee* dan 3051 untuk ETA) dengan skor akhir yang tertinggal dan tidak konsisten, mulai dari 1602–1939 (*Stochastic*), 1707–2162 (*Steepest Ascent)*, hingga sedikit lebih baik di angka 2259 (Sideways Move) dan 2178 (Random Restart). Hasil ini dikarenakan algoritma *Hill Climbing* tidak memperbolehkan perpindahan ke *neighbor* lebih buruk dan hanya menerima *neighbor* lebih baik (kecuali *Sideways Move* yang memperbolehkan *neighbor* dengan nilai sama), sehingga berpotensi *stuck* di *local optima*. Selain itu, pencariannya sangat bergantung pada posisi *initial state* di awal.

Simulated Annealing

Pada *objective shipping fee*, skor akhir SA adalah 2175, 2266, dan 2272, dengan rata-rata 2238. Pada *objective* ETA, skor akhirnya 2738, 2736, dan 2734, dengan rata-rata 2736. Pada grafik terlihat SA sempat pindah ke *neighbor* yang lebih buruk lebih dulu, lalu naik lagi. Hal ini membuat SA bisa keluar dari *local optima* yang menjebak *hill climbing*. Meski demikian, tidak ada *run* yang mencapai *global optima* (2470 untuk *shipping fee* dan 3051 untuk ETA) dan hanya mendekati walau sudah sangat dekat, yaitu 91% dan 90%. Skor akhir tiap *run* juga berbeda-beda karena *initial state*, pemilihan *neighbor*, dan keputusan menerima langkah buruk semuanya acak. Selain itu, suhu terus turun tiap iterasi. Akibatnya, SA cepat berhenti menerima langkah yang lebih buruk dan akhirnya *stuck* di *local optima* sebelum sampai ke *global optima*.

Genetic Algorithm

Untuk objective shipping fee, nilai optimum globalnya adalah 2470. Nilai ini dihitung dengan mencari kombinasi kendaraan yang beratnya tidak lebih dari 400 dan total fee-nya paling besar, karena satu-satunya batasan yang menentukan kendaraan mana yang boleh diangkut adalah berat maksimum 400, dan GA memang menemukan susunan valid dengan skor 2470. Pada 100 iterasi, rata-rata hasil GA mencapai 91,6% dari optimum untuk populasi 10, 97,0% untuk populasi 30, dan 99,8% untuk populasi 60. Hasilnya makin dekat ke optimum pada populasi

yang lebih besar karena variasi susunannya lebih banyak, sehingga GA lebih jarang berhenti di *local optimum.* Berat kendaraan pada state akhir juga selalu 392 sampai 400, artinya skor terutama ditentukan oleh kendaraan mana yang dipilih, bukan oleh susunannya di geladak.

3.2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?

Urutan hasil dari yang terbaik adalah GA, SA, *Sideways Move*, *Steepest Ascent*, *Random Restart*, lalu *Stochastic*. Di antara *hill climbing*, *Sideways Move* paling baik karena bisa terus jalan di area datar. *Steepest Ascent* sedikit di atas *Random Restart*. *Stochastic* paling rendah karena hanya mencuplik 10 *neighbor* tiap iterasi, jadi sering berhenti sebelum sempat naik. *Random Restart* memakai *Stochastic* di tiap *restart*, jadi hasilnya ikut terbatas. SA lebih baik dari semua *hill climbing* karena boleh turun dulu ke *neighbor* yang lebih buruk sesuai dengan probabilitas dan temperatur.

Sementara itu, asil GA terbaik ada pada populasi 60 dengan 100 iterasi, yaitu rata-rata 2465,7 dan skor tertinggi 2470 yang sama dengan optimum. Populasi 10 hanya mencapai rata-rata 2263,7. Berbeda dengan algoritma yang memperbaiki satu *state* saja, GA menyimpan banyak state sekaligus, hingga populasinya cukup besar, sehingga GA lebih mudah keluar dari local optimum.

3.3. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?

Durasi SA berada di sekitar 0,27 sampai 0,30 detik di semua *run*, baik untuk *shipping fee* maupun ETA. Hal ini karena SA selalu menjalankan 1000 iterasi dengan satu evaluasi *neighbor* per iterasi. *Stochastic Hill-Climbing* lebih cepat (0,004 sampai 0,008 detik) karena cepat berhenti di *local optima*. *Steepest Ascent* (0,26 sampai 1,70 detik), *Sideways Move* (0,35 sampai 1,14 detik), dan *Random Restart* (0,72 sampai 1,01 detik) lebih lama dari SA. Sementara itu, durasi GA naik hampir sebanding dengan banyaknya individu yang dievaluasi (populasi dikali iterasi), kira-kira 0,8 sampai 1,1 milidetik per individu. Populasi 10 dengan 100 iterasi membutuhkan 0,75 detik, populasi 30 dengan 50, 100, dan 200 iterasi membutuhkan 1,60, 3,05, dan 6,04 detik, dan populasi 60 dengan 100 iterasi membutuhkan 6,51 detik.

3.4. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?

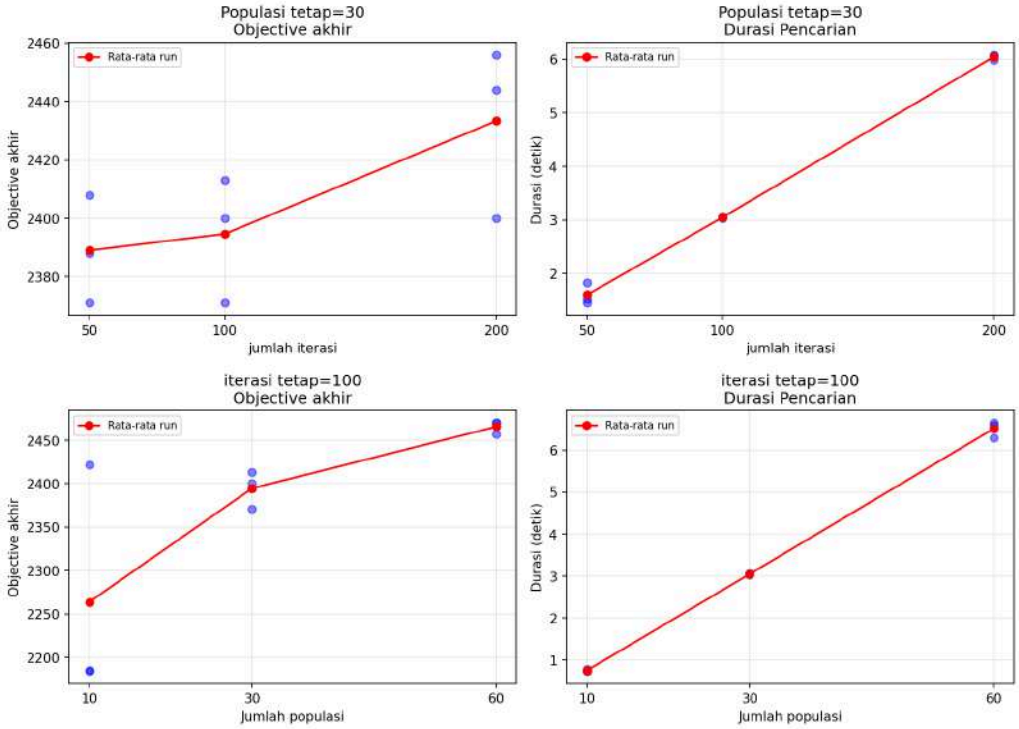
Hill climbing menghasilkan *output* yang tidak konsisten. Selisih hasil terbaik dan terburuk dalam tiga *run* adalah 455 untuk *Steepest Ascent*, 337 untuk *Stochastic*, 270 untuk *Random Restart*, dan 173 untuk *Sideways Move*. Hasilnya sangat tergantung *state* awal. Sementara itu, *Simulated Annealing* menghasilkan *output* yang lebih konsisten. Skor awalnya dapat berbeda jauh (1556 sampai 2007), namun skor akhirnya cukup dekat (2175 sampai 2272, selisih 97), dan pada *objective function* ETA hampir sama (2734 sampai 2738). GA menghasilkan *output* paling konsisten pada populasi 60, dengan selisih 6,1 dan semua *run* di antara 2457 dan 2470.

3.5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

Dari hasil *Genetic Algorithm yang diatas,* bisa disimpulkan bahwa ukuran populasi paling berpengaruh. Pada 100 iterasi, rata-rata akhir naik dari 2263,7 (populasi 10) ke 2394,7 (populasi 30) dan 2465,7 (populasi 60). Simpangan bakunya turun dari 112,0 ke 17,6 dan 6,1, tetapi durasinya naik dari 0,75 ke 3,05 dan 6,51 detik. Sedangkan, penambahan iterasi pengaruhnya lebih kecil. Pada populasi 30, rata-rata akhir naik dari 2389,0 (50 iterasi) ke 2394,7 (100 iterasi), lalu 2433,3 (200 iterasi). Dari grafik, skor naik cepat pada sekitar 20 generasi pertama lalu hampir datar. Kenaikan pada 200 iterasi terutama berasal dari run 2 yang naik dari 2371 ke 2444. Pada objective_function_bonus, skor populasi 30 bahkan sama pada iterasi 50, 100, dan 200 iterasi.

Dengan durasi komputasi yang hampir sama, populasi 60 dengan 100 iterasi (2465,7) lebih naik daripada populasi 30 dengan 200 iterasi (2433,3). Jadi, data ini menambah populasi lebih berguna daripada menambah iterasi.

Untuk *mutation rate*, rata-rata akhirnya 2408,0 pada 0,05, 2394,7 pada 0,15 dan 2440,3 pada 0,30. Karena hanya tiga nilai dan tiga *run,* disimpulkan bahwa menaikkan *mutation rate* sampai 0,30 tidak membuat hasil lebih buruk.


Populasi tetap=30
Objective akhir
Rata-rata run
Objective akhir
jumlah iterasi
Populasi tetap=30
Durasi Pencarian
Rata-rata run
Durasi (detik)
jumlah iterasi
iterasi tetap=100
Objective akhir
Rata-rata run
Objective akhir
Jumlah populasi
iterasi tetap=100
Durasi Pencarian
Rata-rata run
Durasi (detik)
Jumlah populasi

BAB IV
KESIMPULAN

4.1. Kesimpulan

Urutan algoritma berdasarkan rata-rata durasi adalah *Stochastic Hill-Climbing*, *Simulated Annealing*, *Random Restart*, *Sideways Move*, *Steepest Ascent*, lalu *Genetic Algorithm*. Sementara itu, urutan algoritma yang menghasilkan nilai paling baik adalah *Genetic Algorithm*, *Simulated Annealing*, *Sideways Move Hill Climbing*, *Steepest Ascent Hill Climbing*, *Random Restart Hill Climbing*, dan *Stochastic Hill Climbing*. *Hill climbing* lebih cepat dibandingkan algoritma-algoritma lain, tapi rawan *stuck* di *local optima*. SA bisa keluar dari *local optima* tapi tetap dapat berakhir di bawah *global optim*a. *Global optima* untuk *shipping fee* adalah 2470, dan hanya *Genetic Algorithm* yang pernah mencapainya, yaitu dua dari tiga *run* pada populasi 60.

4.1. Saran

Untuk hasil paling optimal, disarankan untuk menggunakan algoritma *Genetic Algorithm* dengan ukuran populasi sebesar mungkin. Waktu yang diperlukan lebih lama, tapi selisih kebutuhan waktu tidak signifikan. Pada percobaan ini, menambah populasi di *Genetic Algorithm* lebih efektif daripada menambah iterasi. Jika ingin tetap lebih cepat namun tidak *stuck* di *local optima*, algoritma *Simulated Annealing* dapat digunakan sebagai alternatif dengan suhu awal yang lebih besar dan pendinginan yang lebih lambat.

REFERENSI

Slide Perkuliahan IF3070 Dasar Inteligensi Artifisial

Algoritma *local search*. (n.d.). In *Wikipedia*. Retrieved October 30, 2025, from https://en.wikipedia.org/wiki/Local_search_(optimization)

GeeksforGeeks. (2025, August 1). Introduction to Hill Climbing in Artificial Intelligence. Retrieved October 30, 2025, from https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/

GeeksforGeeks. (2025, July 23). What is simulated annealing? GeeksforGeeks. Retrieved from https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/

Kotkar, S. (2020, September 20). Simulated Annealing. *Medium*. https://medium.com/analytics-vidhya/simulated-annealing-869e171e763c

Objective function. (n.d.). In *SkillPlus*. Retrieved October 30, 2025, from https://skillplus.web.id/machine-learning-objective-function/

Python Software Foundation. (2026). *random — Generate pseudo-random numbers*. Python 3.12.5 Documentation. Retrieved October 22, 2026, from https://docs.python.org/3/library/random.html

LAMPIRAN

- Link Repository Kelompok 12: https://github.com/larashtm/IF3070_G12_Yakin
- Lampiran Pernyataan Penggunaan AI:

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami*:

Nama/NIM : Fathimah Nurhumaida Ramadhani/18223052, Laras Hati Mahendra/18223118, Tyara Penelope Lumban Gaol/18224122
Fakultas/Sekolah : STEI-K
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Intelligensi Artifisial
Judul Tugas : Optimasi Penataan Muatan Kapal Kargo Menggunakan Algoritma Local Search
Menyatakan bahwa saya/kami* **menggunakan**/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	**Y**a/Tidak*	0
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/**Tidak***	0
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	**Ya**/Tidak*	1
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya/**Tidak***	0
Lainnya, Jelaskan:	Ya/Tidak*	-

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Tyara Penelope Lumban Gaol

18224122